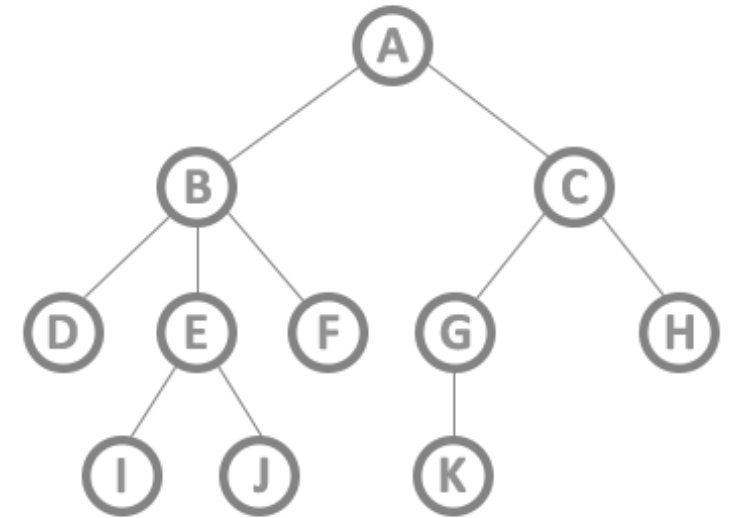
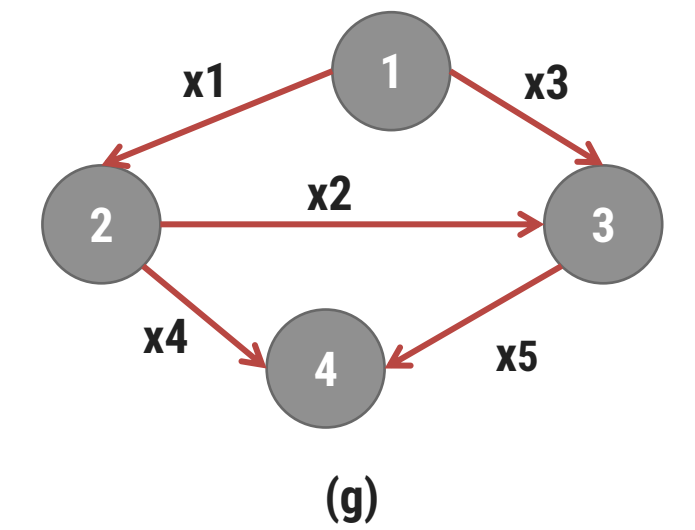
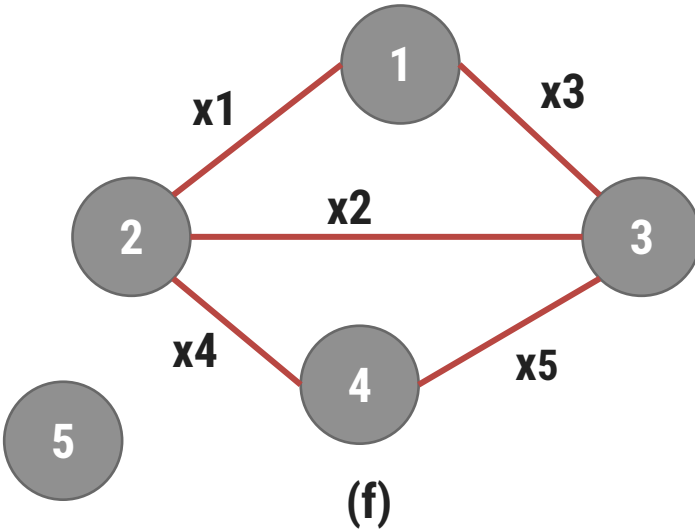
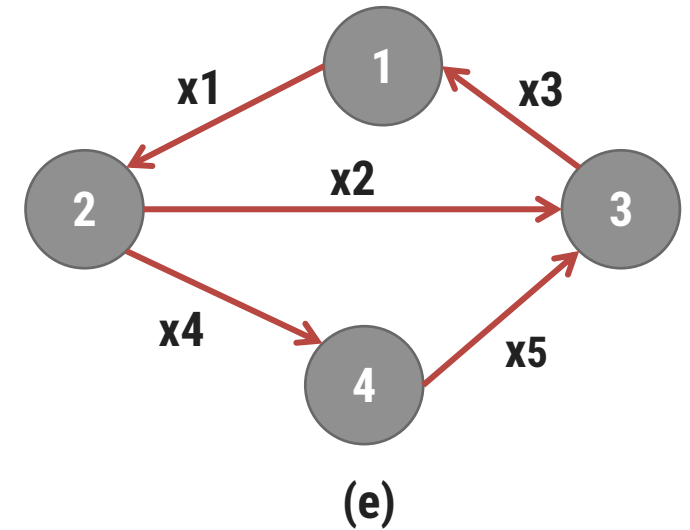
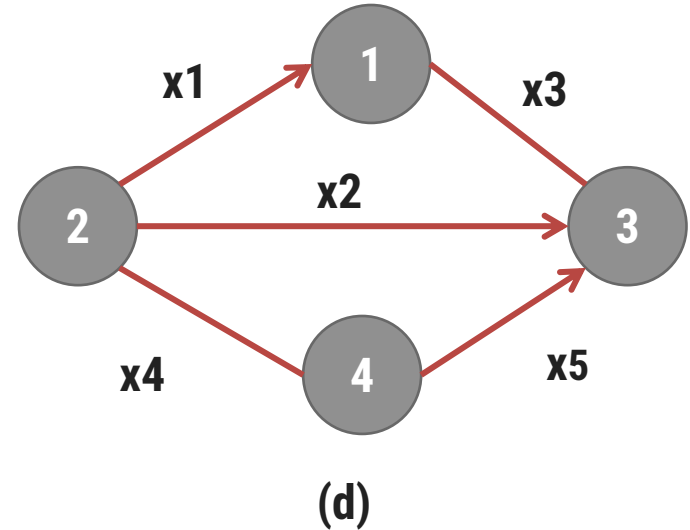
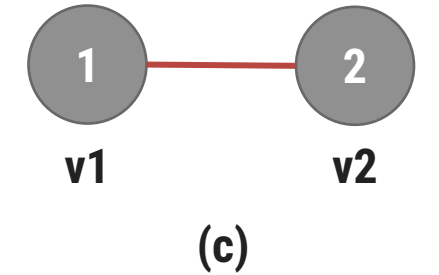
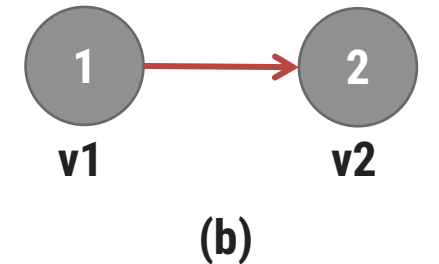
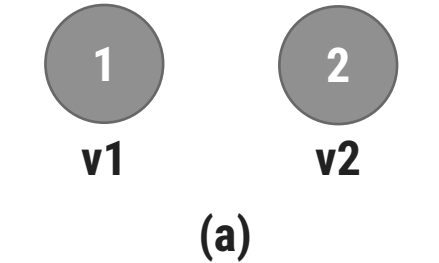


Non-Linear Data Structure Tree



Basic Notations of Graph Theory



Basic Notations of Graph Theory

- ▶ Consider diagrams shown in above figure
- ▶ Every diagrams represent Graphs
- ▶ Every diagram consists of a **set of vertices** which are shown by **dots** or **circles** and are sometimes labelled $V_1, V_2, V_3 \dots$ OR $1, 2, 3 \dots$
- ▶ In every diagrams, certain pairs of such points are connected by lines or arcs
- ▶ Note that every arc start at one point and ends at another point

Basic Notations of Graph Theory

► Graph

- A graph G consist of a **non-empty set V** called the **set of vertices** of the graph, a **set E** which is the **set of edges** and a **mapping** from the set of edges E to a set of **pairs of elements of V**
- It is also convenient to write a graph as $G=(V,E)$
- Notice that definition of graph implies that to every edge of a graph G , we can associate a pair of nodes of the graph. If an edge $x \in E$ is thus associated with a pair of nodes (u,v) where $u, v \in V$ then we says that edge x connect u and v

► Adjacent Nodes

- Any two nodes which are connected by an edge in a graph are called adjacent nodes

Graph – Concepts & Definitions

▶ Directed & Undirected Edge

- In a graph $G=(V,E)$ an **edge** which is **directed** from one end to another end is called a **directed edge**, while the edge which has no specific direction is called **undirected edge**

▶ Directed graph (Digraph)

- A graph in which **every edge is directed** is called directed graph or digraph e.g. **b, e & g** are directed graphs

▶ Undirected graph

- A graph in which **every edge is undirected** is called undirected graph e.g. **c & f** are undirected graphs

▶ Mixed Graph

- If **some** of the **edges** are **directed** and **some are undirected** in graph then the graph is called mixed graph e.g. **d** is mixed graph

Graph – Concepts & Definitions

► Loop (Sling)

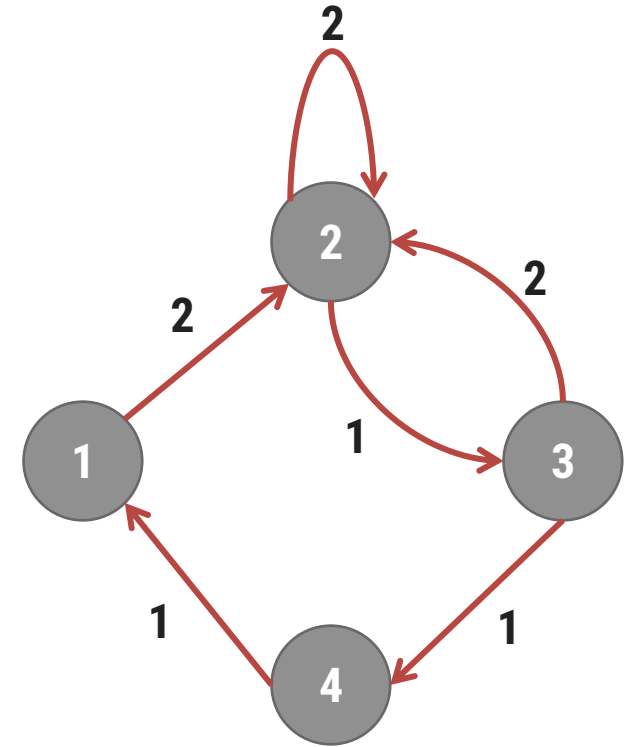
- An **edge** of a graph **which joins a node to itself** is called a loop (sling).
- The *direction of a loop is of no significance* so it can be considered either a directed or an undirected.

► Distinct Edges

- In case of directed edges, **two possible edges** between any pair of nodes which **are opposite in direction** are considered **Distinct**.

► Parallel Edges

- In some directed as well as undirected graphs, we may have **certain pairs of nodes joined by more than one edges**, such edges are called **Parallel** edges.



Graph – Concepts & Definitions

▶ Multigraph

- Any **graph** which **contains** some **parallel edges** is called **multigraph**
- If there is no more than one edge between a pair of nodes then such a graph is called **Simple graph**

▶ Weighted Graph

- A graph in which **weights are assigned to every edge** is called weighted graph

▶ Isolated Node

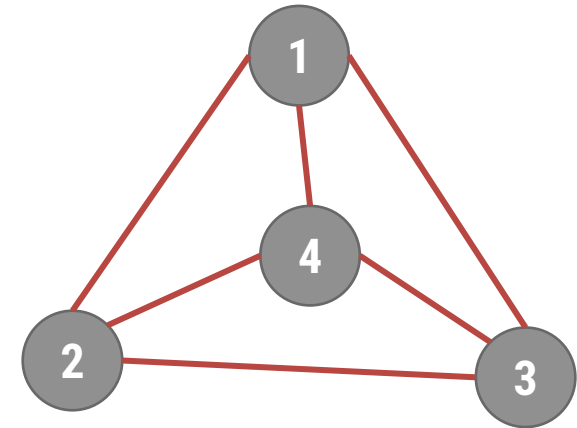
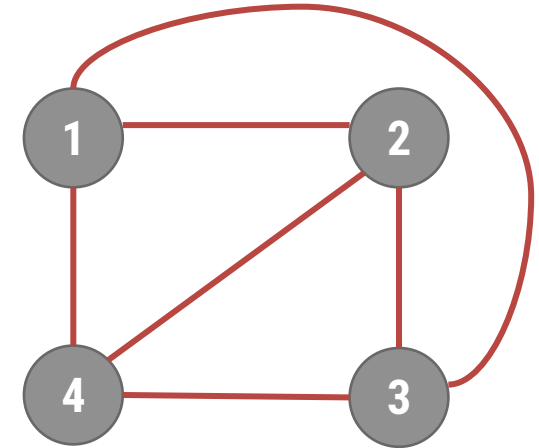
- In a graph a **node** which is **not adjacent to any other node** is called isolated node

▶ Null Graph

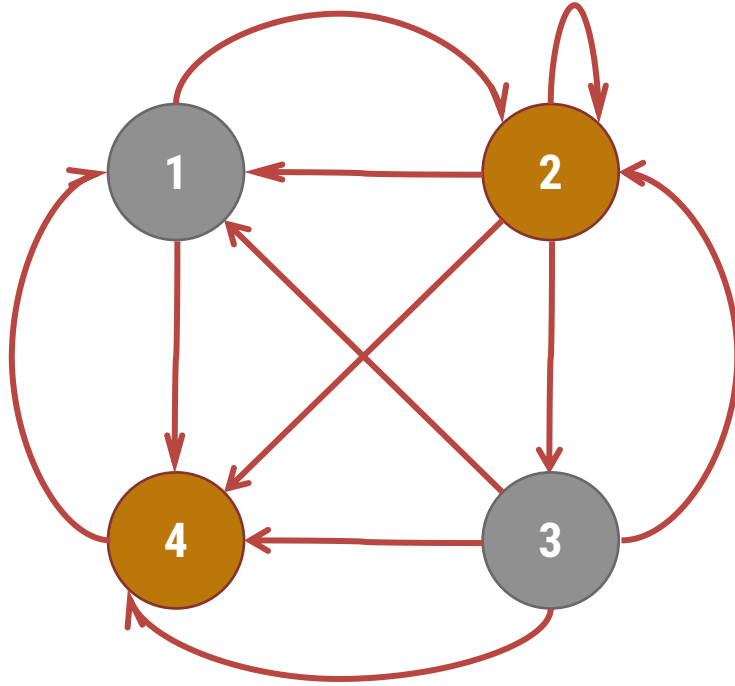
- A graph **containing only isolated nodes** are called null graph. In other words set of edges in null graph is empty

Graph – Concepts & Definitions

- ▶ For a given **graph** there is **no unique diagram** which represents the graph.
- ▶ We can obtain a variety of diagrams by locating the nodes in an arbitrary numbers.
- ▶ Following both diagrams represents same Graph.
- ▶ **Indegree of Node**
 - The **no of edges** which have **V as their terminal node** is call as indegree of node V.
- ▶ **Outdegree of Node**
 - The **no of edges** which have **V as their initial node** is call as outdegree of node V.
- ▶ **Total degree of Node**
 - Sum of indegree and outdegree of node V is called its Total Degree or Degree of vertex.



Path of the Graph



Some of the path from 2 to 4

P1 = ((2,4))

P2 = ((2,3), (3,4))

P3 = ((2,1), (1,4))

P4 = ((2,3), (3,1), (1,4))

P5 = ((2,3), (3,2), (2,4))

P6 = ((2,2), (2,4))

► Let $G=(V, E)$ be a simple digraph such that the terminal node of any edge in the sequence is the initial node of the edge, if any appearing next in the sequence defined as **path of the graph**.

► Length of Path

→ The number of edges appearing in the sequence of the path is called length of path.

Graph – Concepts & Definitions

▶ Simple Path (Edge Simple)

- A **path** in a diagram in which **the edges are distinct** is called simple path or edge simple
- Path P5, P6 are Simple Paths

▶ Elementary Path (Node Simple)

- A **path** in which **all the nodes through which it traverses** are **distinct** is called elementary path
- Path P1, P2, P3 & P4 are elementary Path
- Path P5, P6 are Simple but not Elementary

▶ Cycle (Circuit)

- A **path** which **originates and ends in the same node** is called cycle (circuit)
- E.g. $C1 = ((2,2))$, $C2 = ((1,2),(2,1))$, $C3 = ((2,3), (3,1), (1,2))$

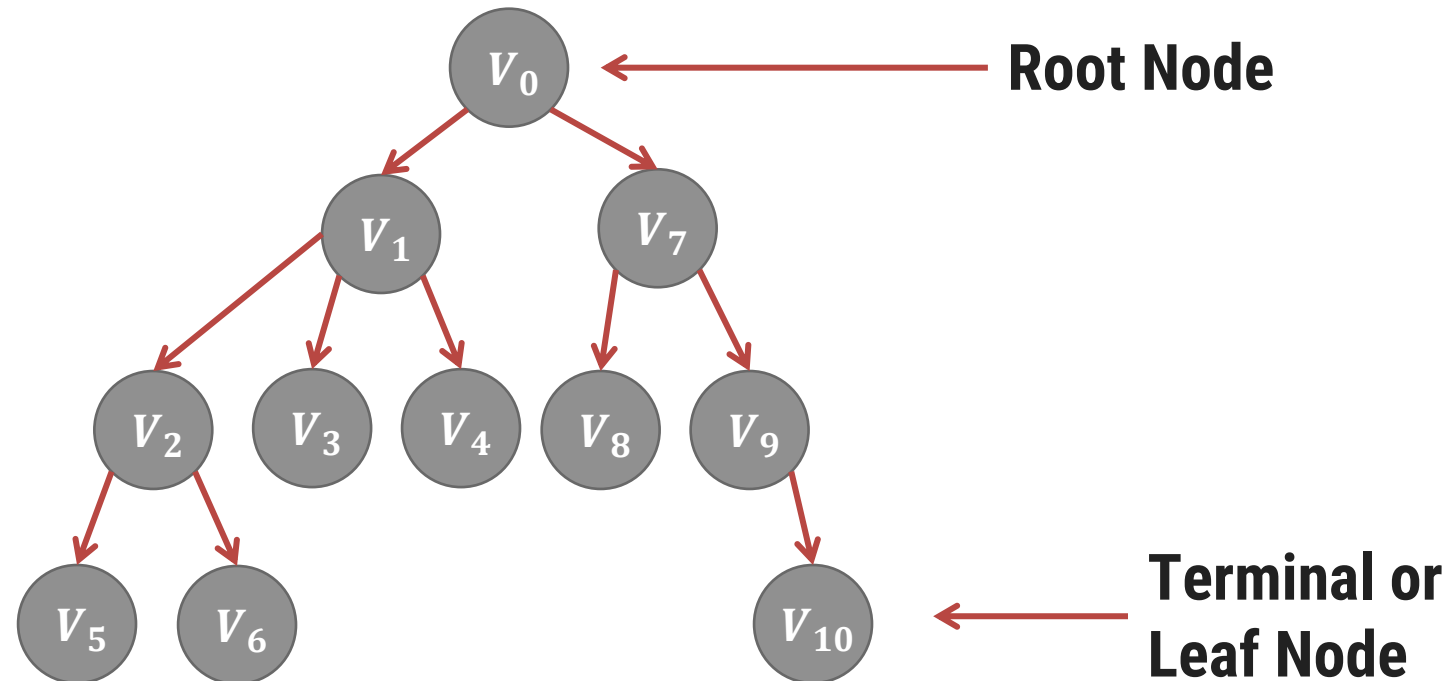
▶ Acyclic Diagram

- A simple **diagraph which does not have any cycle** is called Acyclic Diagram.

Tree– Concepts & Definitions

► Directed Tree

- ➔ A directed tree is an acyclic digraph which has one node called its root with in degree 0, while all other nodes have in degree 1.
- ➔ Every directed tree must have at least one node.
- ➔ An isolated node is also a directed tree.



Definition

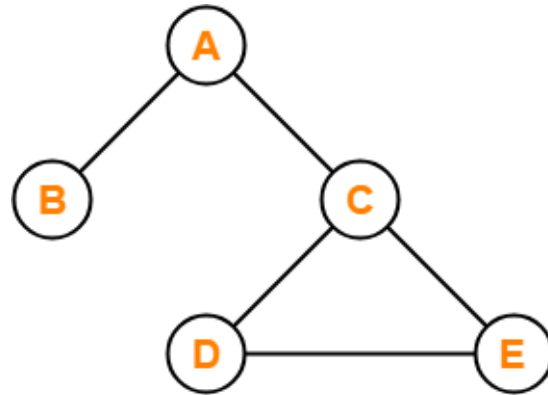
- ▶ Tree is a non-linear data structure which organizes data in a hierarchical structure and this is a recursive definition.

OR

- ▶ A tree is a connected graph without any circuits.

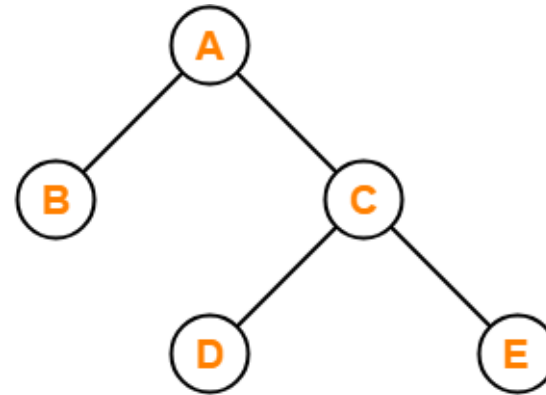
OR

- ▶ If in a graph, there is one and only one path between every pair of vertices, then graph is called as a tree.



X

This graph is not a Tree



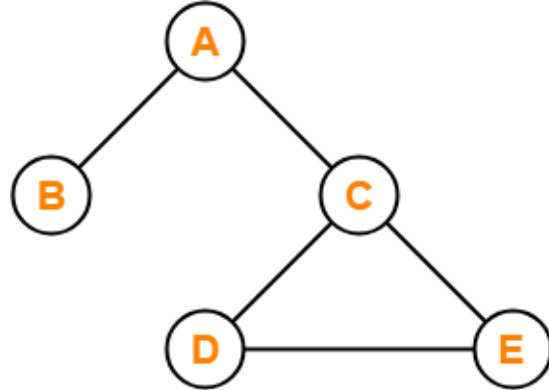
✓

This graph is a Tree

Properties of tree data structure

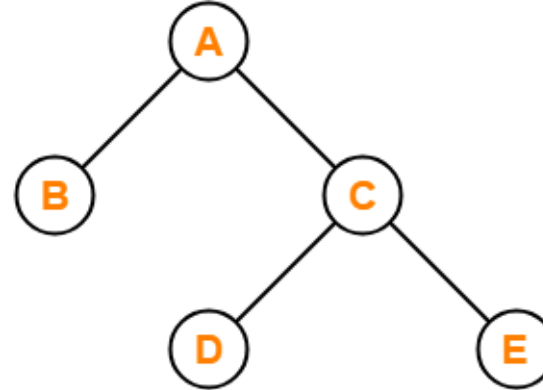
► Properties-

- The important properties of tree data structure are-
- There is one and only one path between every pair of vertices in a tree.
- A tree with n vertices has exactly $(n-1)$ edges.
- A graph is a tree if and only if it is minimally connected.
- Any connected graph with n vertices and $(n-1)$ edges is a tree.



X

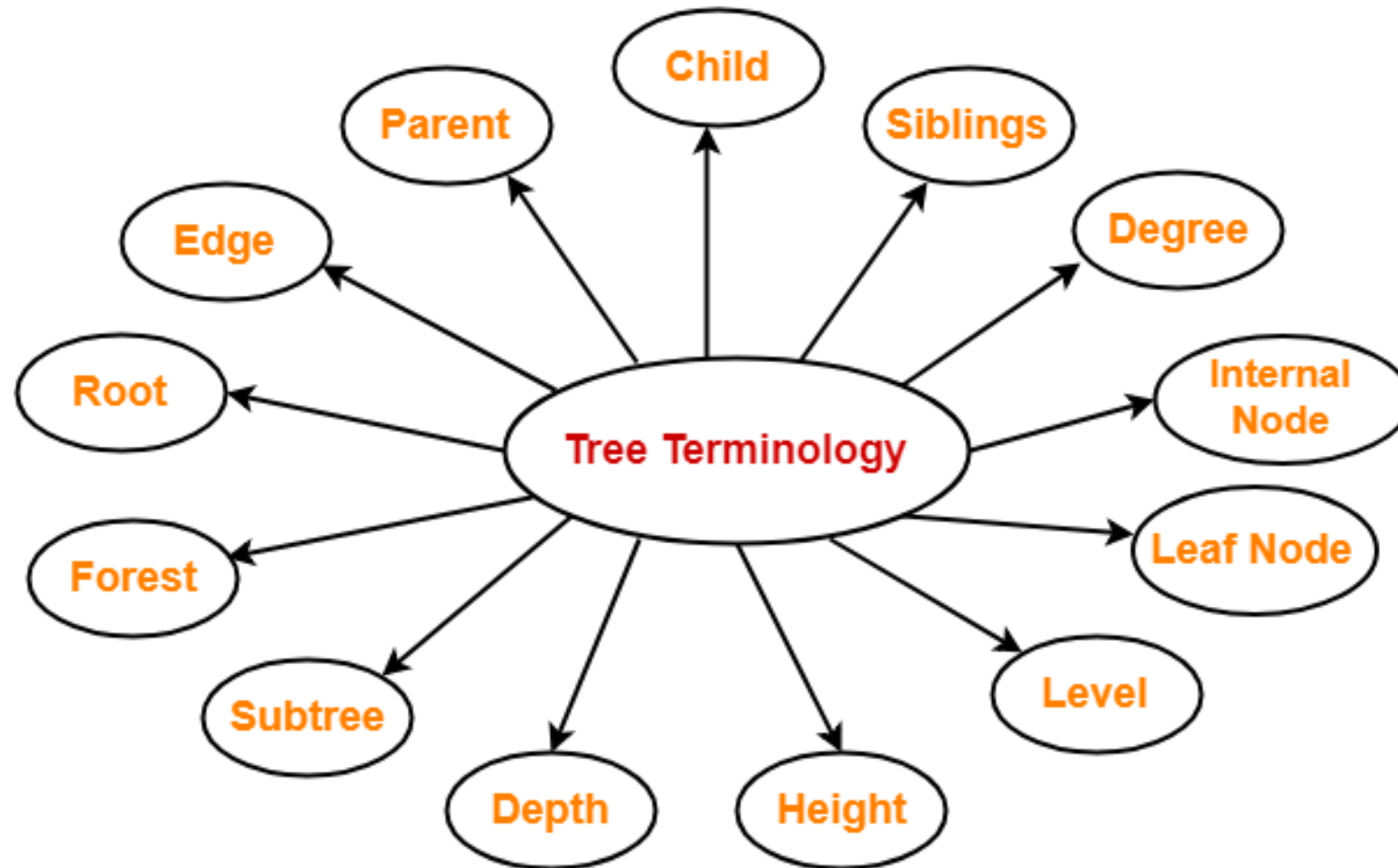
This graph is not a Tree



✓

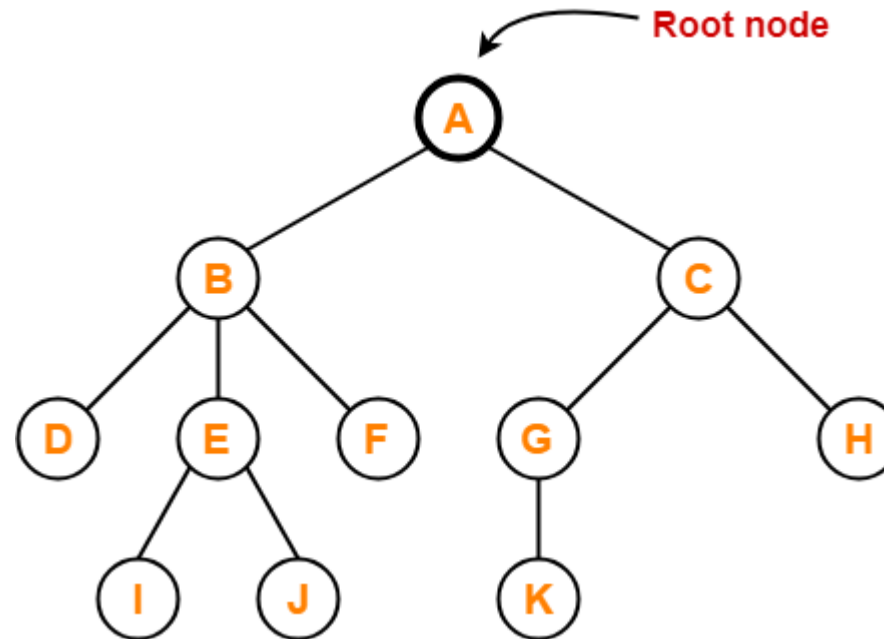
This graph is a Tree

Tree Terminology:



1. Root

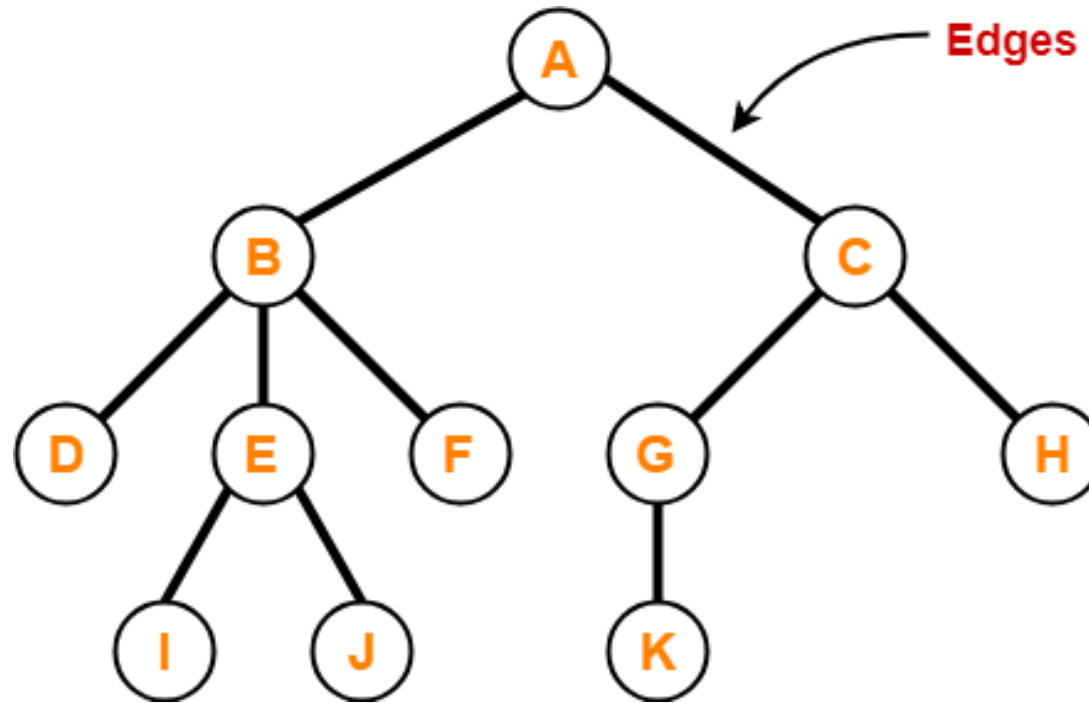
- ▶ The first node from where the tree originates is called as a **root node**.
- ▶ In any tree, there must be only one root node.
- ▶ We can never have multiple root nodes in a tree data structure



Here, node A is the only root node.

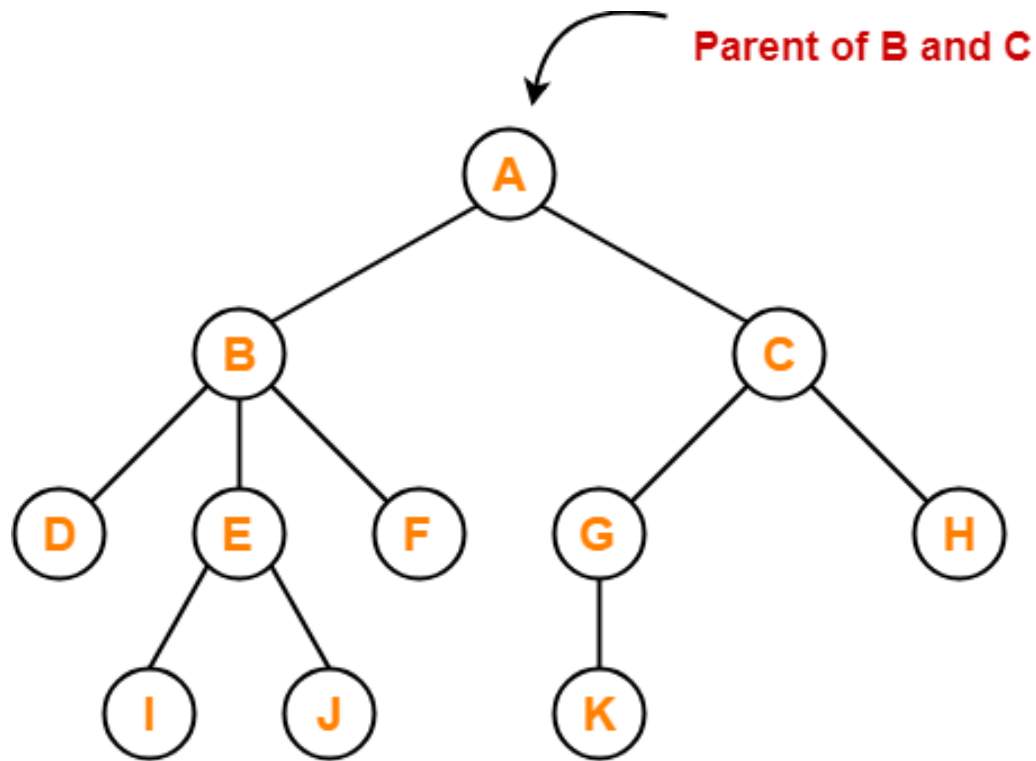
2. Edge-

- ▶ The connecting link between any two nodes is called as an edge.
- ▶ In a tree with n number of nodes, there are exactly $(n-1)$ number of edges.



3. Parent-

- ▶ The node which has a branch from it to any other node is called as a parent node.
- ▶ In other words, the node which has one or more children is called as a parent node.
- ▶ In a tree, a parent node can have any number of child nodes.

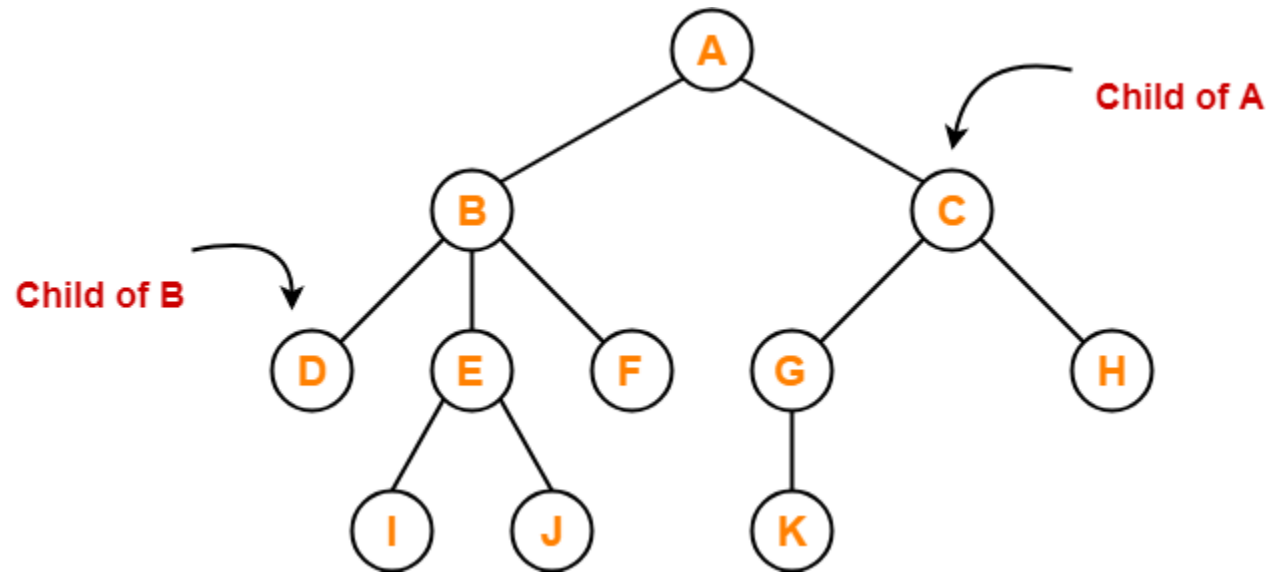


Here,

- Node A is the parent of nodes B and C
- Node B is the parent of nodes D, E and F
- Node C is the parent of nodes G and H
- Node E is the parent of nodes I and J
- Node G is the parent of node K

4. Child-

- ▶ The node which is a descendant of some node is called as a child node.
- ▶ All the nodes except root node are child nodes.

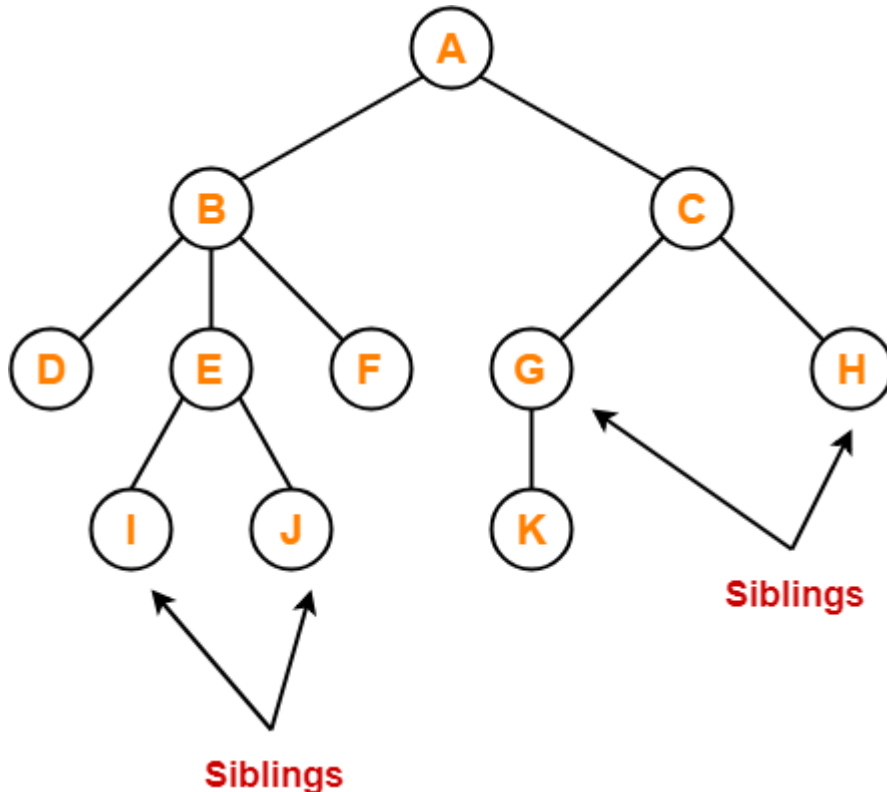


Here,

- Nodes B and C are the children of node A
- Nodes D, E and F are the children of node B
- Nodes G and H are the children of node C
- Nodes I and J are the children of node E
- Node K is the child of node G

5. Siblings-

- ▶ Nodes which belong to the same parent are called as siblings.
- ▶ In other words, nodes with the same parent are sibling nodes.

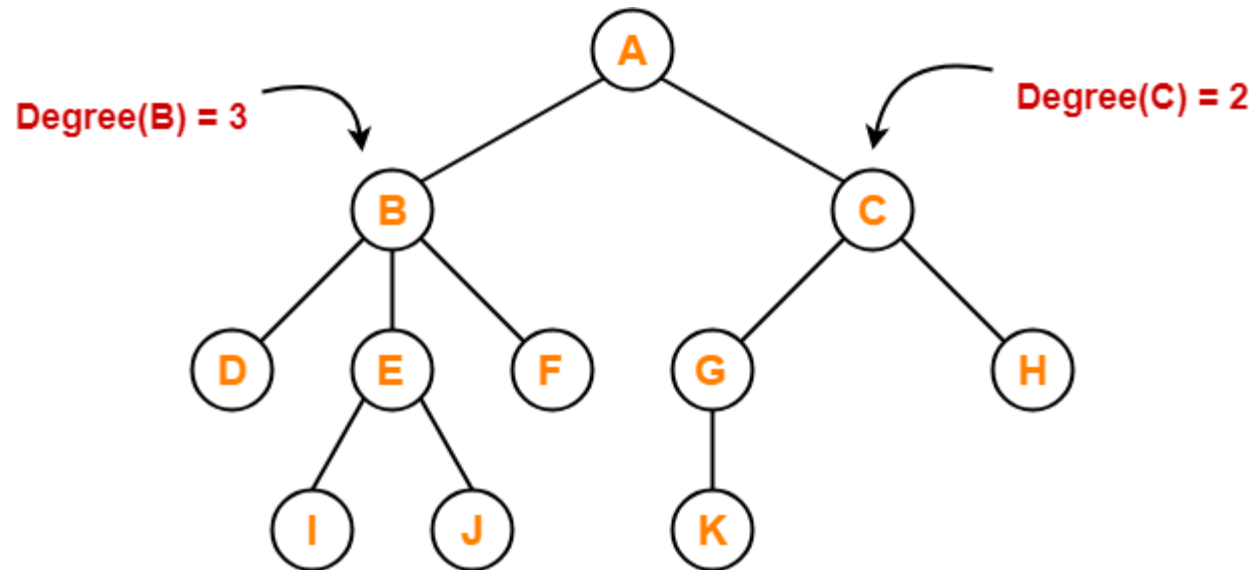


Here,

- Nodes B and C are siblings
- Nodes D, E and F are siblings
- Nodes G and H are siblings
- Nodes I and J are siblings

6. Degree-

- ▶ Degree of a node is the total number of children of that node.
- ▶ Degree of a tree is the highest degree of a node among all the nodes in the tree.

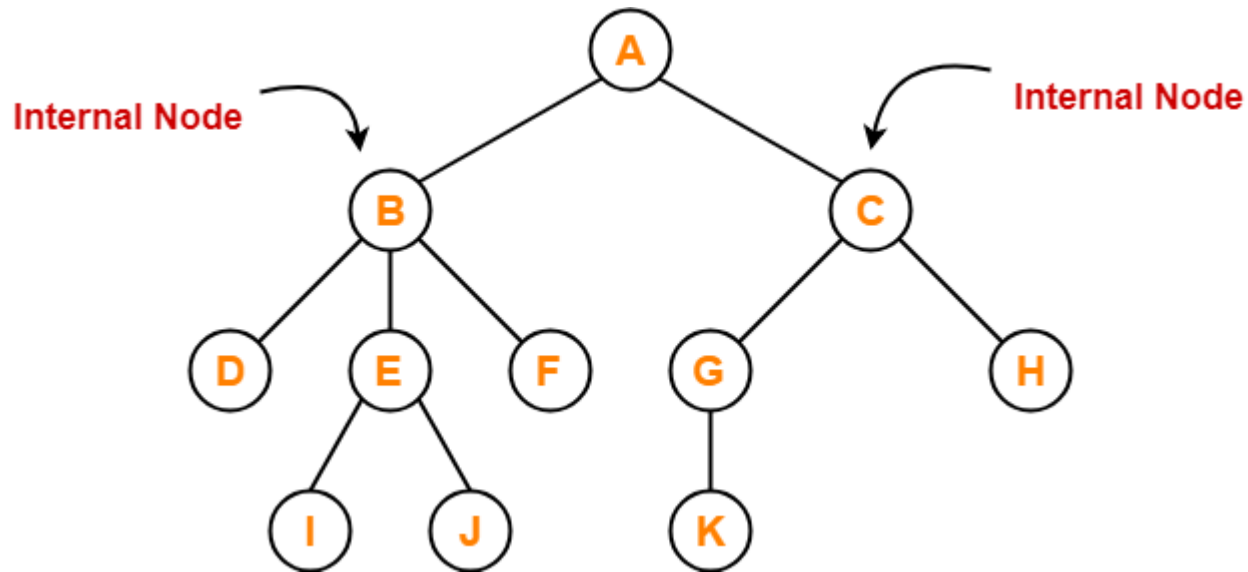


Here,

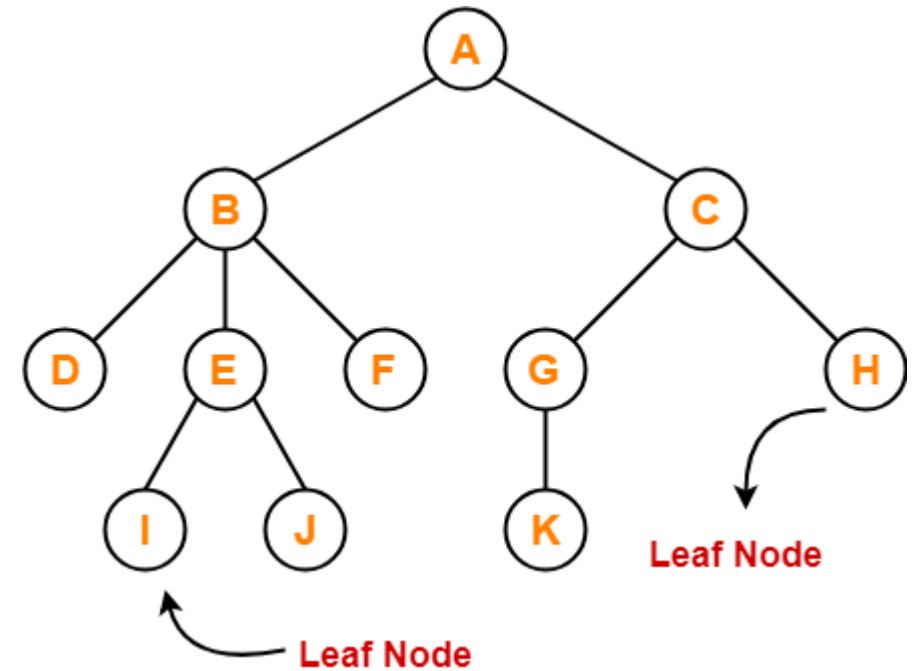
- Degree of node A = 2
- Degree of node B = 3
- Degree of node C = 2
- Degree of node D = 0
- Degree of node E = 2
- Degree of node F = 0
- Degree of node G = 1
- Degree of node H = 0
- Degree of node I = 0
- Degree of node J = 0
- Degree of node K = 0

7-8 Internal and Leaf Node

- ▶ The node which has at least one child is called as an internal node.
- ▶ Internal nodes are also called as non-terminal nodes.
- ▶ Every non-leaf node is an internal node.
- ▶ The node which does not have any child is called as a leaf node.
- ▶ Leaf nodes are also called as external nodes or terminal nodes.



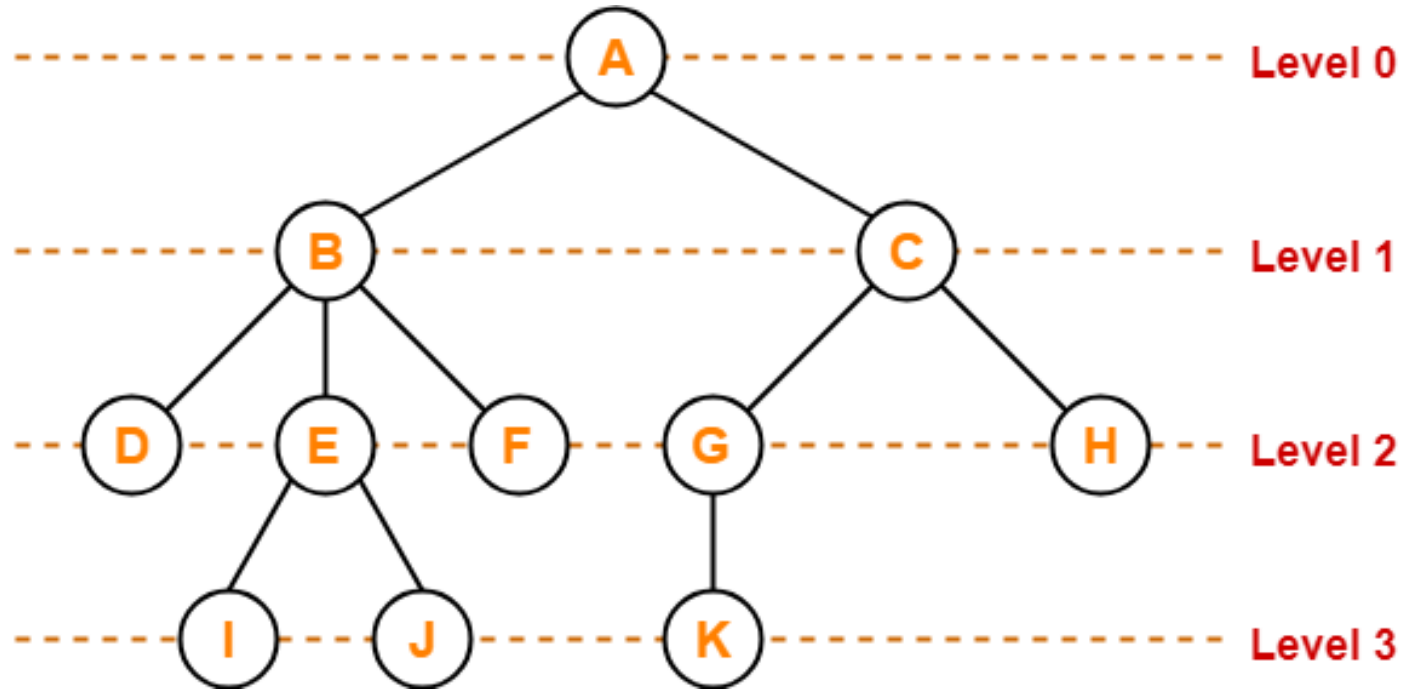
Here, nodes A, B, C, E and G are internal nodes.



Here, nodes D, I, J, F, K and H are leaf nodes.

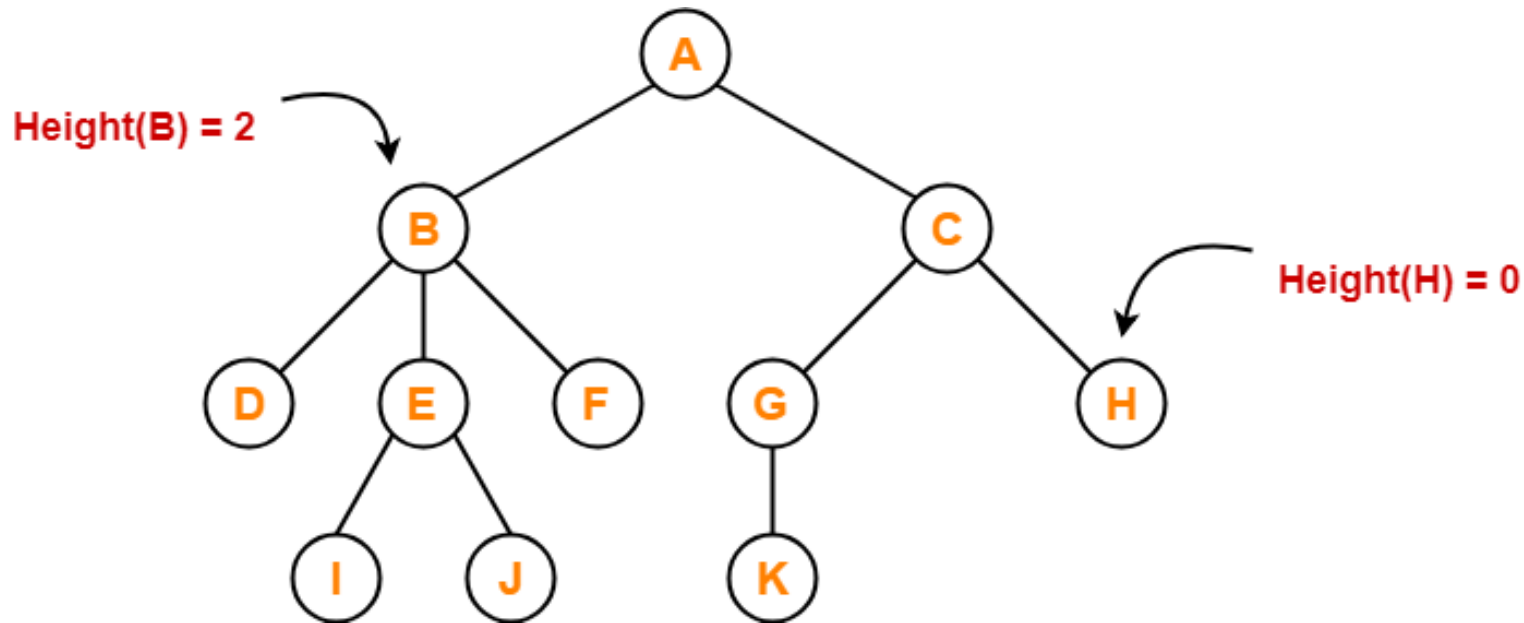
9. Level-

- ▶ In a tree, each step from top to bottom is called as level of a tree.
- ▶ The level count starts with 0 and increments by 1 at each level or step.



10. Height-

- ▶ Total number of edges that lies on the longest path from any leaf node to a particular node is called as height of that node.
- ▶ Height of a tree is the height of root node.
- ▶ Height of all leaf nodes = 0

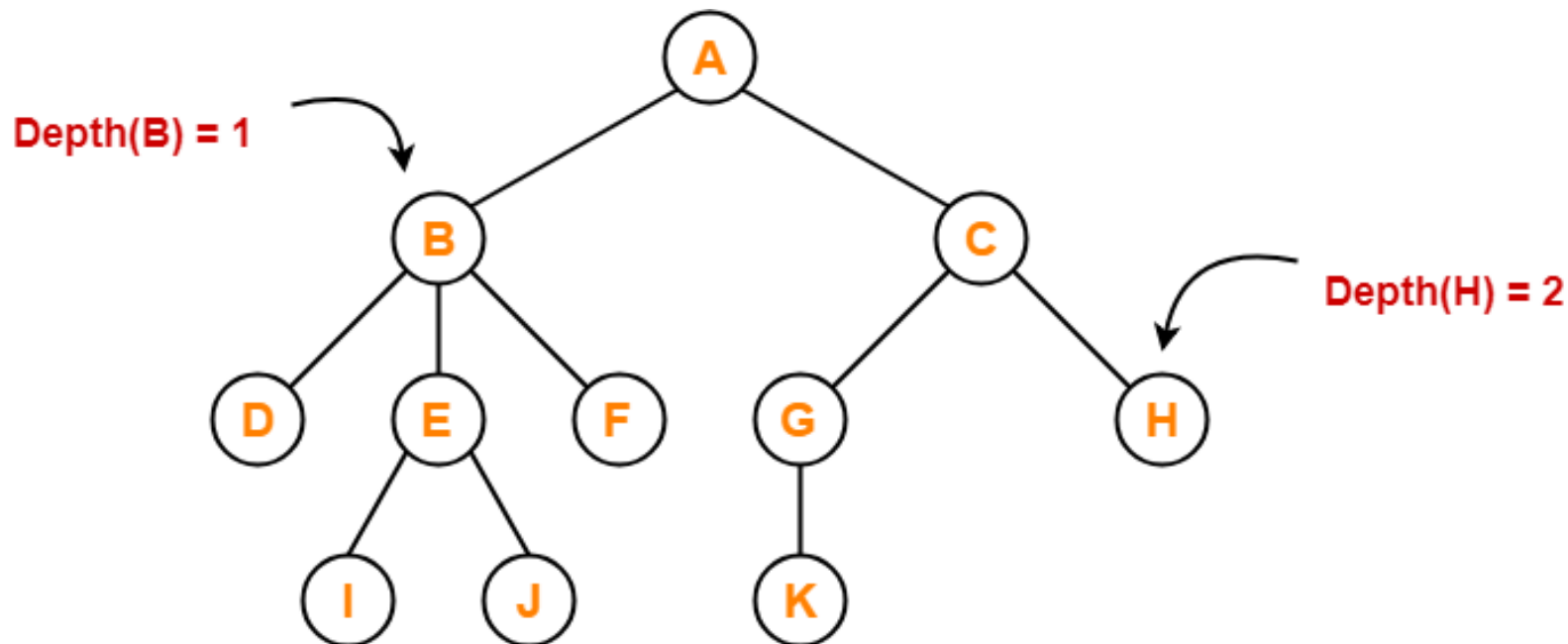


Here,

- Height of node A = 3
- Height of node B = 2
- Height of node C = 2
- Height of node D = 0
- Height of node E = 1
- Height of node F = 0
- Height of node G = 1
- Height of node H = 0
- Height of node I = 0
- Height of node J = 0
- Height of node K = 0

11. Depth-

- ▶ Total number of edges from root node to a particular node is called as depth of that node.
- ▶ Depth of a tree is the total number of edges from root node to a leaf node in the longest path.
- ▶ Depth of the root node = 0
- ▶ The terms “level” and “depth” are used interchangeably.

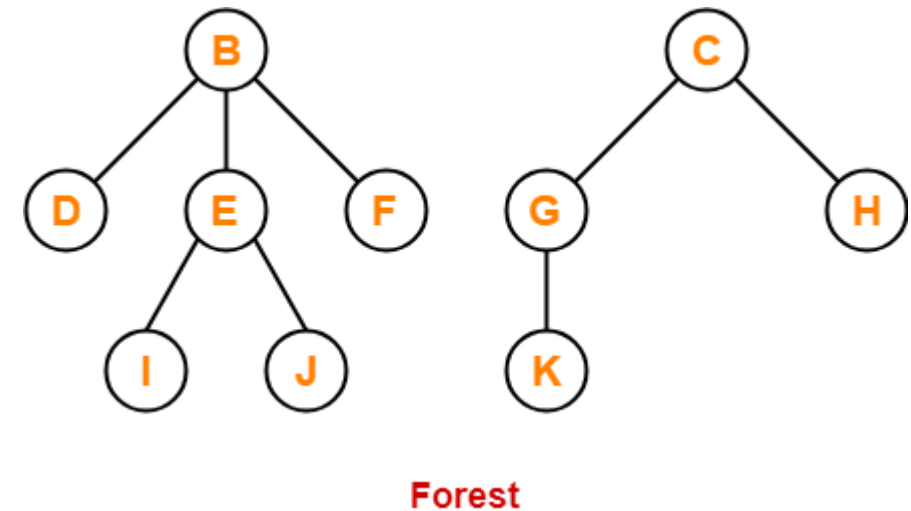
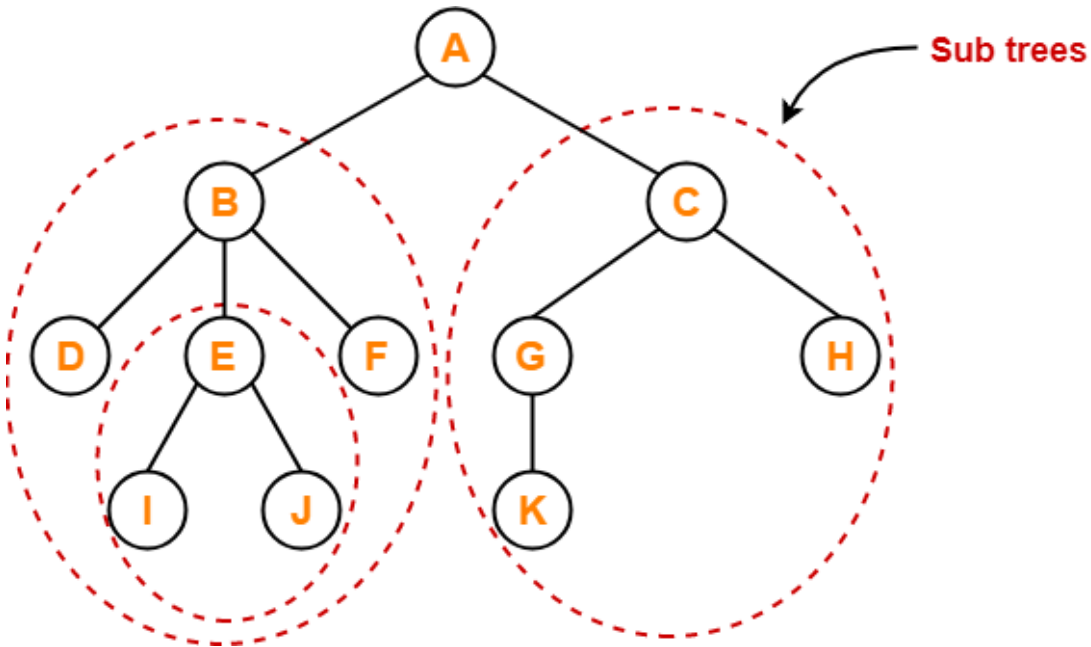


Here,

- Depth of node A = 0
- Depth of node B = 1
- Depth of node C = 1
- Depth of node D = 2
- Depth of node E = 2
- Depth of node F = 2
- Depth of node G = 2
- Depth of node H = 2
- Depth of node I = 3
- Depth of node J = 3
- Depth of node K = 3

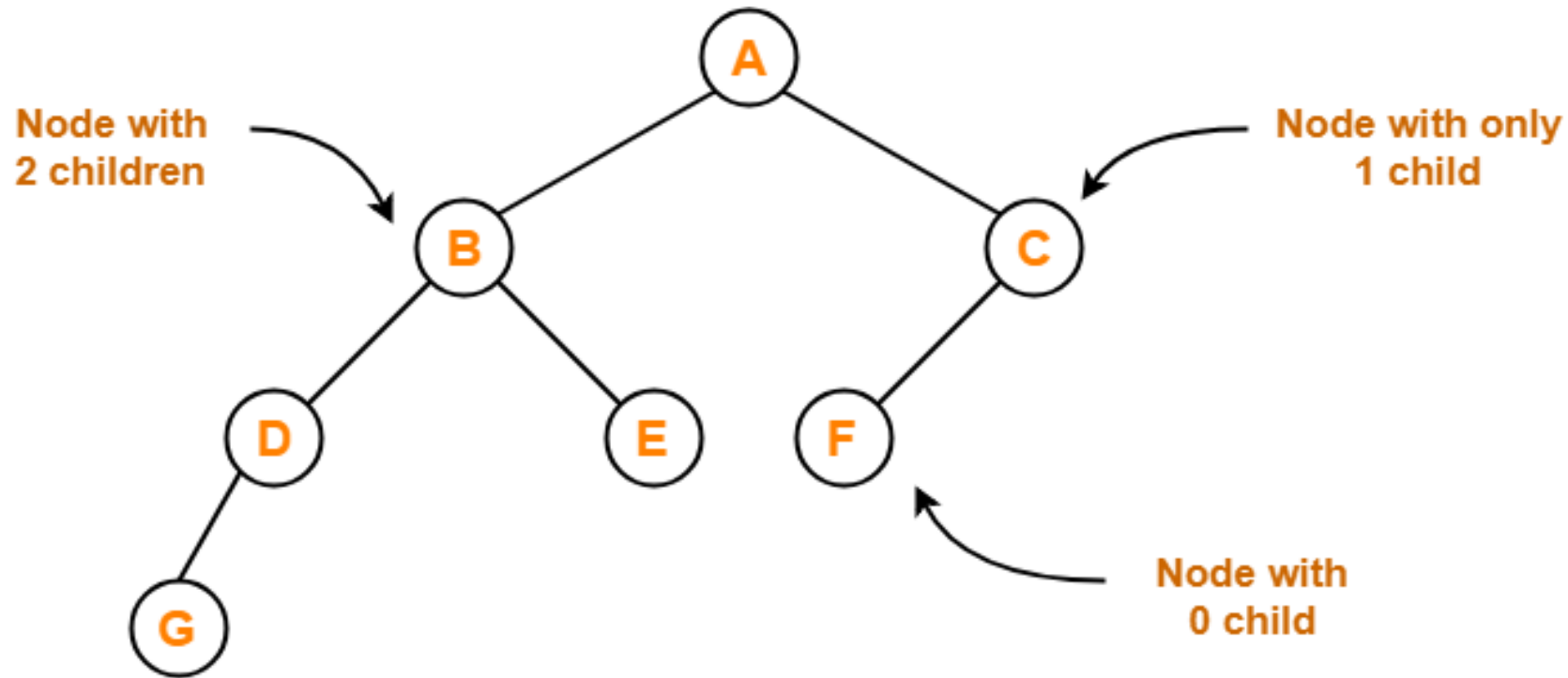
12-13 Subtree and Forest

- ▶ In a tree, each child from a node forms a subtree recursively.
- ▶ Every child node forms a subtree on its parent node.
- ▶ A forest is a set of disjoint trees.



Binary Tree

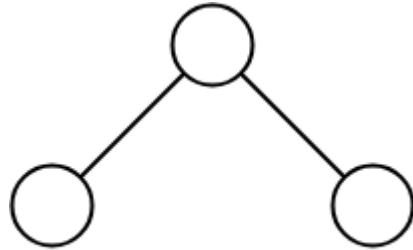
- ▶ Binary tree is a special tree data structure in which each node can have at most 2 children.
- ▶ Thus, in a binary tree,
- ▶ Each node has **either 0 child or 1 child or 2 children.**



Binary Tree Example

Un-labeled Binary Tree-

- ▶ A binary tree is unlabeled if its nodes are not assigned any label

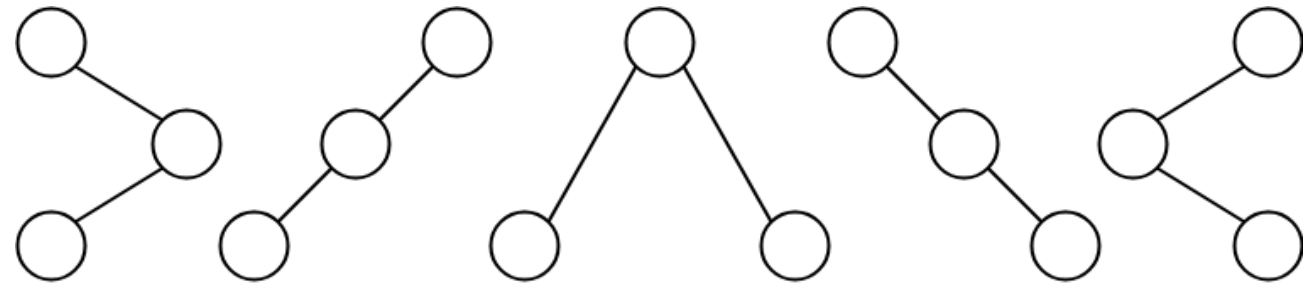


Unlabeled Binary Tree

$$\text{Number of different Binary Trees possible with 'n' unlabeled nodes} = \frac{2^n C_n}{n+1}$$

Consider we want to draw all the binary trees possible with 3 un-labeled nodes.
Using the above formula, we have-

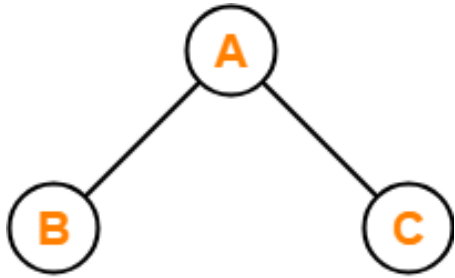
$$\begin{aligned} \text{Number of binary trees possible with 3 un-labeled nodes} \\ &= {}^{2 \times 3}C_3 / (3 + 1) \\ &= {}^6C_3 / 4 \\ &= 5 \end{aligned}$$



Binary Trees Possible With 3 Unlabeled Nodes

Labelled Binary Tree-

- ▶ A binary tree is labeled if all its nodes are assigned a label.



Labeled Binary Tree

Number of different Binary Trees possible
with 'n' labeled nodes

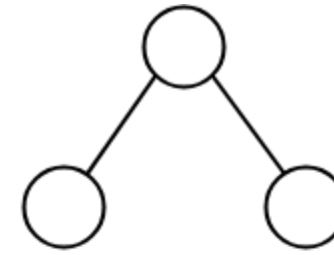
$$= \frac{2^n C_n}{n+1} \times n!$$

Consider we want to draw all the binary trees possible with 3 labeled nodes.
Using the above formula, we have-

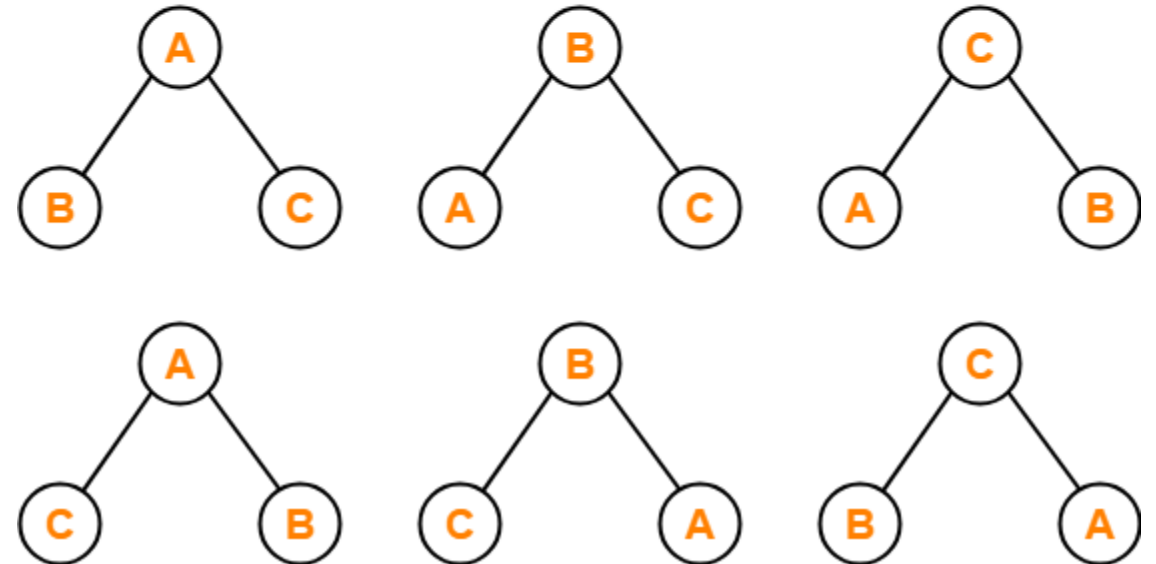
$$\begin{aligned} &\text{Number of binary trees possible with 3 labeled nodes} \\ &= \{ {}^{2 \times 3}C_3 / (3 + 1) \} \times 3! \\ &= \{ {}^6C_3 / 4 \} \times 6 \\ &= 5 \times 6 \\ &= 30 \end{aligned}$$

Similarly,

- Every other unlabeled structure gives rise to 6 different labeled structures.
- Thus, in total 30 different labeled binary trees are possible.



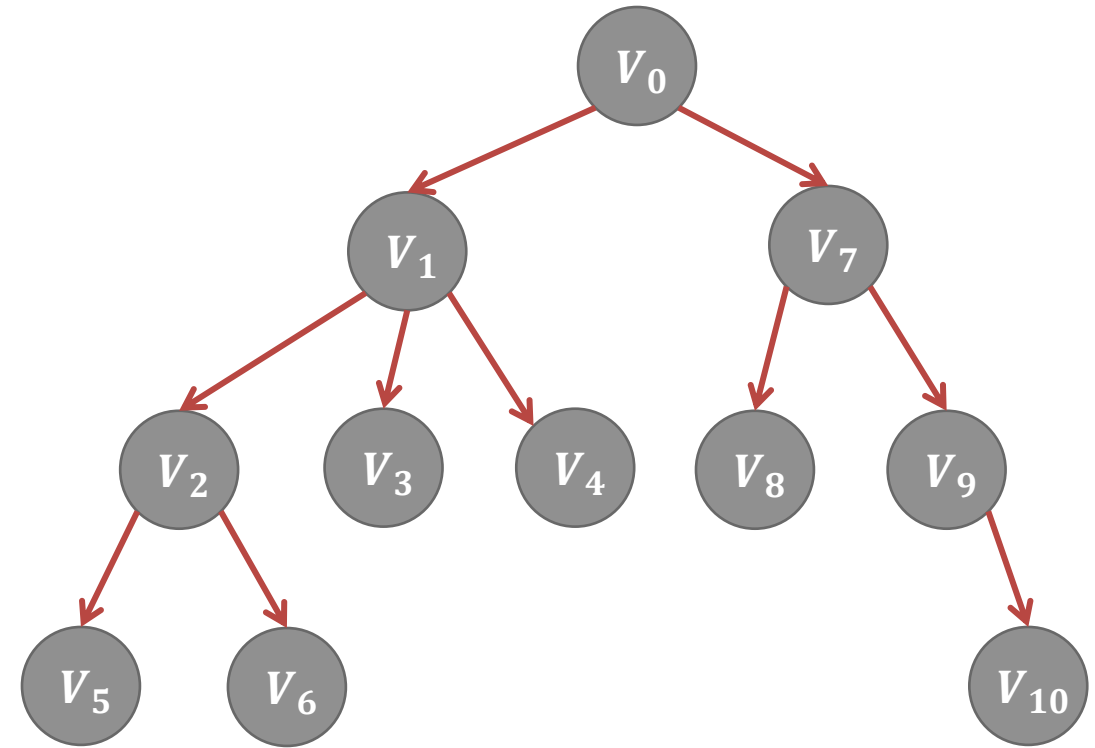
It Gives Rise to Following 6 Labeled Structures



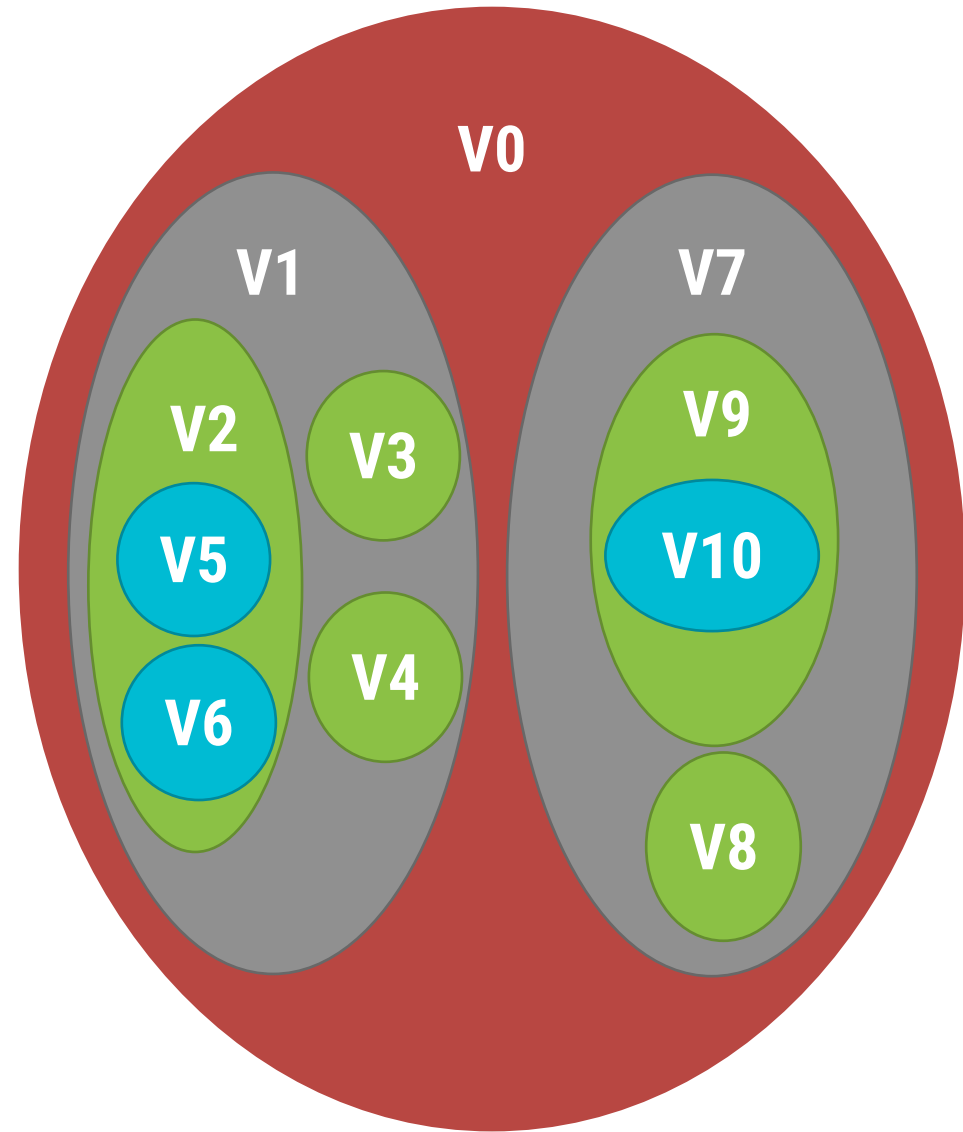
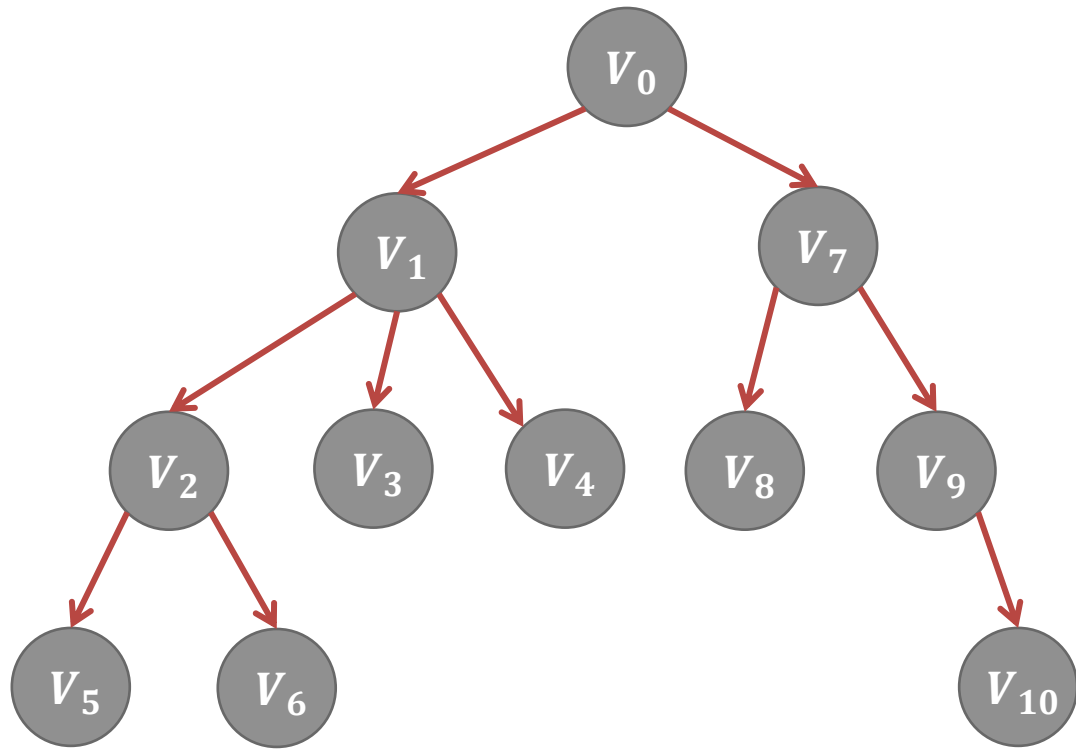
Representation of Directed Tree

► Other way to represent directed tree are

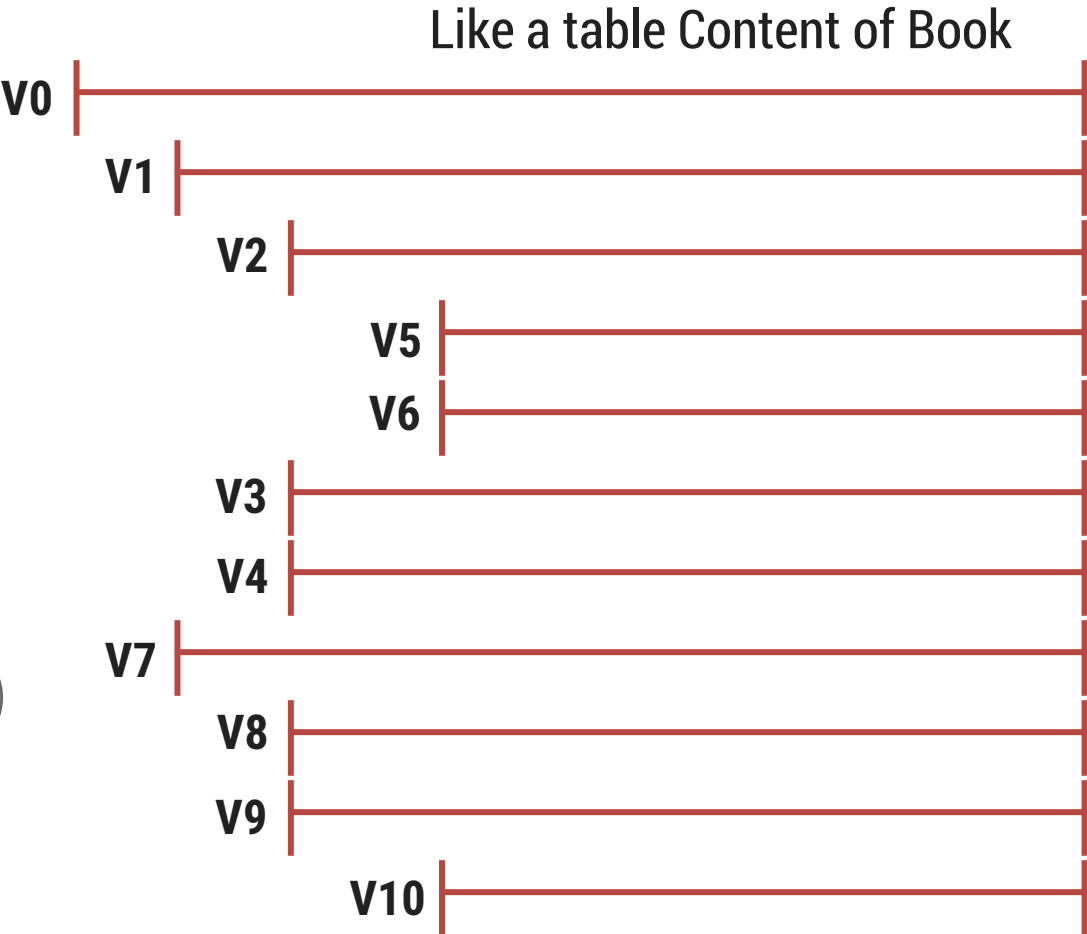
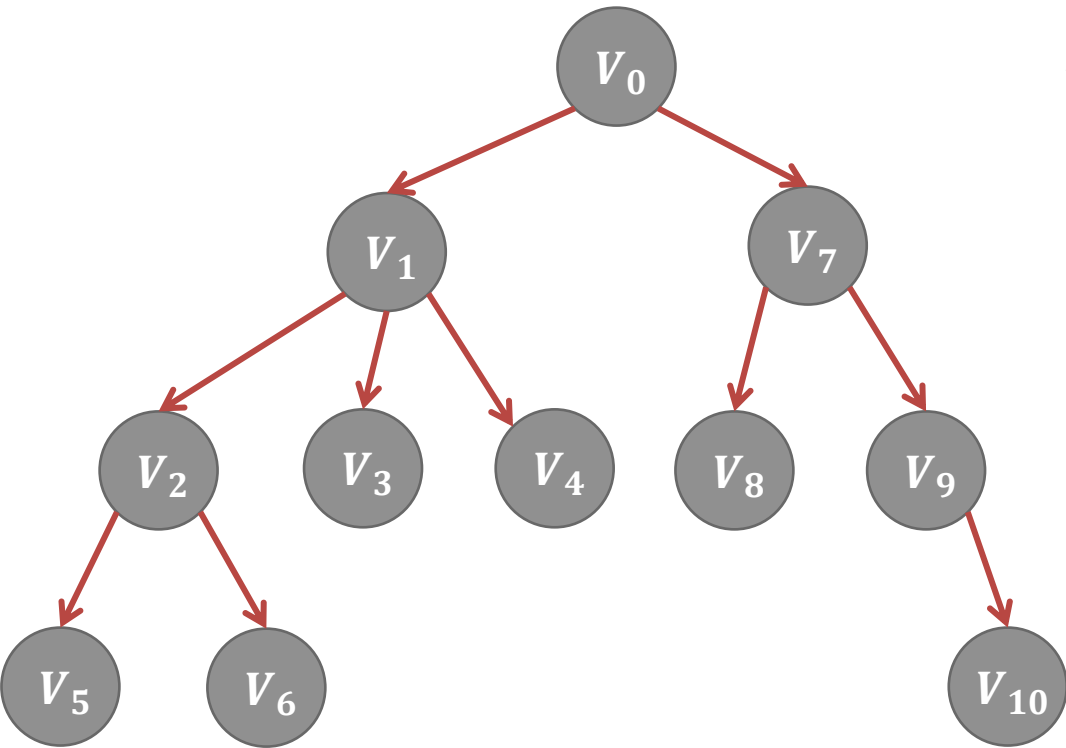
- Venn Diagram
- Nesting of Parenthesis
- Like table content of Book
- Level Format



Venn Diagram



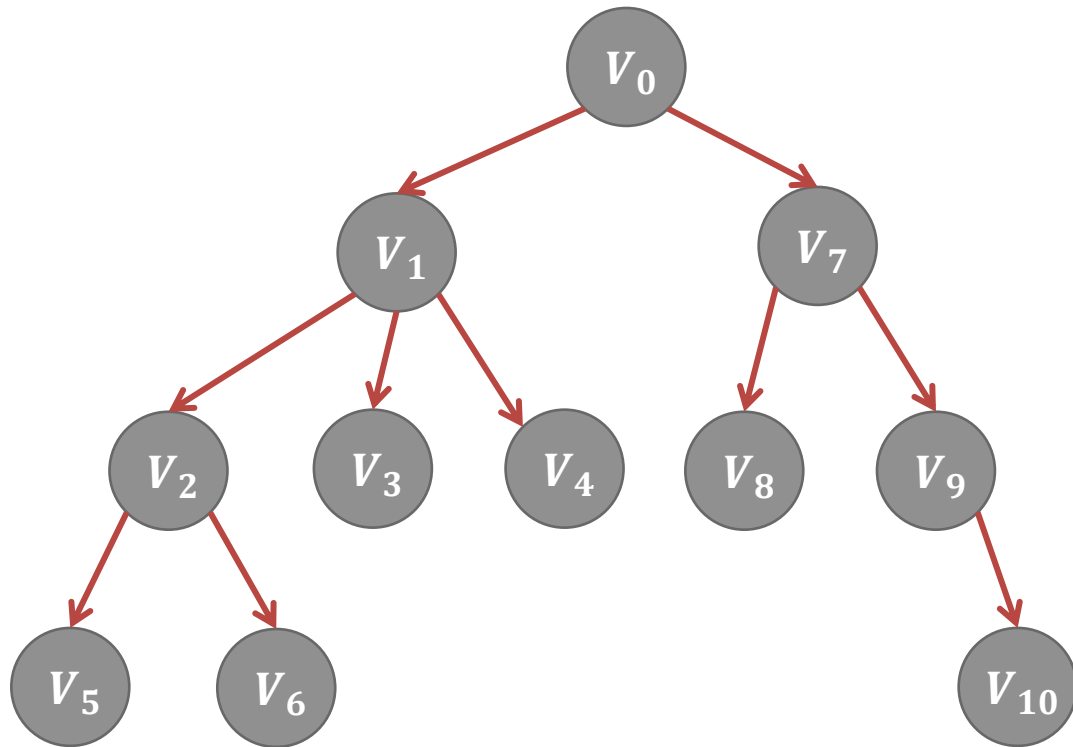
Nesting of Parenthesis



(V₀ (V₁ (V₂ (V₅) (V₆)) (V₃) (V₄)) (V₇ (V₈) (V₉ (V₁₀))))

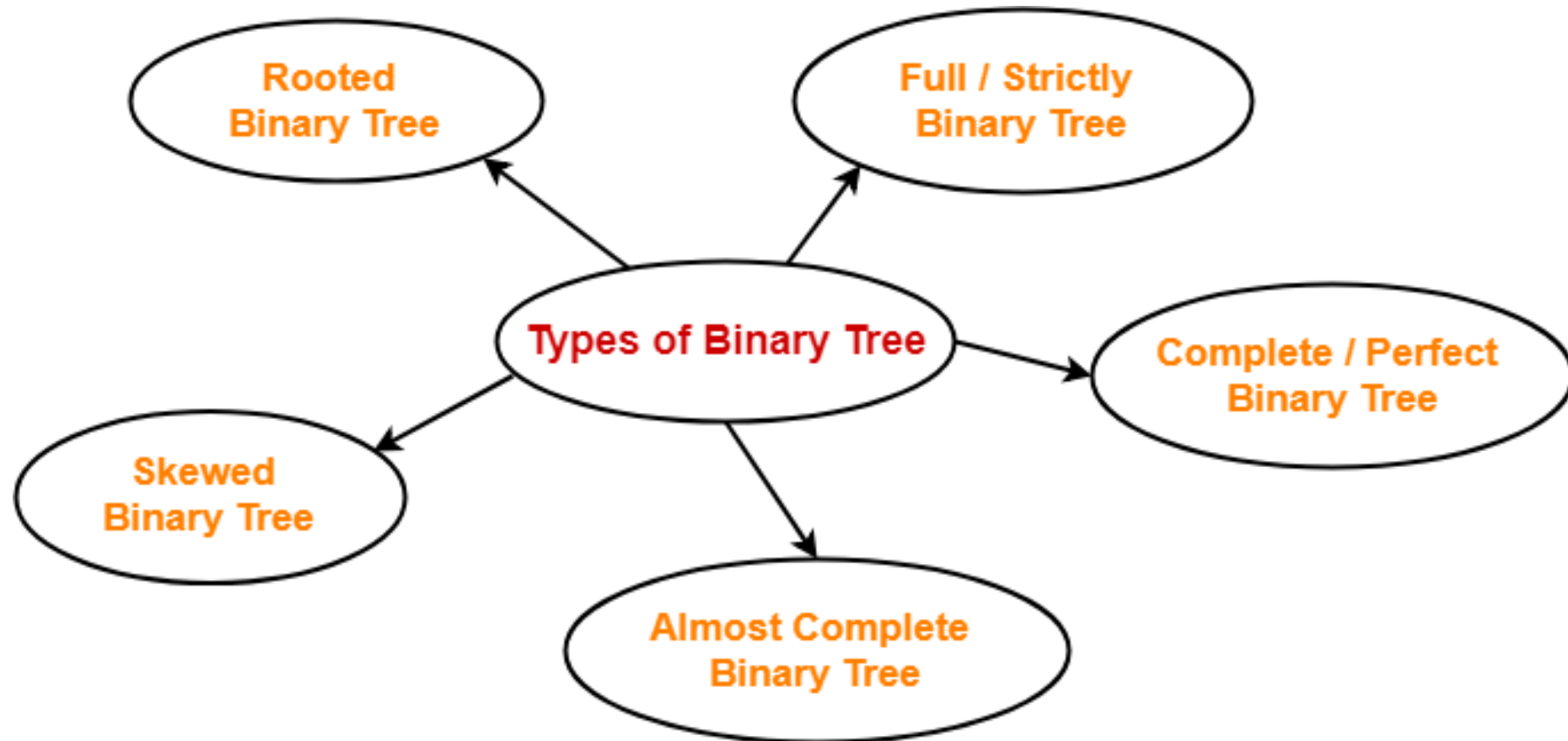
Nesting of Parenthesis

Level Format



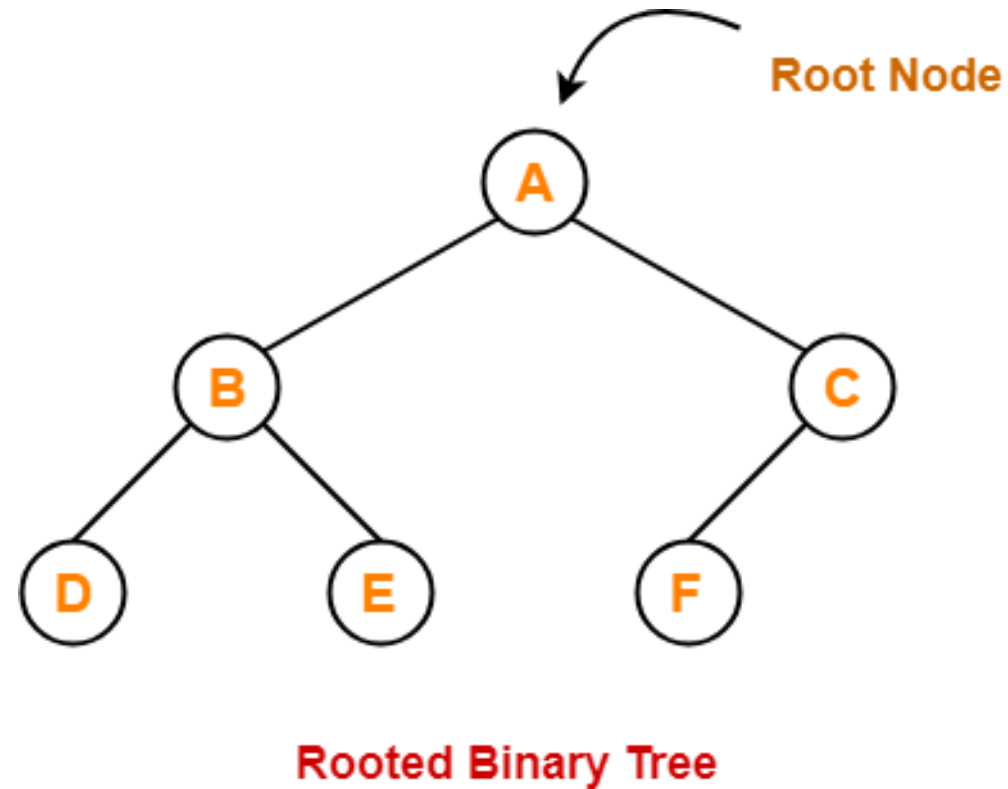
1 V_0
2 V_1
3 V_2
4 V_5
4 V_6
3 V_3
3 V_4
2 V_7
3 V_8
3 V_9
4 V_{10}

Types of Binary Trees-



1. Rooted Binary Tree-

- ▶ A rooted binary tree is a binary tree that satisfies the following 2 properties-
- ▶ It has a root node.
- ▶ Each node has at most 2 children.

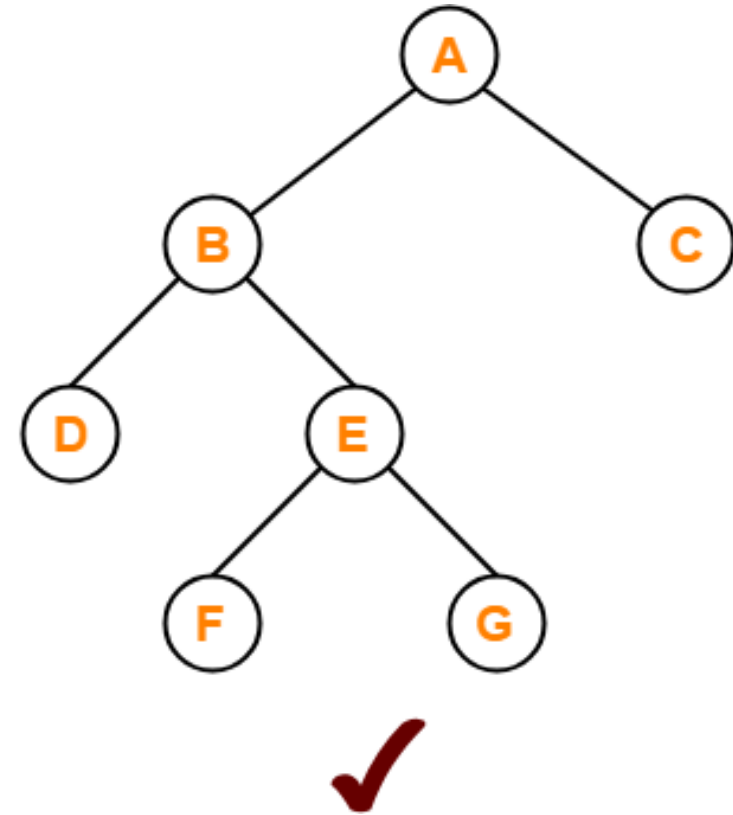
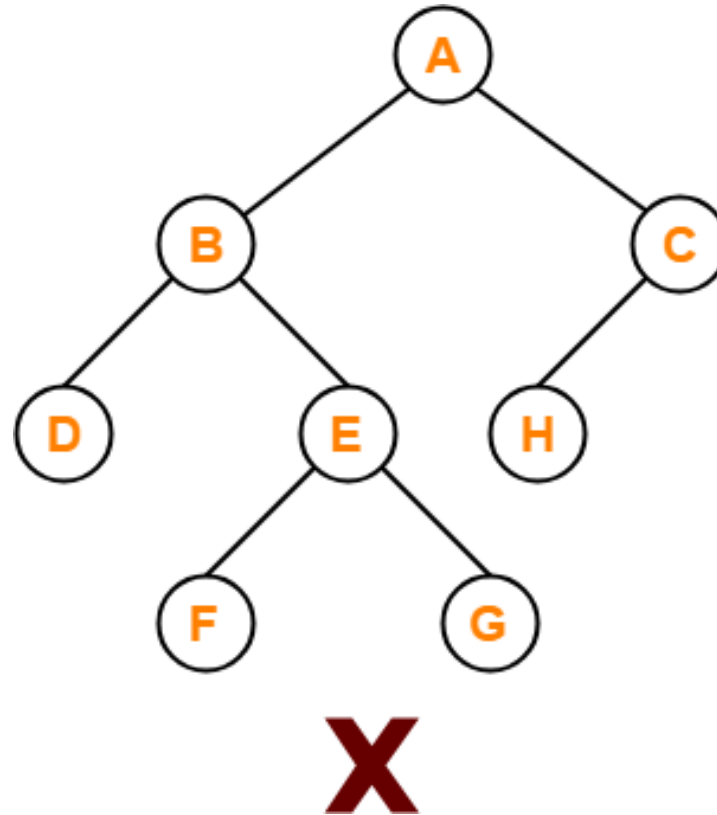


2. Full / Strict Binary Tree-

- ▶ A binary tree in which every node has either 0 or 2 children is called as a Full binary tree.
- ▶ Full binary tree is also called as Strict binary tree.

Here,

- First binary tree is not a full binary tree.
- This is because node C has only 1 child.

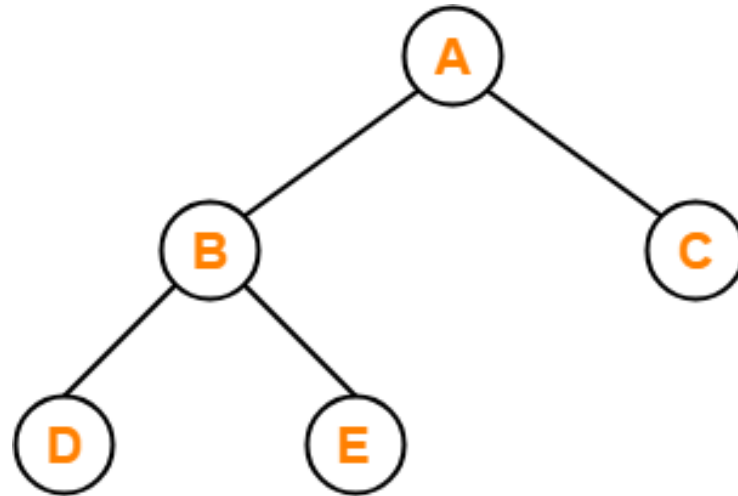


3. Complete / Perfect Binary Tree-

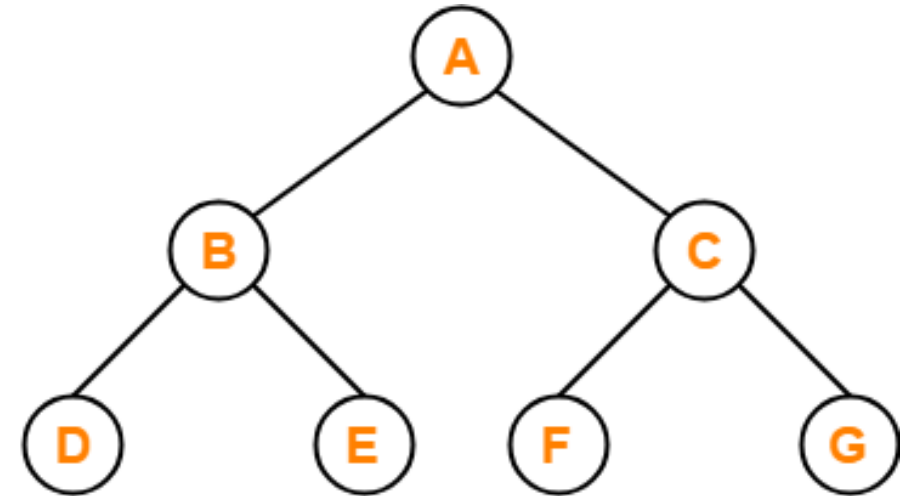
- ▶ A complete binary tree is a binary tree that satisfies the following 2 properties-
- ▶ Every internal node has exactly 2 children.
- ▶ All the leaf nodes are at the same level.
- ▶ Complete binary tree is also called as Perfect binary tree.

Here,

- First binary tree is not a complete binary tree.
- This is because all the leaf nodes are not at the same level.



X



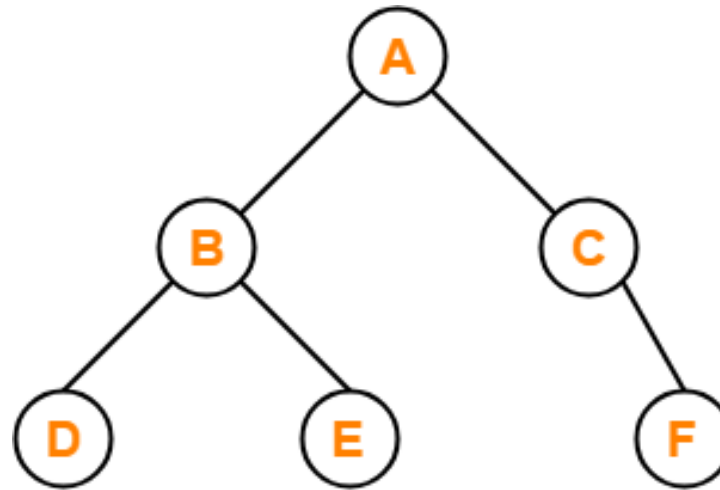
✓

4. Almost Complete Binary Tree-

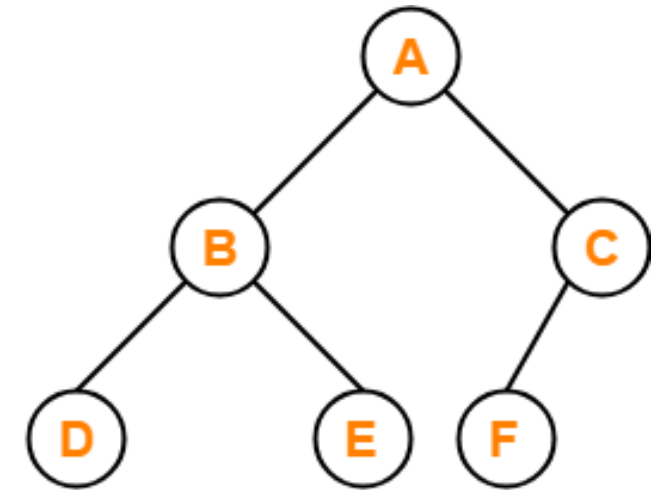
- ▶ An almost complete binary tree is a binary tree that satisfies the following 2 properties-
- ▶ All the levels are completely filled except possibly the last level.
- ▶ The last level must be strictly filled from left to right.

Here,

- First binary tree is not an almost complete binary tree.
- This is because the last level is not filled from left to right



X



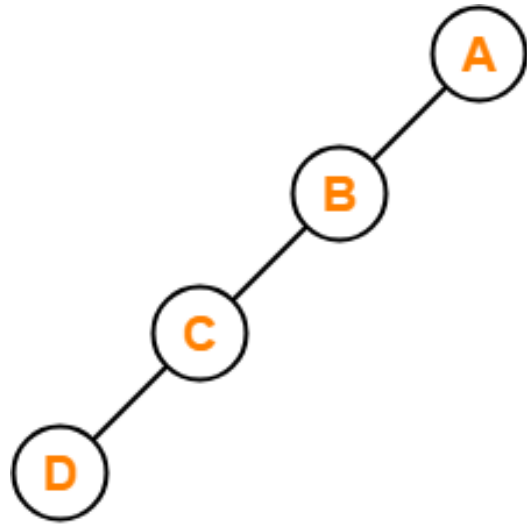
✓

5. Skewed Binary Tree-

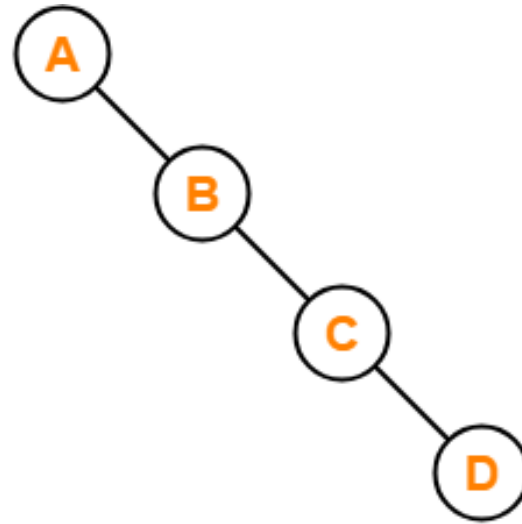
- ▶ A skewed binary tree is a binary tree that satisfies the following 2 properties-
- ▶ All the nodes except one node has one and only one child.
- ▶ The remaining node has no child.

OR

- ▶ A skewed binary tree is a binary tree of n nodes such that its depth is $(n-1)$.



Left Skewed Binary Tree

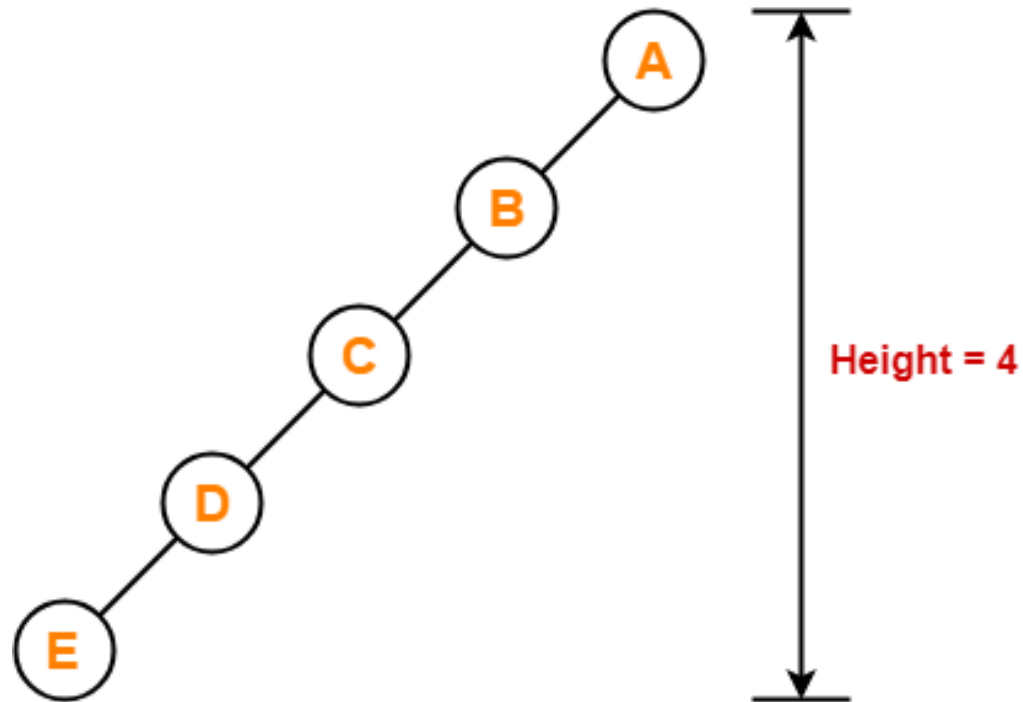


Right Skewed Binary Tree

Binary Tree Properties-

P1. Minimum number of nodes in a binary tree of height $H = H + 1$.

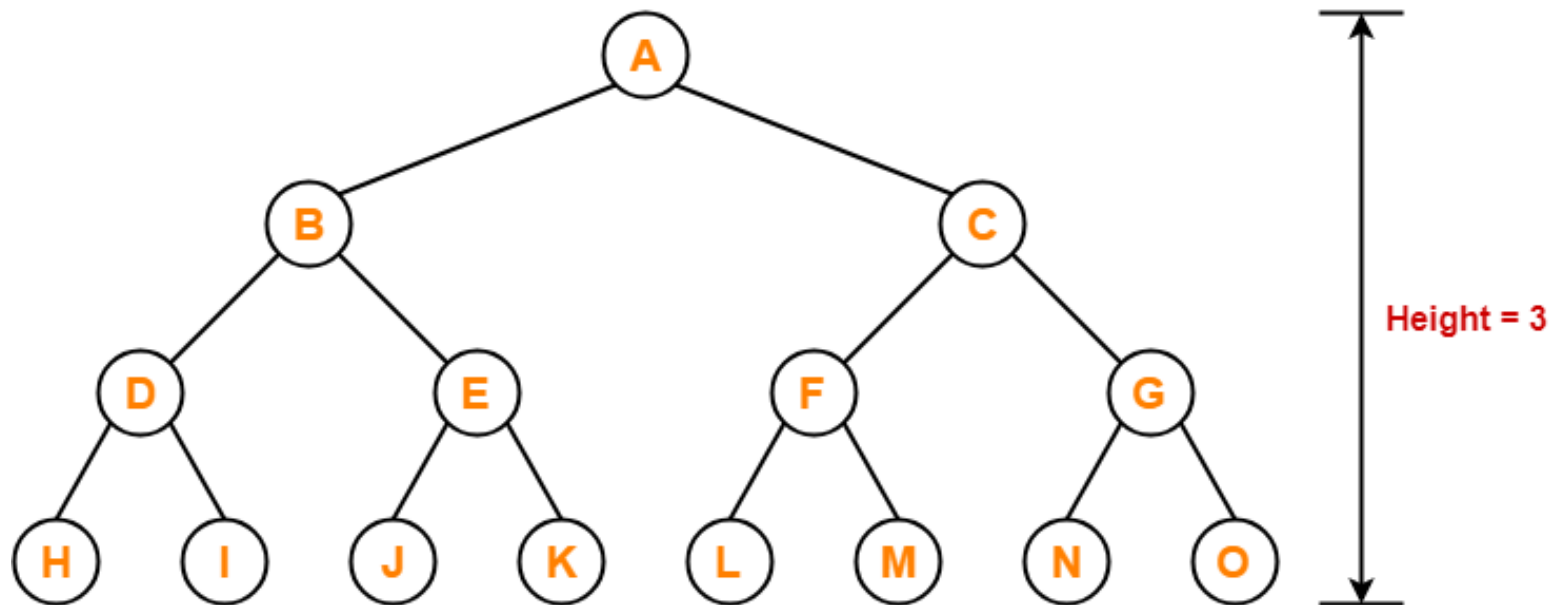
► Example: To construct a binary tree of height = 4, we need at least $4 + 1 = 5$ nodes.



P2: Maximum number of nodes in a binary tree of height $H = 2^{H+1} - 1$

► Example-

- Maximum number of nodes in a binary tree of height 3
- $= 2^{3+1} - 1$
- $= 16 - 1$
- $= 15$ nodes
- Thus, in a binary tree of height = 3, maximum number of nodes that can be inserted = 15.



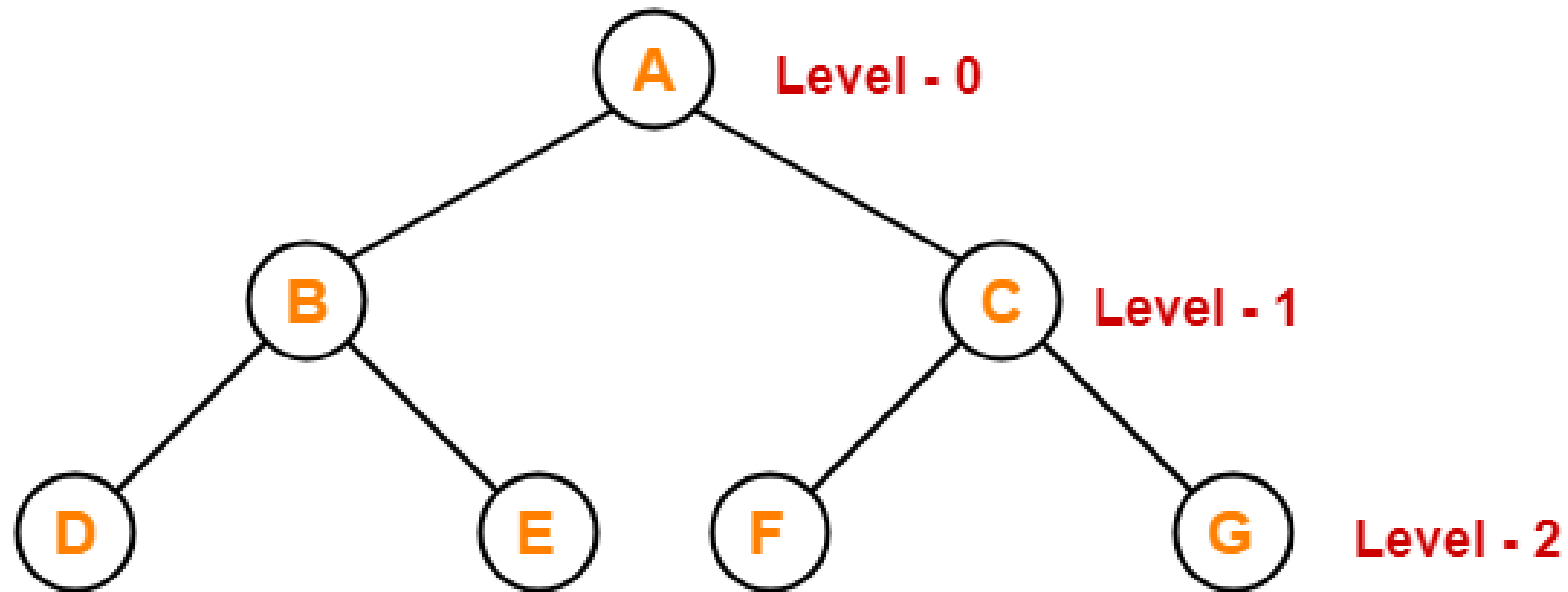
► P4: Maximum number of nodes at any level 'L' in a binary tree = 2^L

→ Maximum number of nodes at level-2 in a binary tree

→ $= 2^2$

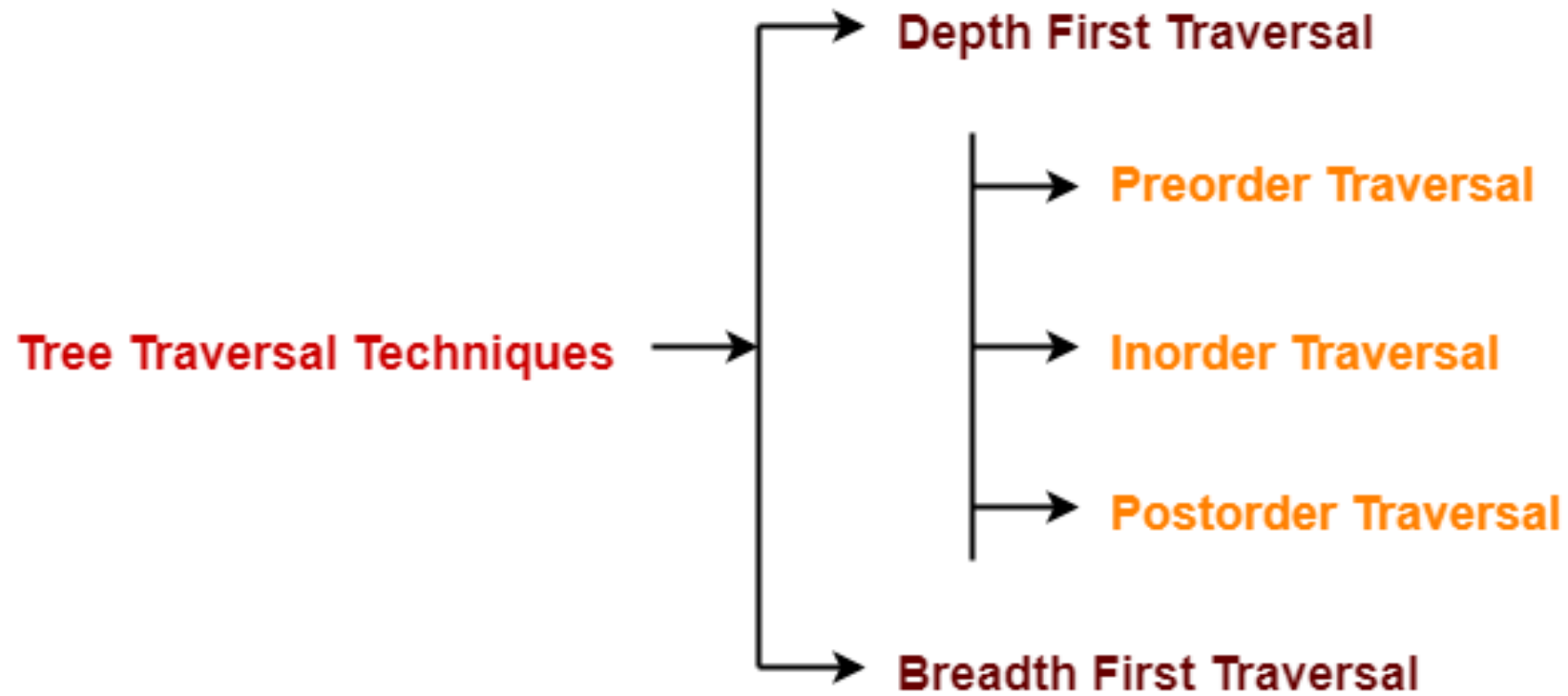
→ $= 4$

→ Thus, in a binary tree, maximum number of nodes that can be present at level-2 = 4.



Tree Traversal-

- ▶ Tree Traversal refers to the process of visiting each node in a tree data structure exactly once.



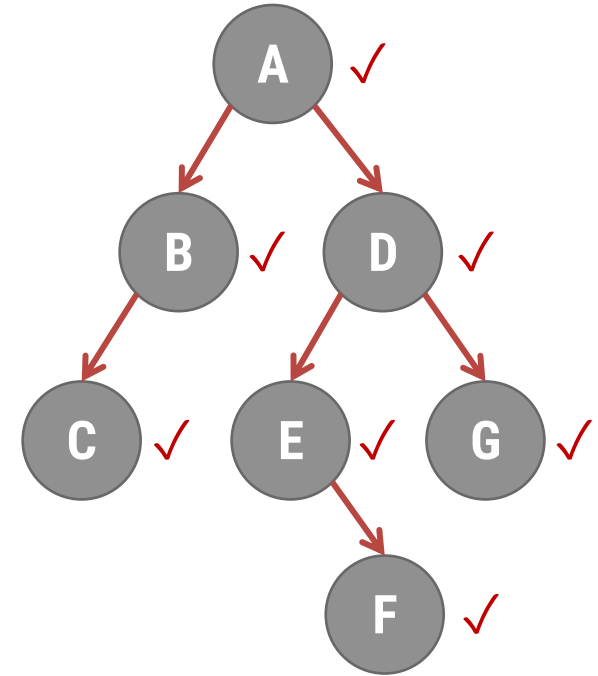
Preorder Traversal

► Preorder (**Root** → **Left** → **Right**)

1. **Process** the **root node**
2. **Traverse** the **left subtree** in preorder
3. **Traverse** the **right subtree** in preorder

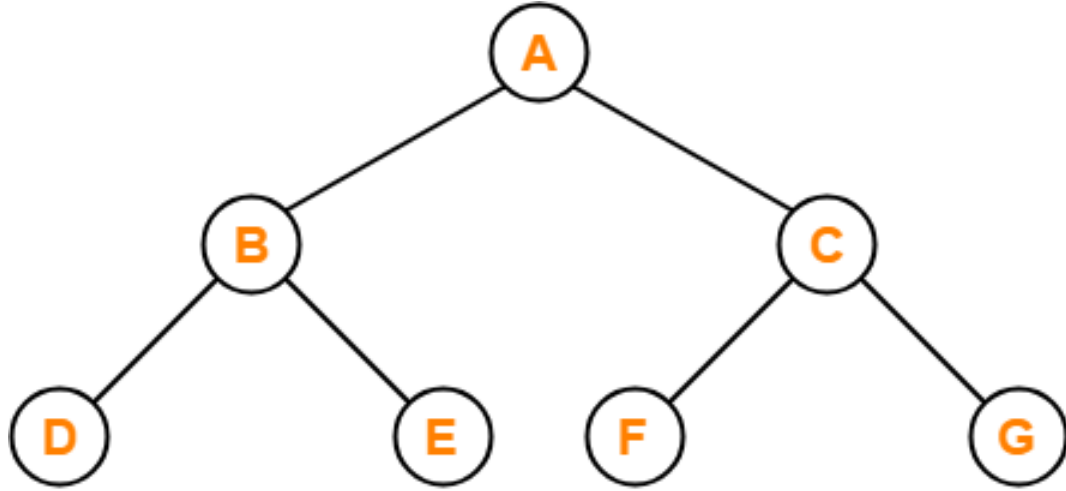
► If particular **subtree is empty** (i.e., node has no left or right descendant) the traversal is performed by **doing nothing**.

► In other words, a **null subtree** is **considered to be fully traversed** when it is encountered.

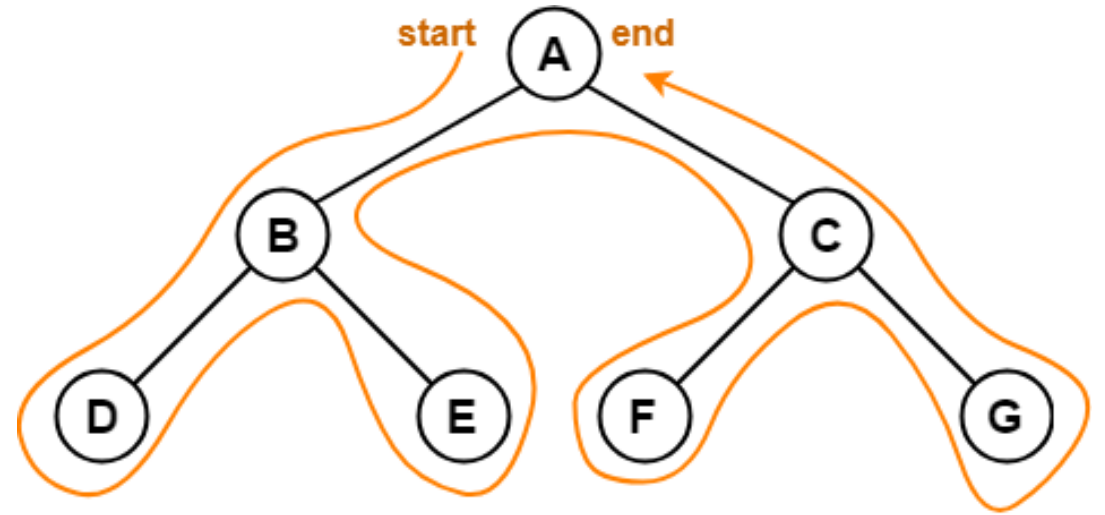


Preorder traversal of a given tree as

A B C D E F G



Preorder Traversal : A , B , D , E , C , F , G



Preorder Traversal : A , B , D , E , C , F , G

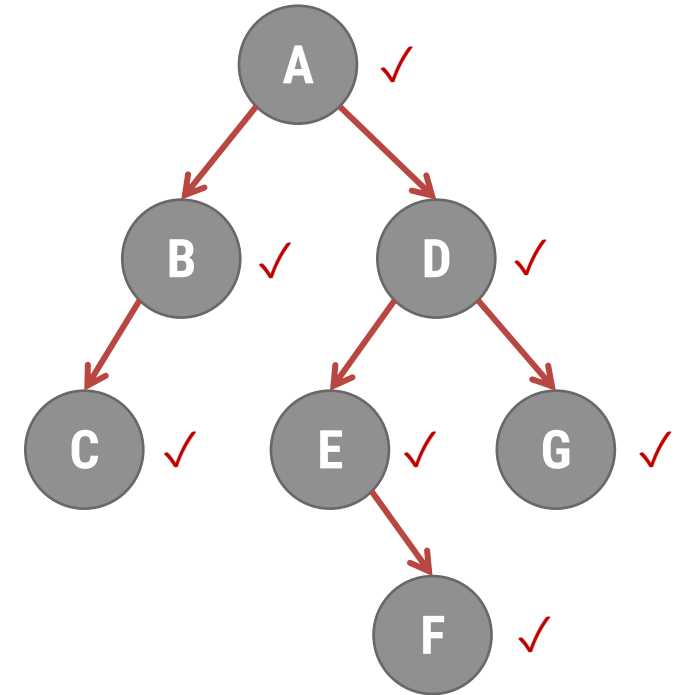
Applications-

- Preorder traversal is used to get prefix expression of an expression tree.
- Preorder traversal is used to create a copy of the tree.

Inorder Traversal

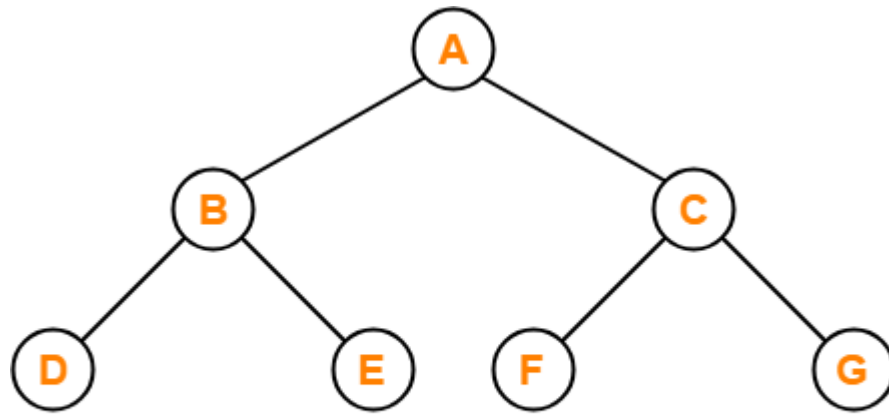
► Inorder (**Left** → **Root** → **Right**)

1. **Traverse** the **left subtree** in Inorder
2. **Process** the **root node**
3. **Traverse** the **right subtree** in Inorder

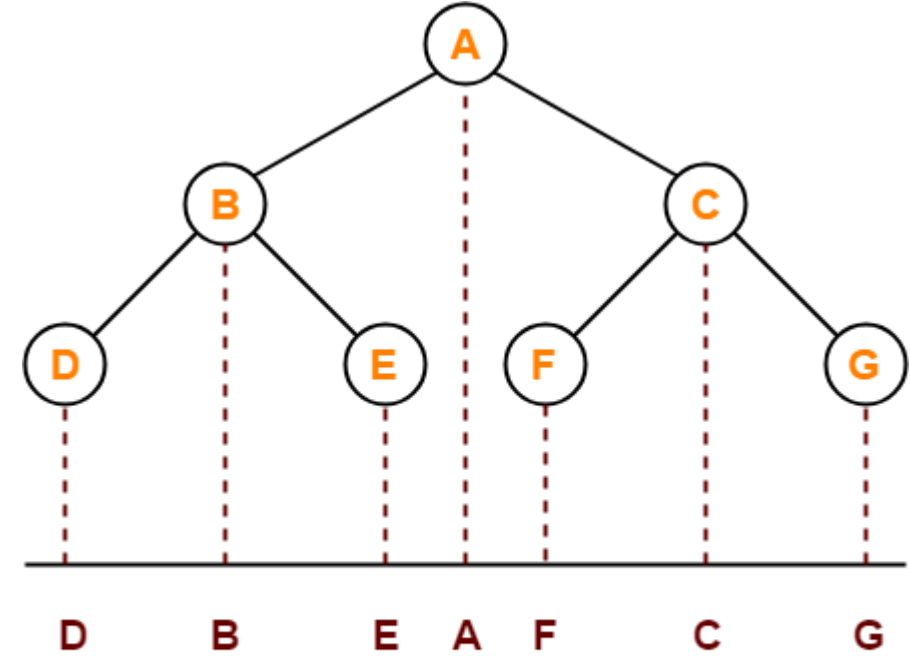


Inorder traversal of a given tree as

C B A E F D G



Inorder Traversal : D , B , E , A , F , C , G



Inorder Traversal : D , B , E , A , F , C , G

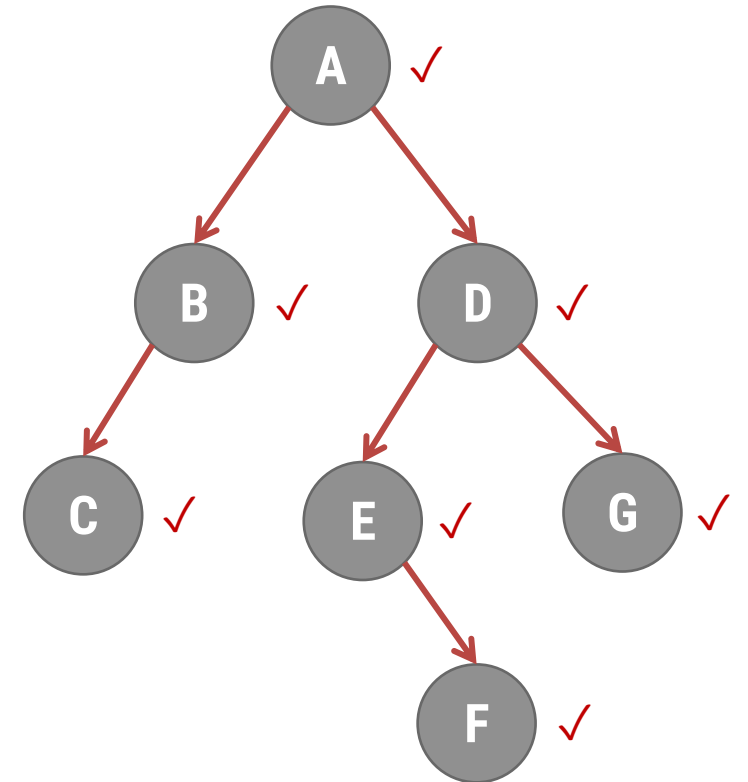
Application-

- Inorder traversal is used to get infix expression of an expression tree.

Postorder Traversal

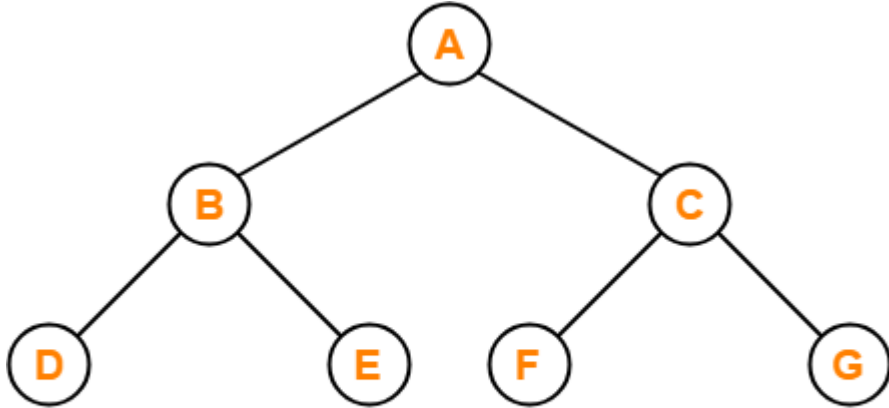
► Postorder (**Left** → **Right** → **Root**)

1. **Traverse** the **left subtree** in Postorder
2. **Traverse** the **right subtree** in Postorder
3. **Process** the **root node**

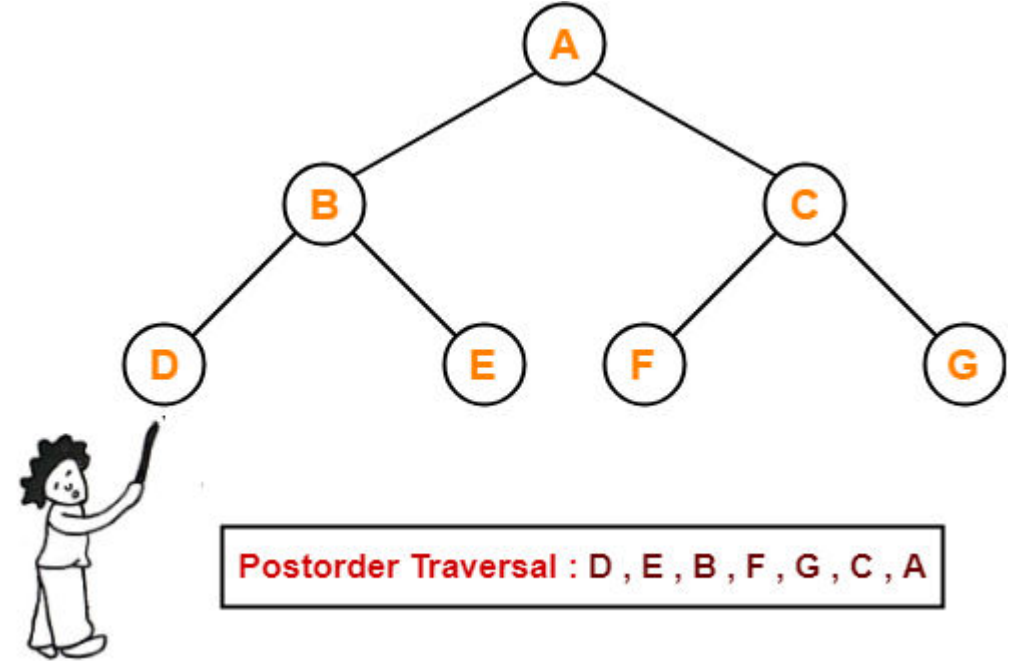


Postorder traversal of a given tree as

C B F E G D A



Postorder Traversal : D , E , B , F , G , C , A

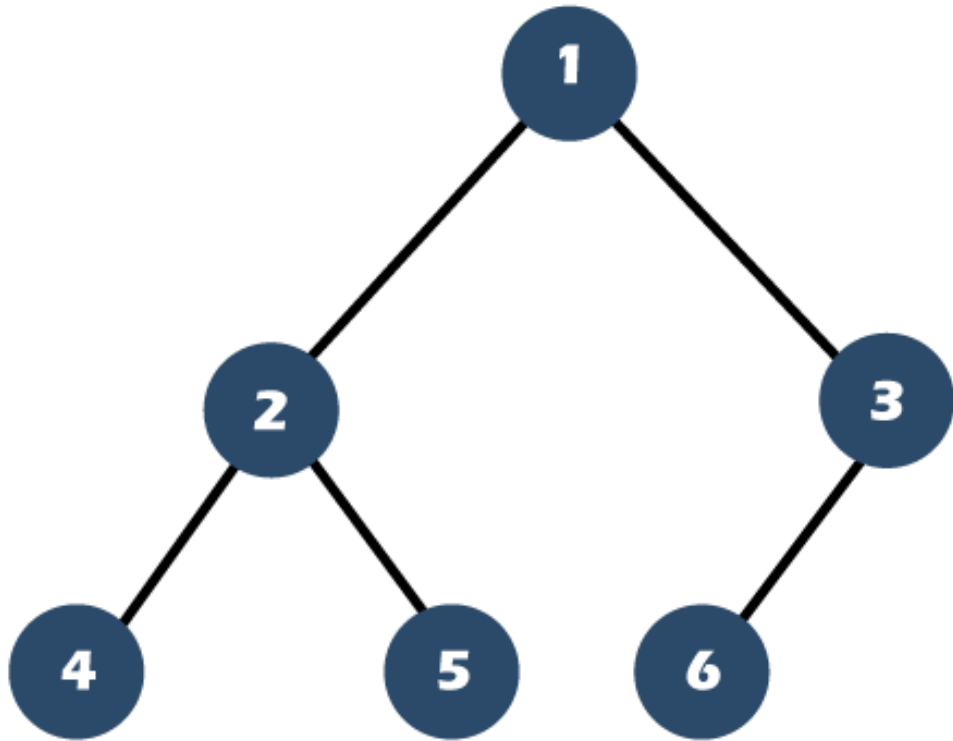


Postorder Traversal : D , E , B , F , G , C , A

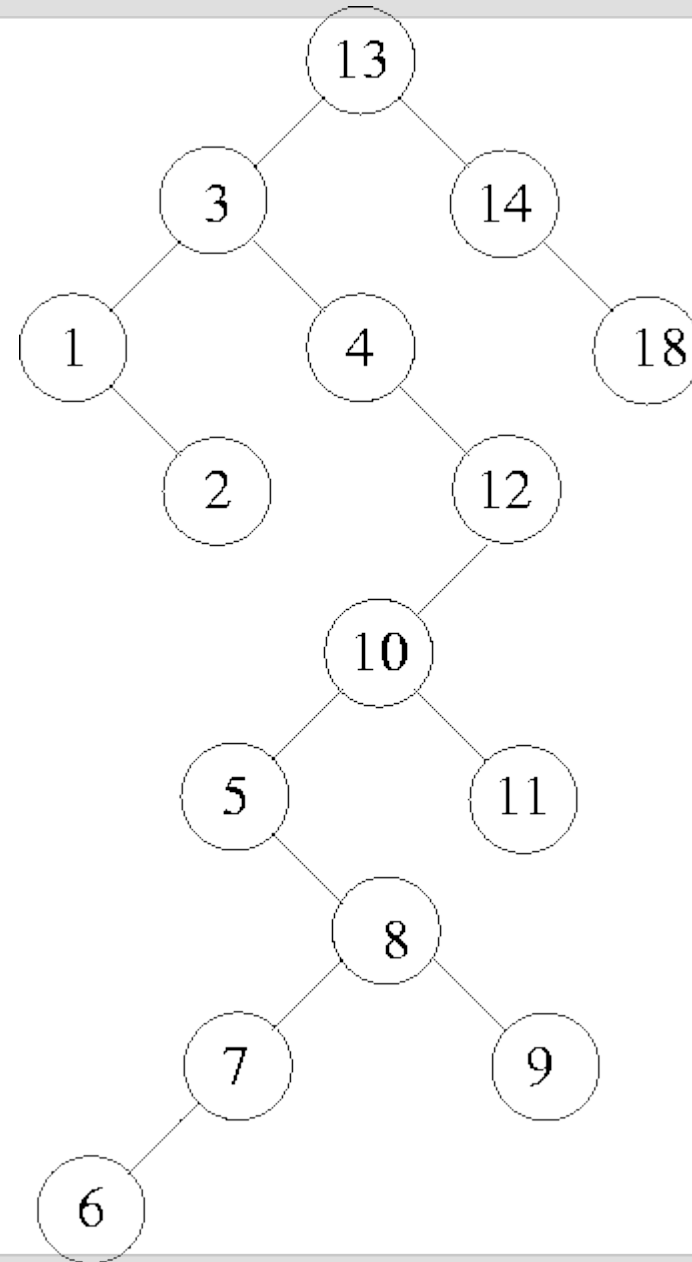
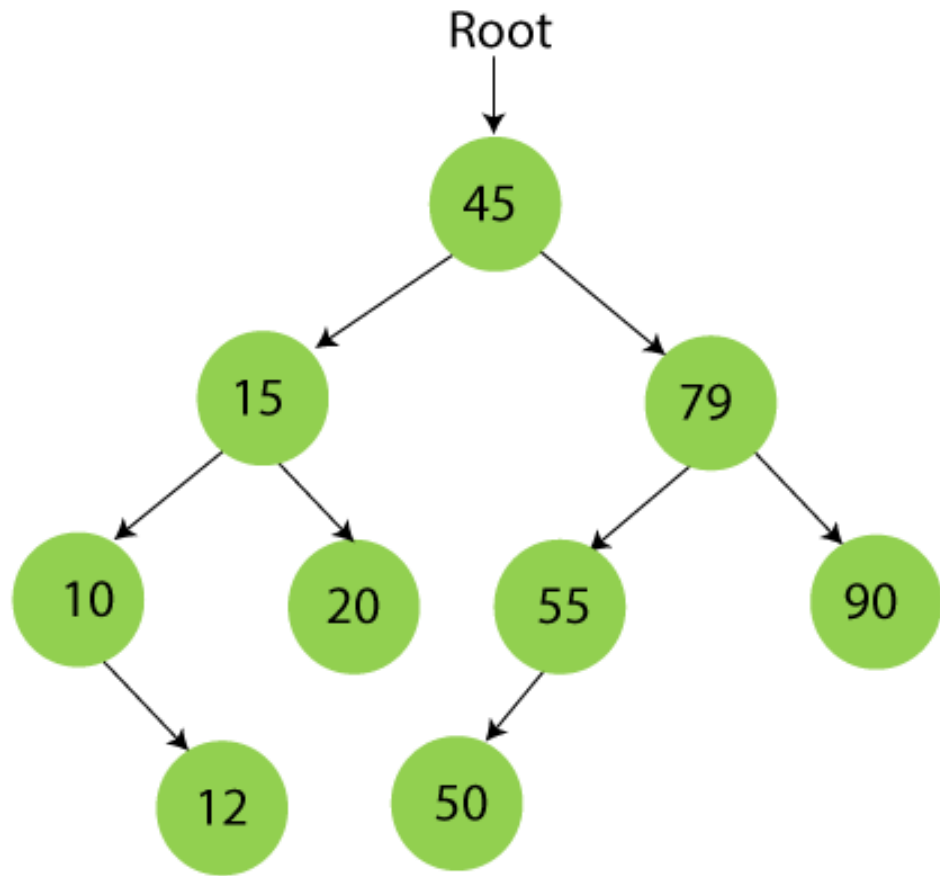
Applications-

- Postorder traversal is used to get postfix expression of an expression tree.
- Postorder traversal is used to delete the tree.
- This is because it deletes the children first and then it deletes the parent.

Exercise:

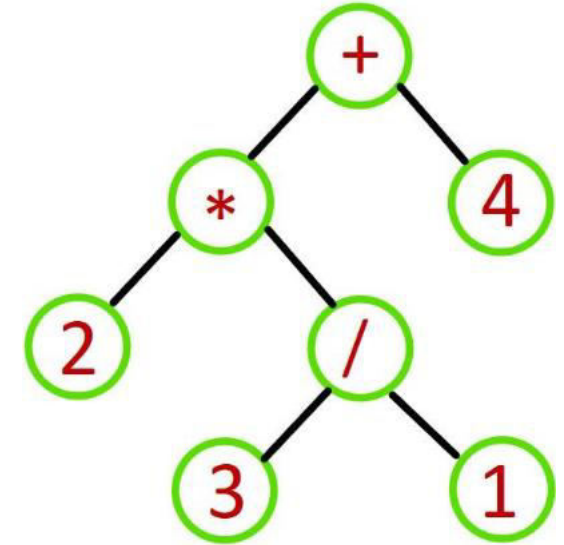


Exercise:



Example: Expression tree

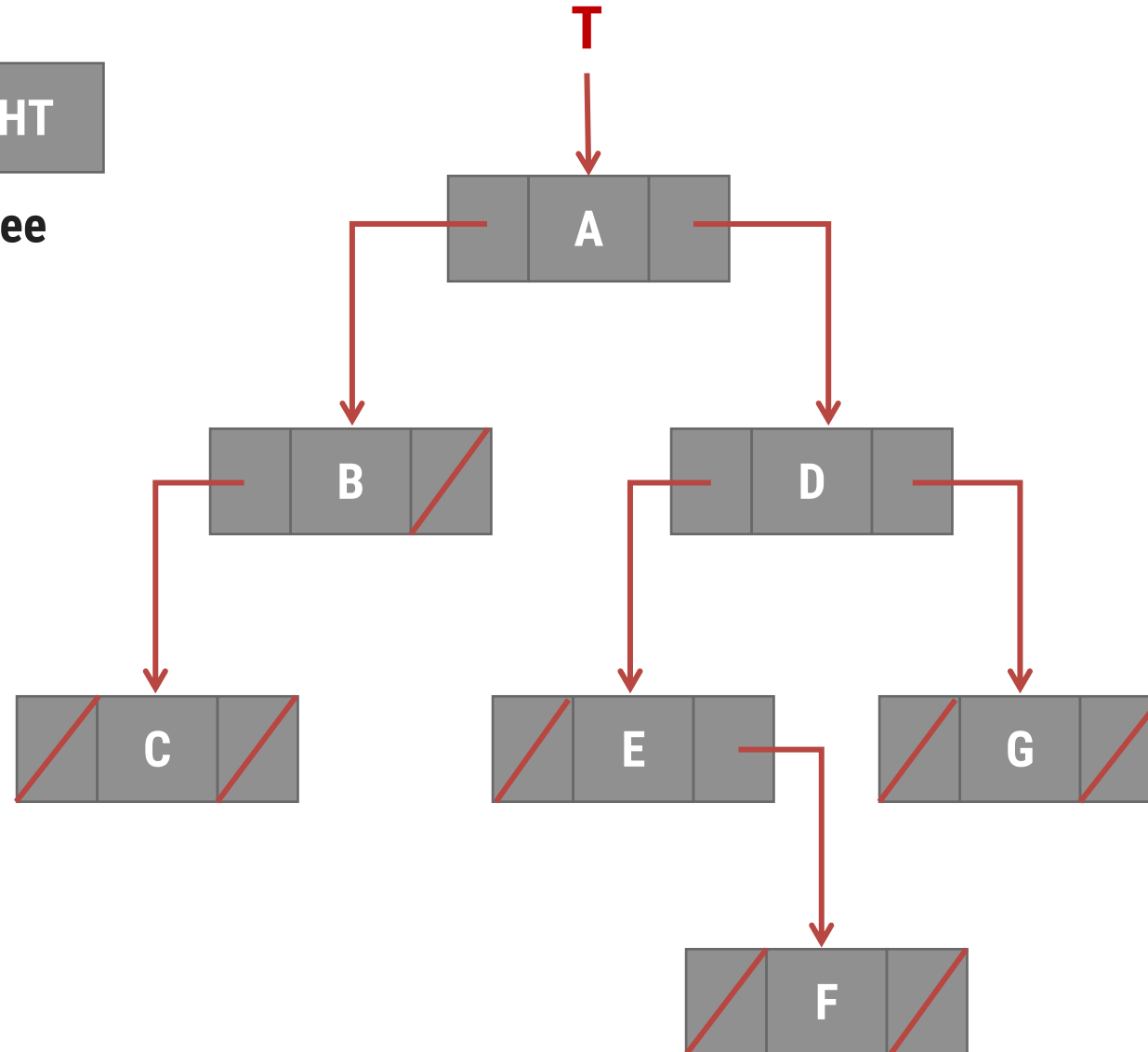
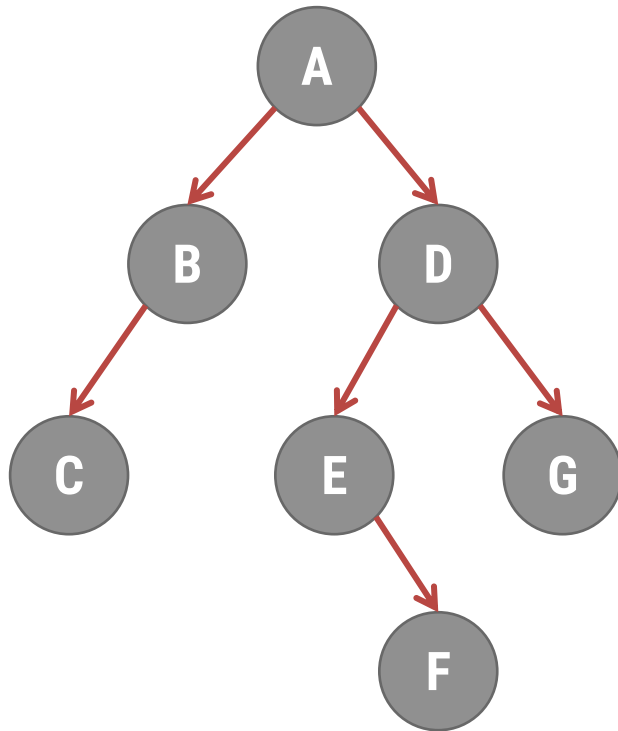
► Infix: $2 * (3 / 1) + 4$



Linked Representation of Binary Tree



Typical node of Binary Tree



Algorithm of Binary Tree Traversal

- ▶ Preorder Traversal - Procedure: RPREORDER(T)
- ▶ Inorder Traversal - Procedure: RINORDER(T)
- ▶ Postorder Traversal - Procedure: RPOSTORDER(T)

Procedure: RPREORDER(T)

- ▶ This procedure **traverses the tree** in **preorder**, in a recursive manner.
- ▶ **T is root node address** of given binary tree
- ▶ Node structure of binary tree is described as below

LEFT	DATA	RIGHT
------	------	-------

Typical node of Binary Tree

```
void pre-order(root):  
    if root != NULL:  
        print(root->data)  
        pre-order(root->left)  
        pre-order(root->right)
```

Procedure: RINORDER(T)

- ▶ This procedure **traverses the tree** in **InOrder**, in a recursive manner.
- ▶ **T is root node address** of given binary tree.
- ▶ Node structure of binary tree is described as below.

LEFT	DATA	RIGHT
------	------	-------

Typical node of Binary Tree

```
void pre-order(root):  
    if root != NULL:  
        pre-order(root->left)  
        print(root->data)  
        pre-order(root->right)
```

Procedure: RPOSTORDER(T)

- ▶ This procedure **traverses the tree** in **PostOrder**, in a recursive manner.
- ▶ **T is root node address** of given binary tree.
- ▶ Node structure of binary tree is described as below.

LEFT	DATA	RIGHT
------	------	-------

Typical node of Binary Tree

```
void pre-order(root):  
    if root != NULL:  
        pre-order(root->left)  
        pre-order(root->right)  
        print(root->data)
```

Construct Binary Tree from Traversal

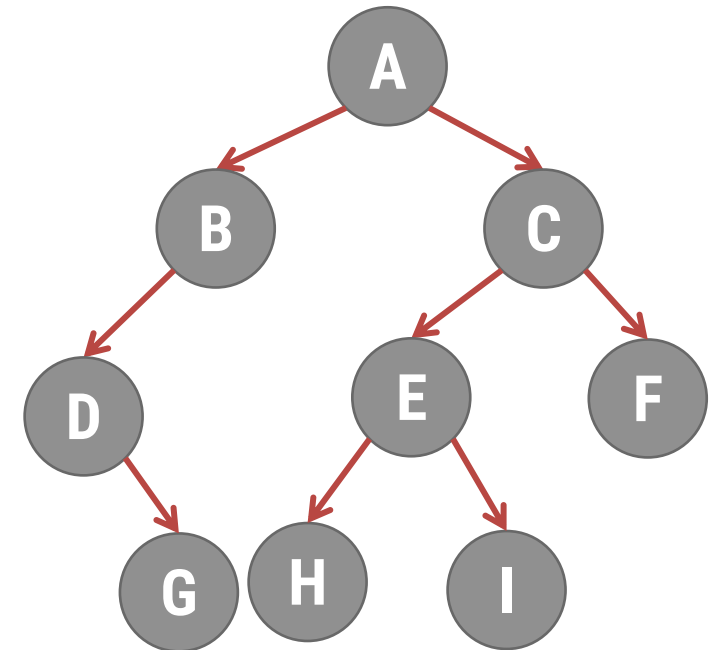
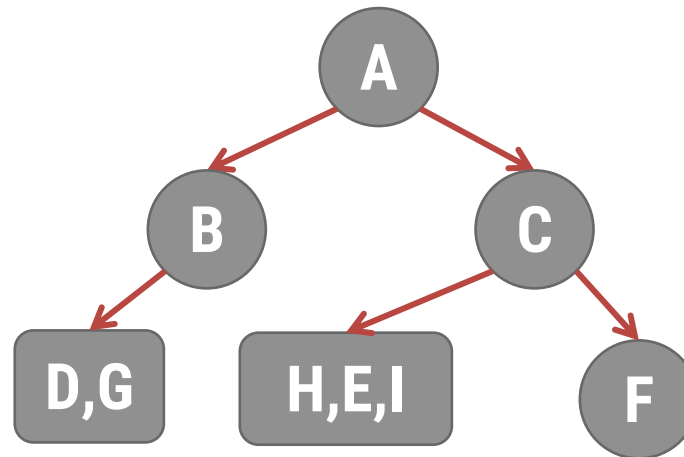
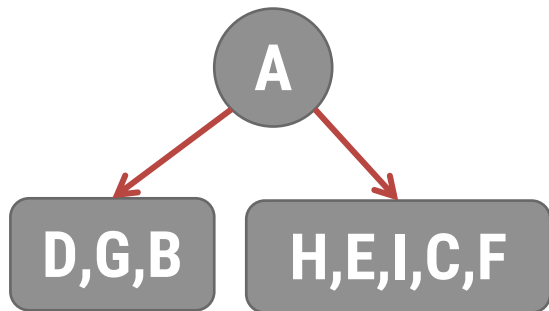
Construct a Binary tree from the given **Inorder** and **Postorder** traversals

Inorder : D G B A H E I C F
Postorder : G D B H I E F C A

- Step 1: Find the root node
 - Preoder Traversal – first node is root node
 - Postoder Traversal last node is root node
- Step 2: Find Left & Right Sub Tree
 - Inorder traversal gives Left and right sub tree

Postorder : G D B H I E F C **A**

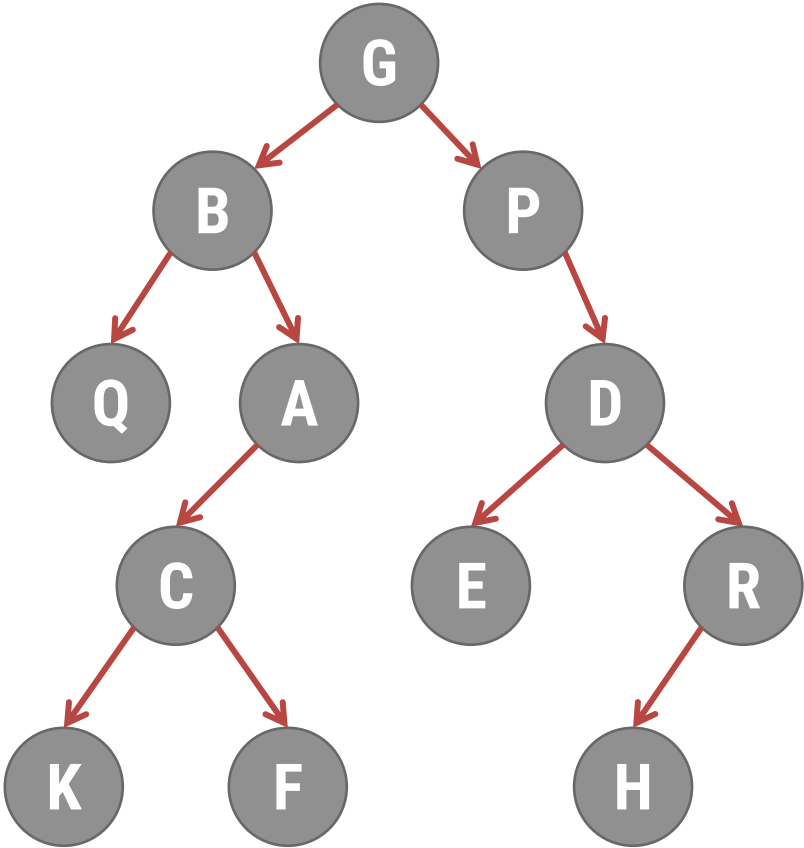
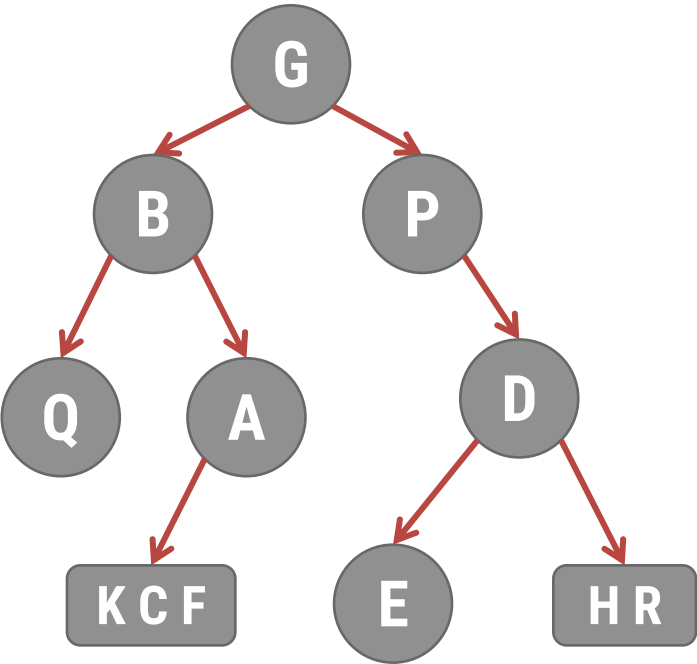
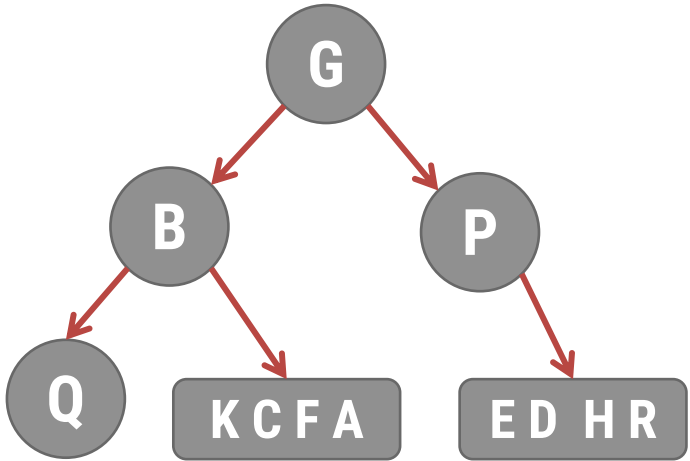
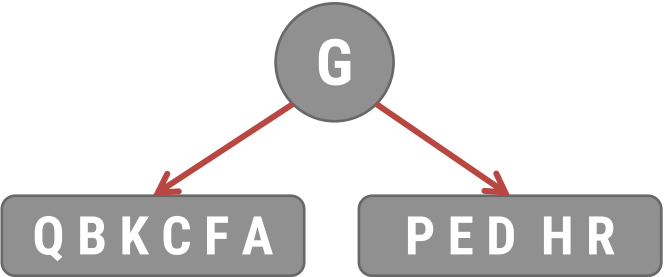
Inorder : D G B **A** H E I C F



Construct Binary Tree from Traversal

Preorder : **G** B Q A C K F P D E R H

Inorder : Q B K C F A **G** P E D H R



Exercise

The preorder traversal sequence of a binary search tree is-

30 , 20 , 10 , 15 , 25 , 23 , 39 , 35 , 42

What one of the following is the postorder traversal sequence of the same tree?

- a) 10 , 20 , 15 , 23 , 25 , 35 , 42 , 39 , 30
- b) 15 , 10 , 25 , 23 , 20 , 42 , 35 , 39 , 30
- c) 15 , 20 , 10 , 23 , 25 , 42 , 35 , 39 , 30
- d) 15 , 10 , 23 , 25 , 20 , 35 , 42 , 39 , 30

Construct a tree

Q. 1: Inorder: [40, 20, 50, 10, 60, 30], Preorder: [10, 20, 40, 50, 30, 60]

Q. 2: Inorder: [9, 3, 15, 20, 7], Preorder: [3, 9, 20, 15, 7]

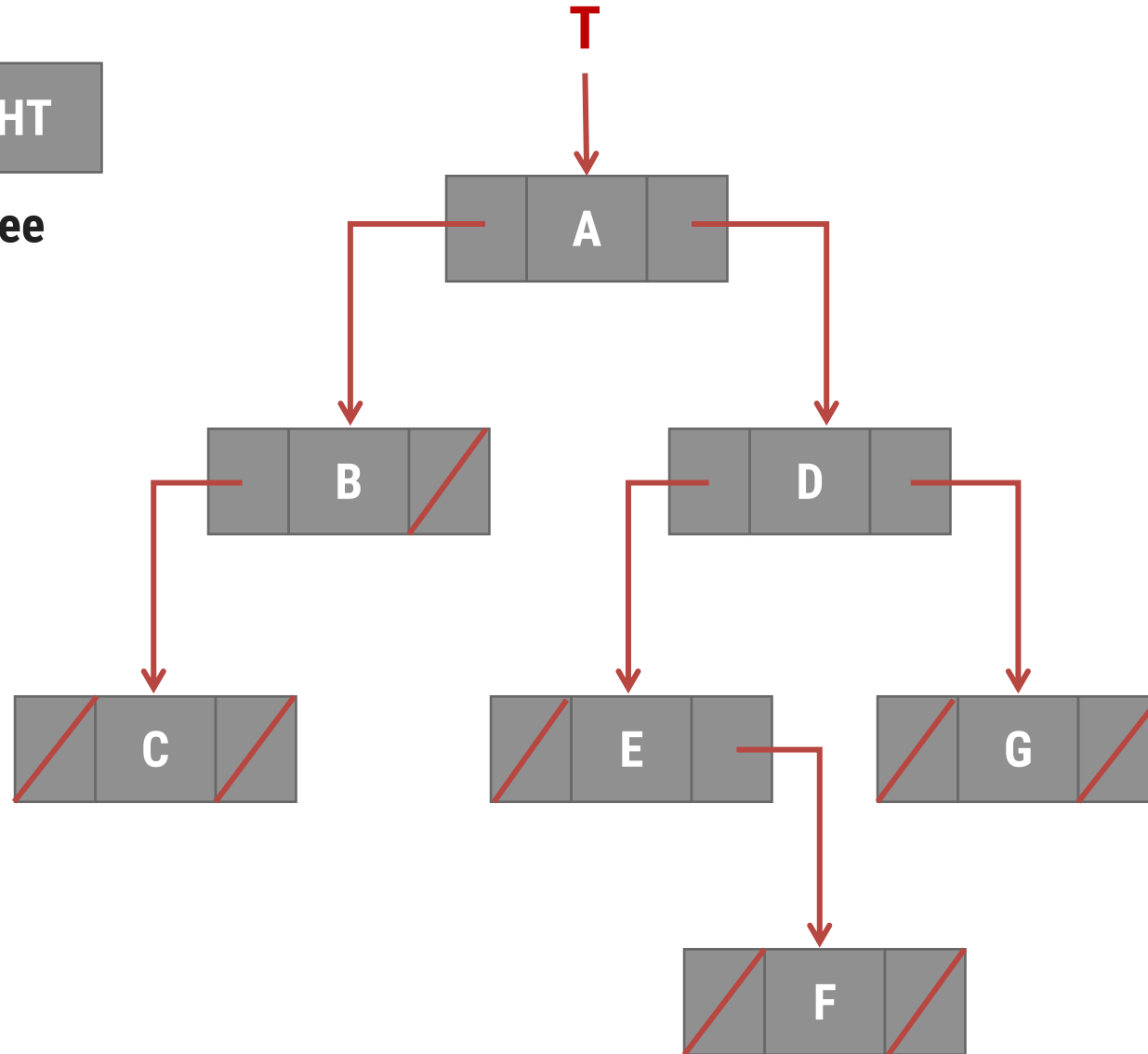
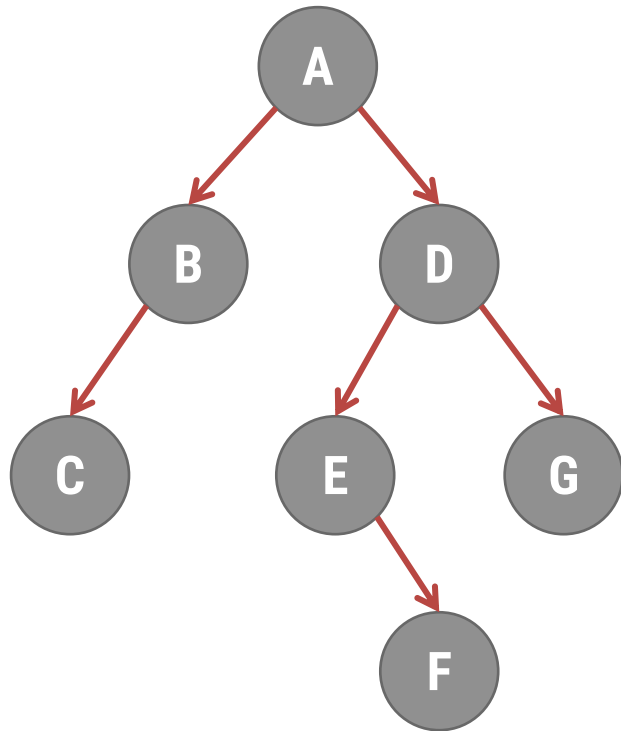
Q.3: Inorder: [10, 4, 18, 22, 9], Postorder: [10, 18, 9, 22, 4]

Q.4: Inorder: [4, 8, 2, 5, 1, 6, 3, 7], Postorder: [8, 4, 5, 2, 6, 7, 3, 1]

Linked Representation of Binary Tree



Typical node of Binary Tree



Threaded Binary Tree

- ▶ The **wasted NULL** links in the binary tree storage representation can be **replaced by threads**
- ▶ A binary **tree** is **threaded according** to particular **traversal order**. e.g.: Threads for the inorder traversals of tree are pointers to its higher nodes, for this traversal order
- ▶ **In-Threaded Binary Tree**
 - If left link of **node P is null**, then this link is **replaced by** the **address of its predecessor**
 - If right link of **node P is null**, then this link is **replaced by** the **address of its successor**
- ▶ Because the left or right **link** of a **node** can denote **either structural link** or **a thread**, we must somehow be able to distinguish them

Threaded Binary Tree

- ▶ **Method 1:-** Represent **thread a Negative address**
- ▶ **Method 2:-** To have a **separate Boolean flag** for each of left and right pointers, node structure for this is given below

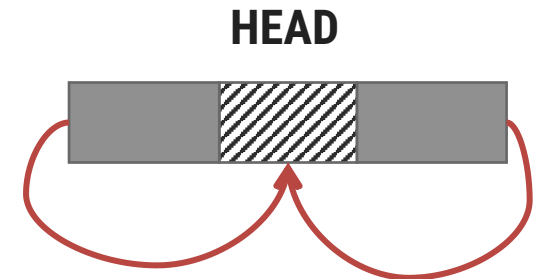
LEFT	LTHREAD	DATA	RTHREAD	RIGHT
------	---------	------	---------	-------

Typical node of Threaded Binary Tree

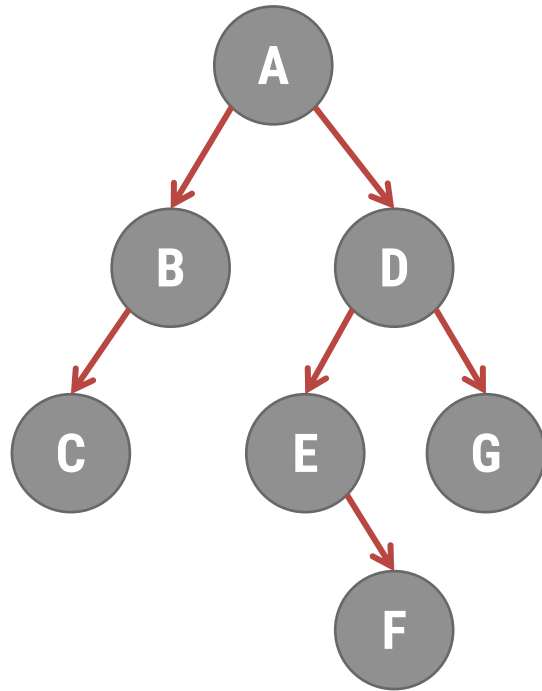
- **LTHREAD = true** = Denotes leaf thread link
- **LTHREAD = false** = Denotes leaf structural link
- **RTHREAD = true** = Denotes right threaded link
- **RTHREAD = false** = Denotes right structural link

Head node is simply another node which serves as the predecessor and successor of first and last tree nodes.

Tree is attached to the left branch of the head node.

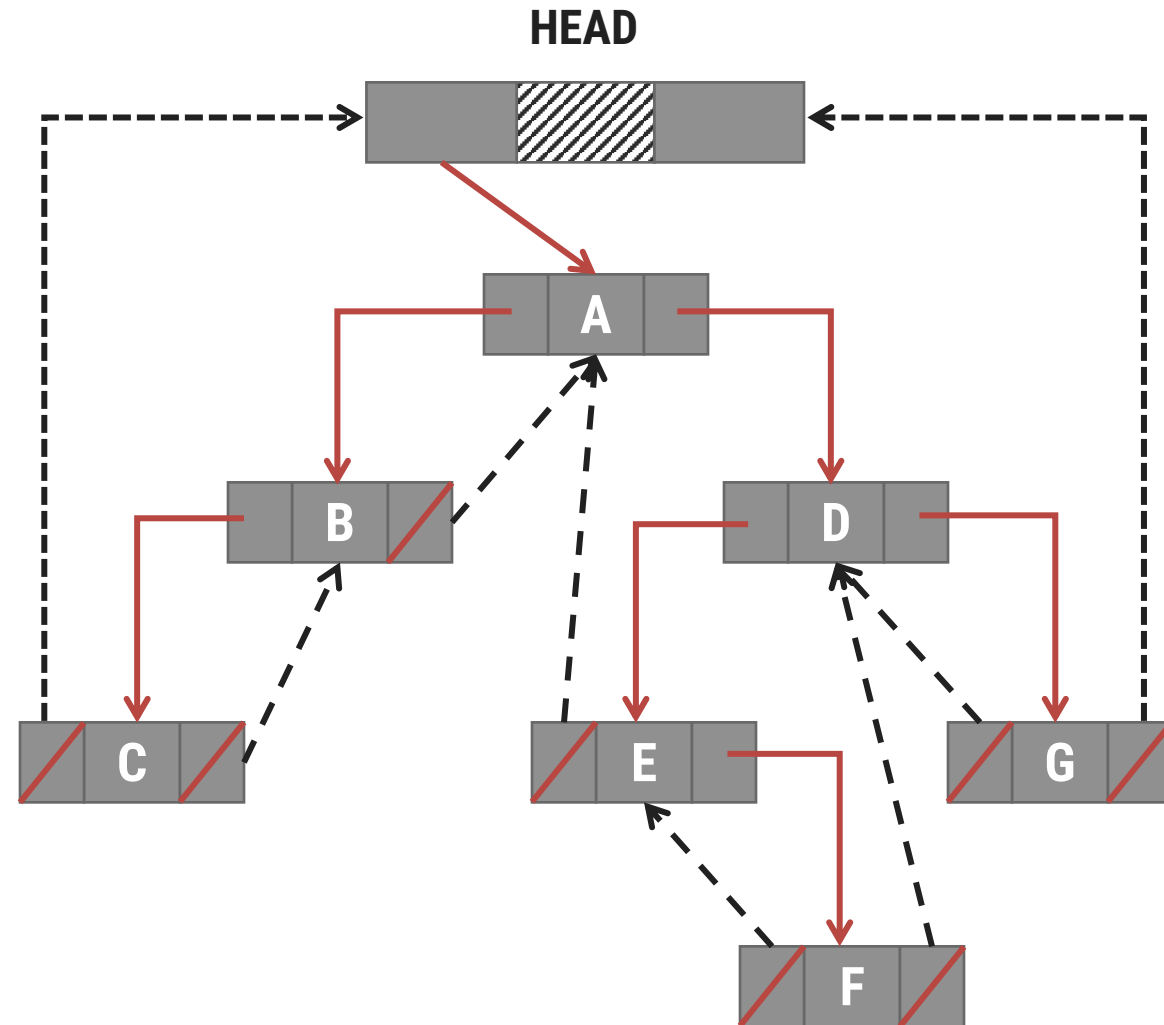


Threaded Binary Tree



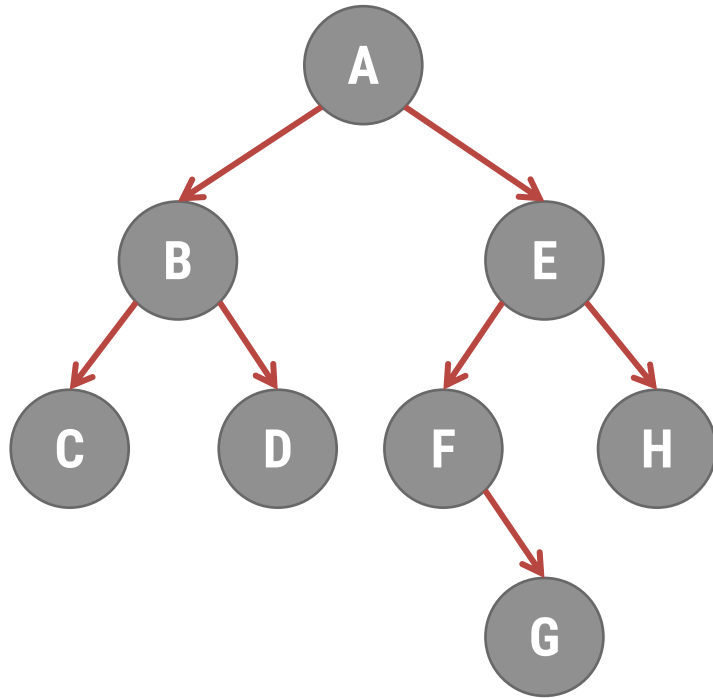
Inorder Traversal

C B A E F D G



Fully In-Threaded Binary Tree

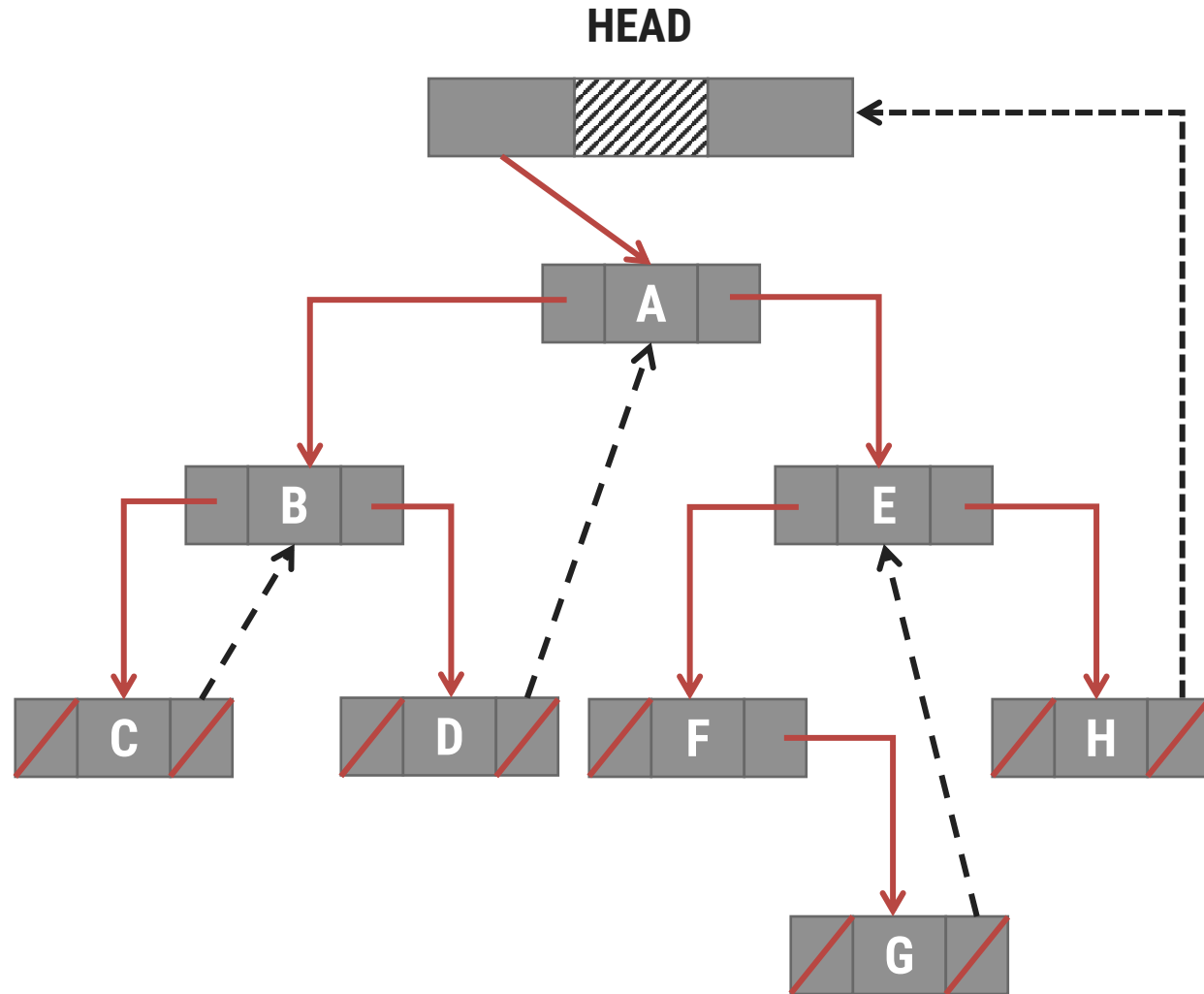
Threaded Binary Tree



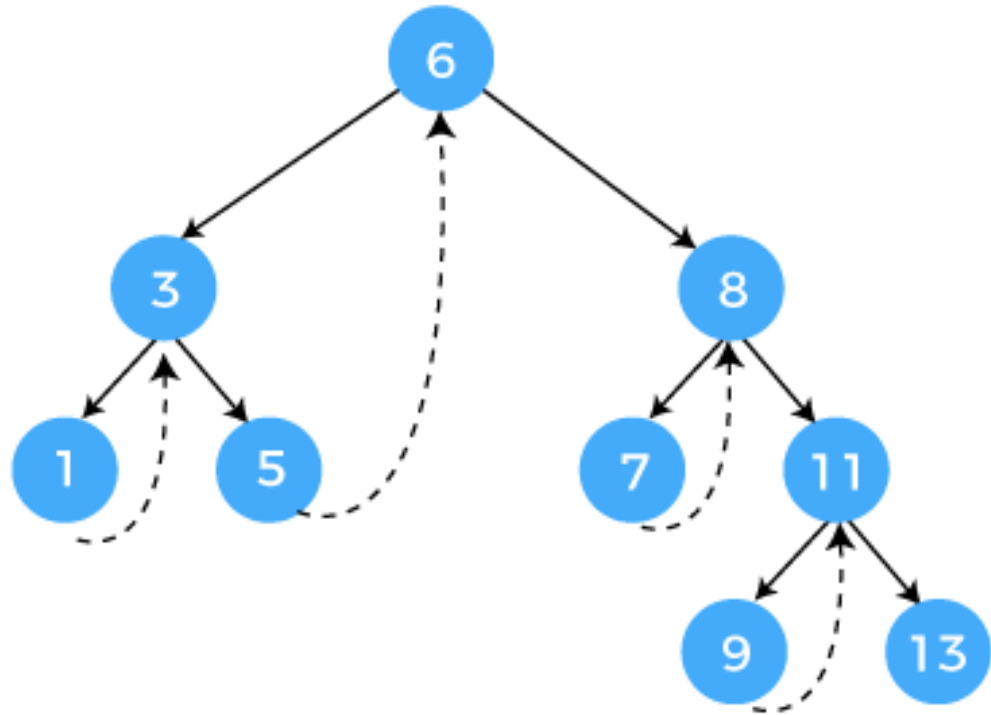
Inorder Traversal

C B D A F G E H

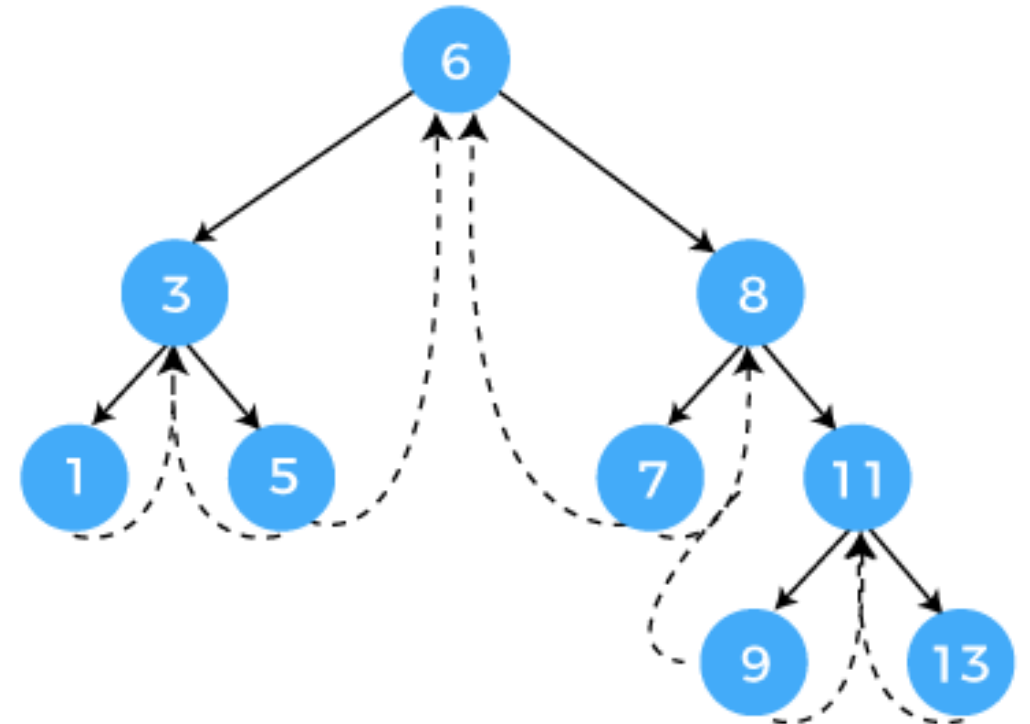
Construct Right In-Threaded Binary Tree of given Tree



One way and Two way threaded binary tree



Single Threaded Binary Tree



Double Threaded Binary Tree

Advantages of Threaded Binary Tree

- ▶ **Inorder traversal is faster** than unthreaded version as stack is not required.
- ▶ **Effectively determines** the **predecessor and successor** for inorder traversal, for unthreaded tree this task is more difficult.
- ▶ **A stack is required** to provide upward pointing information **in binary tree** which **threading provides without stack**.
- ▶ It is possible to **generate successor or predecessor** of any node **without** having over head of **stack** with the help of threading.

Disadvantages of Threaded Binary Tree

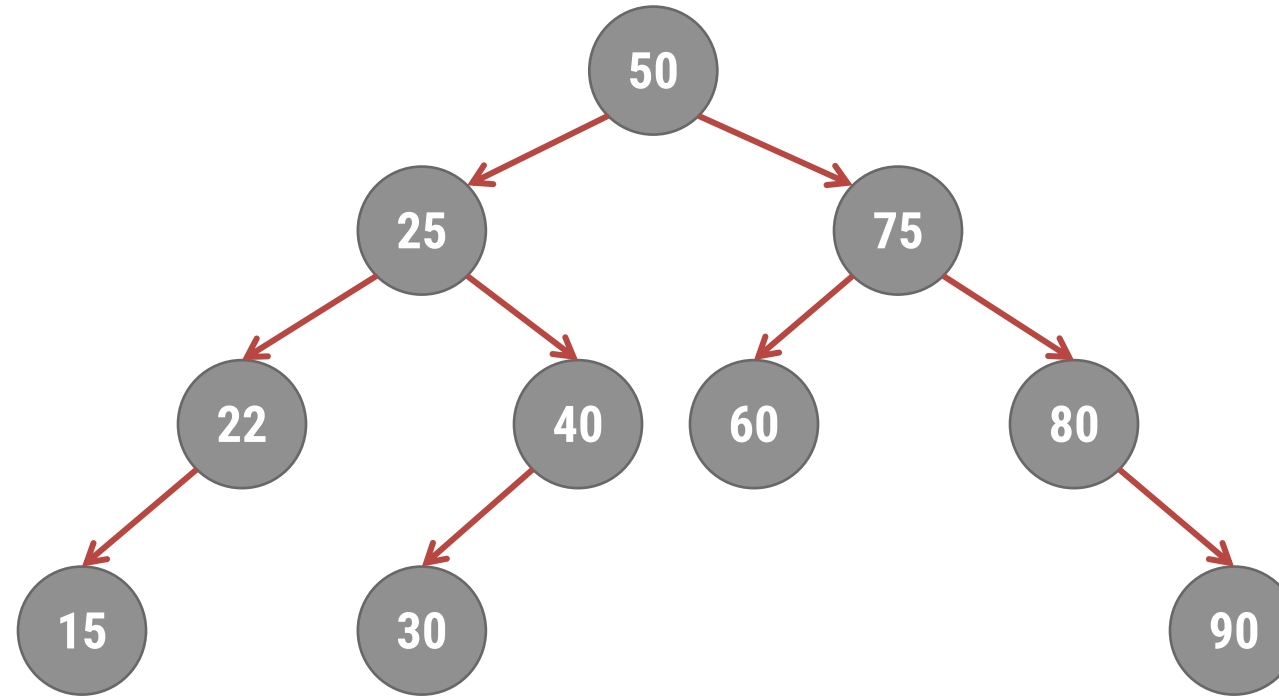
- ▶ Threaded trees are **unable to share common sub trees**.
- ▶ If **Negative addressing is not permitted** in programming language, **two additional fields are required**.
- ▶ **Insertion** into and **deletion** from threaded binary tree are **more time consuming** because both thread and structural link must be maintained.

Binary Search Tree (BST)

- ▶ A **binary search tree** is a **binary tree** in which **each node** possessed a key that **satisfy** the **following conditions**
 1. All **key** (if any) in **the left sub tree** of the root **precedes the key** in the **root**
 2. The **key in the root precedes** all **key** (if any) in the **right sub tree**
 3. The **left and right sub trees** of the root are again **search trees**

Construct Binary Search Tree (BST)

Construct binary search tree for the following data
50 , 25 , 75 , 22 , 40 , 60 , 80 , 90 , 15 , 30



Construct binary search tree for the following data
10, 3, 15, 22, 6, 45, 65, 23, 78, 34, 5

Insert

```
INSERT(root, item)
```

```
new = newNode(item)//Create a new node
```

```
if root == NULL:
```

```
    return new
```

```
If root→data < item
```

```
    root→right = INSERT(root→right, item)
```

```
If root→data > item
```

```
    root→left = INSERT(root→left, item)
```

```
return root
```

```
struct Node* newNode(int item)
```

```
    temp = (struct Node*)malloc(sizeof(struct Node));
```

```
    temp->data = item;
```

```
    temp->left = temp->right = NULL;
```

```
    return temp;
```

Search a node in Binary Search Tree

- ▶ To search for target value.
- ▶ We first compare it with the key at root of the tree.
- ▶ If it is not same, we go to either Left sub tree or Right sub tree as appropriate and repeat the search in sub tree.
- ▶ If we have **In-Order List** & we want to search for specific node it requires **$O(n)$ time**.
- ▶ In case of **Binary tree** it requires **$O(\log_2 n)$** time to search a node.

Algorithm: Search

Search (root, item)

if (root==NULL)

“Not found”

if (item == root → data)

“Element found”

return root

else if (item < root → data)

return Search(root → left, item)

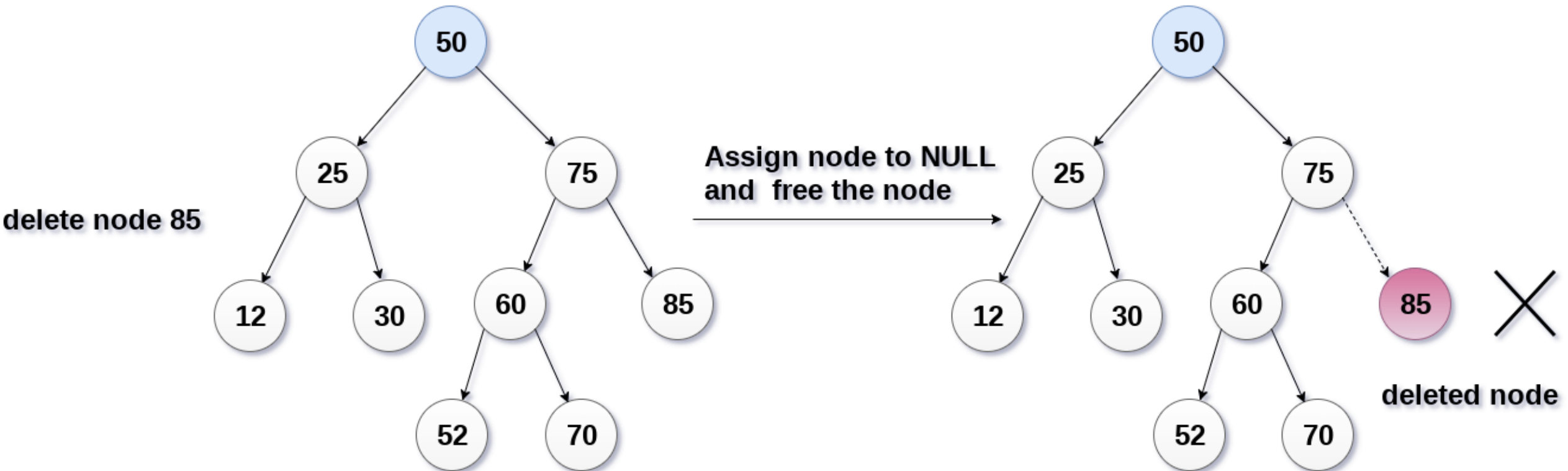
else

return Search(root → right, item)

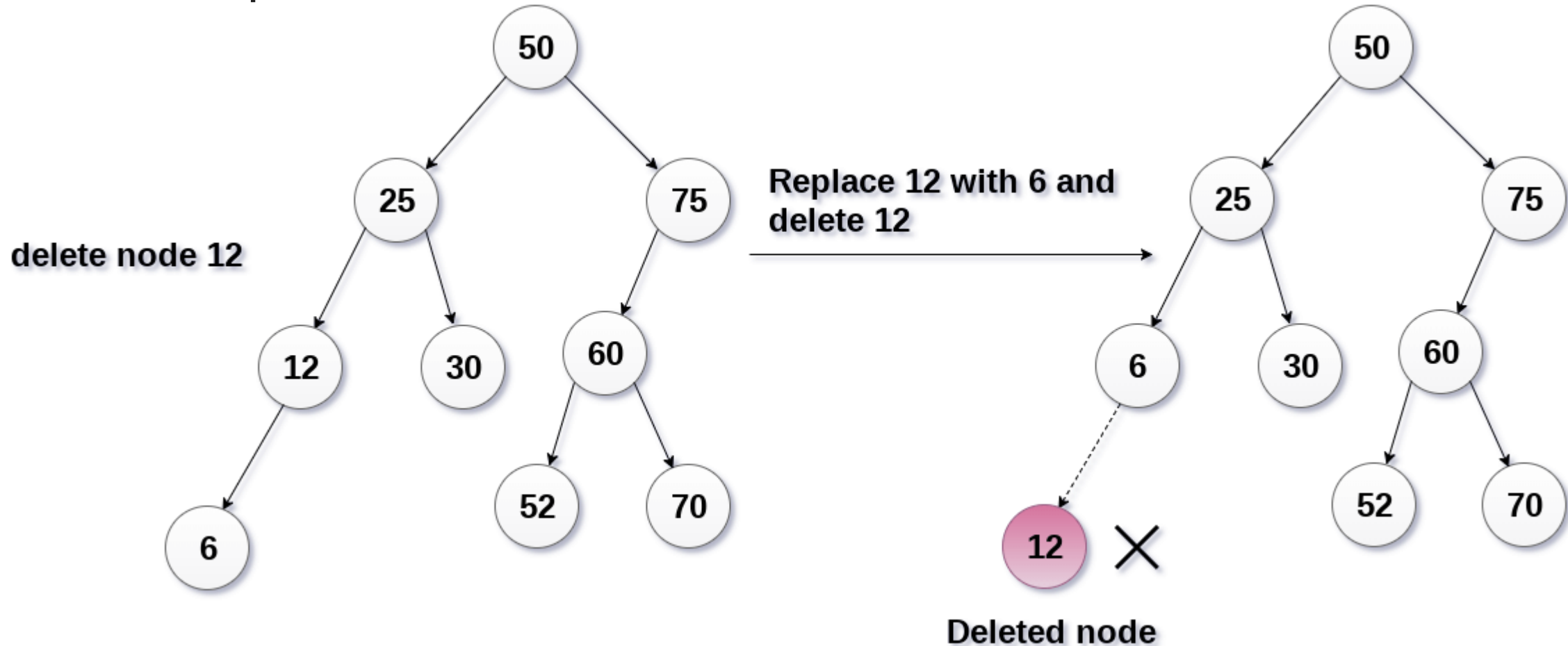
- In a balanced BST, the time complexity for searching an element is $O(\log(n))$, where n is the number of nodes in the tree. This efficiency is achieved by halving the search space at each level as we compare the target value with the current node's value and proceed left or right.
- In an unbalanced BST, the worst-case time complexity for searching can be $O(n)$ if the tree is completely skewed, as each node needs to be traversed in a linear fashion.

Deletion

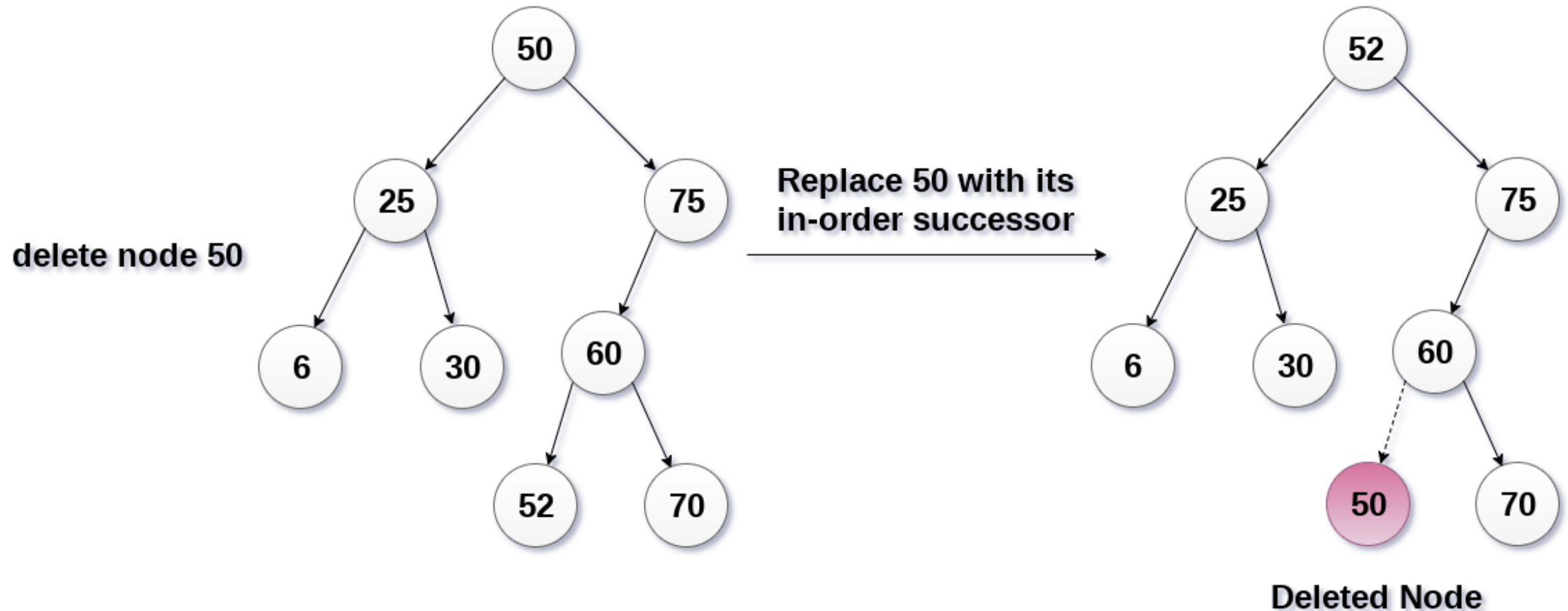
1. The node to be deleted is a leaf node: Replace the leaf node with the NULL and simple free the allocated space.



2. The node to be deleted has only one child: Replace the node with its child and delete the child node, which now contains the value which is to be deleted. Simply replace it with the NULL and free the allocated space.



3. The node to be deleted has two children: The node which is to be deleted, is replaced with its in-order **successor or predecessor** recursively until the node value (to be deleted) is placed on the leaf of the tree. After the procedure, replace the node with NULL and free the allocated space.



- ▶ If the **node is leaf** (both left and right will be NULL), remove the node directly and free its memory.
- ▶ If the node has **only right child** (left will be NULL), make the node points to the right node and free the node.
- ▶ If the node has **only left child** (right will be NULL), make the node points to the left node and free the node.
- ▶ If the node **has both left and right child**,
 - ➔ 1.find the smallest node in the right subtree. say min
 - ➔ 2.make node->data = min
 - ➔ 3.Again delete the min node.

Algorithm: Delete

DELETE (root, item)

IF root = NULL

 "item not found in the tree"

IF (root→data < item)

 root→right = DELETE(root→right, item)

ELSE IF (root→data > item)

 root→left = DELETE(root→left, item)

ELSE

 IF (root→left == NULL && root→right == NULL)

 free(root)

 return NULL

 ELSE IF (root→left == NULL)

 temp = root→right

 free(root)

 return temp

 ELSE IF (root→right == NULL)

 temp = root→left

 free(root)

 return temp

ELSE

 int min = rightmin(root→right)

 root→data = min

 root→right = DELETE(root→right, min)

return root

Exercise

- ▶ **Construct Binary search tree for the following element.**

47, 12, 75, 88, 90, 73, 57, 1, 85, 50, 62

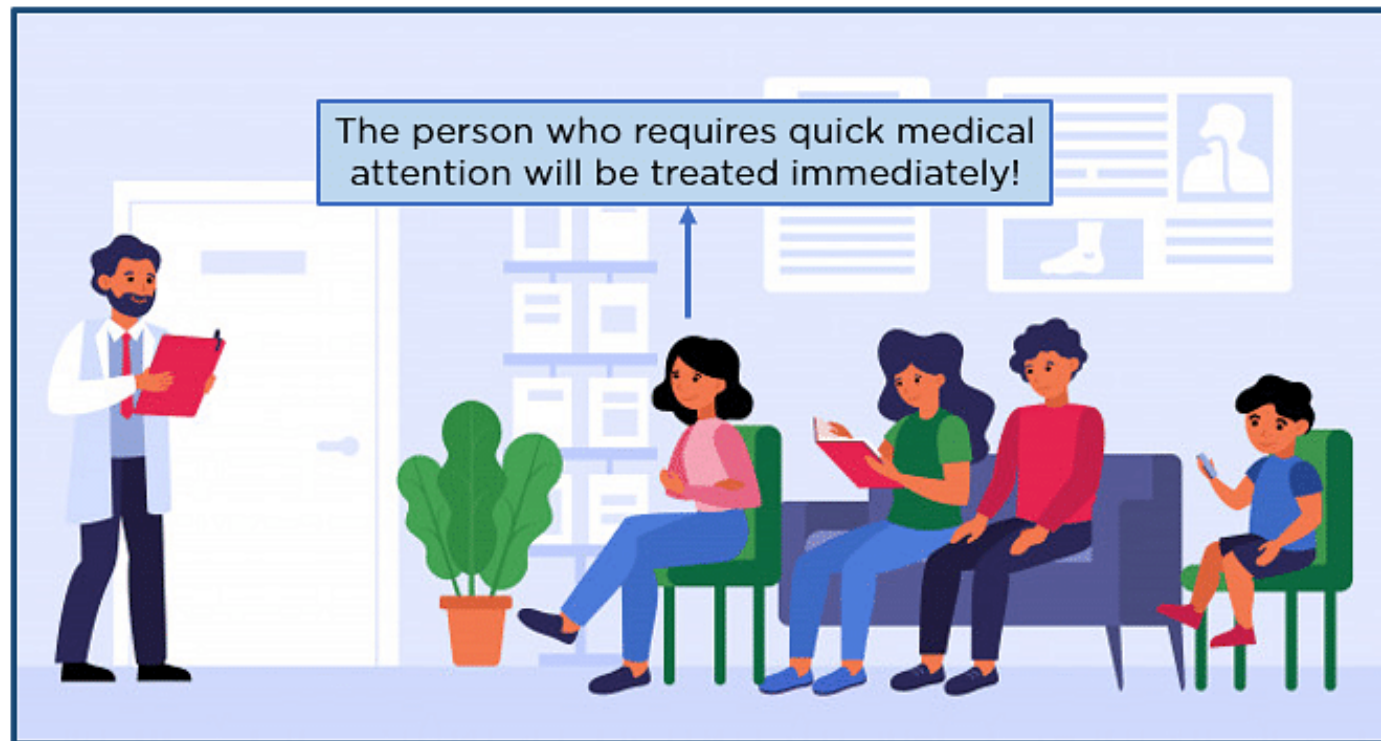
Priority Queue

- ▶ A priority queue serves as a specialized data structure where elements are organized based on their priority values.
- ▶ Ensuring that higher-priority items are processed before lower-priority ones. This organization enables efficient retrieval of the most critical elements.
- ▶ Various implementations, such as arrays, linked lists, heaps, or binary search trees, offer different trade-offs in terms of efficiency and flexibility.
- ▶ Priority queues find extensive applications in scenarios demanding real-time processing, where prompt handling of high-priority tasks is crucial

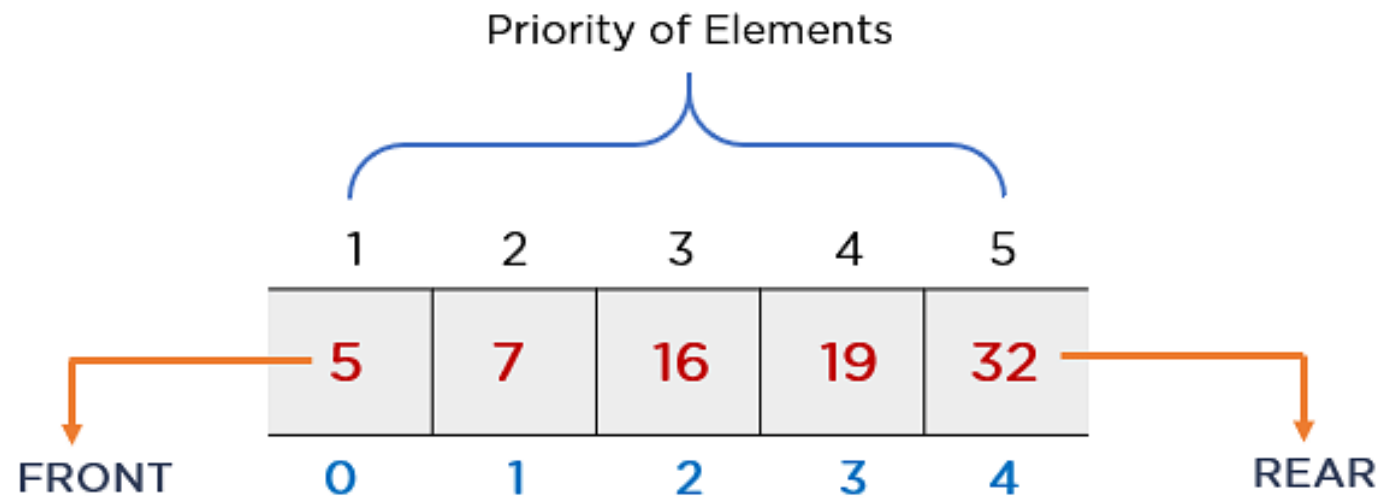
Example

- ▶ The hospital emergency queue is an ideal real-life example of a priority queue. In this queue of patients, the patient with the most critical situation is the first in a queue, and the patient who doesn't need immediate medical attention will be the last. In this queue, the priority depends on the medical condition of the patients.

Hospital Emergency Queue



- ▶ The priority of elements determines the order of removal in a queue, i.e., the element with higher priority will leave the queue first, whereas the element with the lowest priority at last



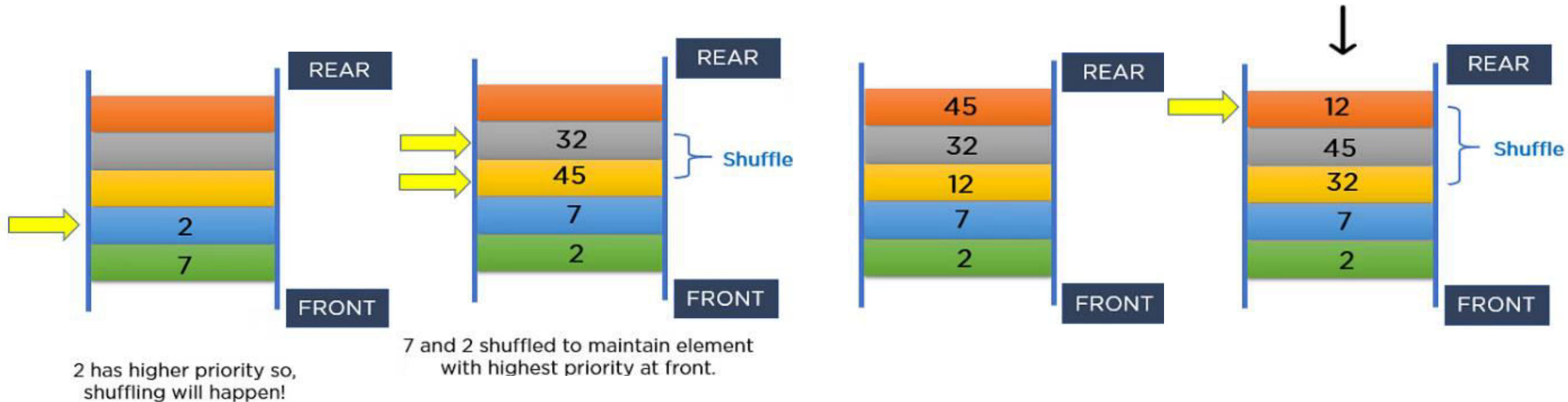
Characteristics

A queue is called a Priority Queue when it has the following characteristics:

- ▶ Each item in the queue must have a priority associated with it.
- ▶ Higher or lower priority elements must be dequeued before lower or higher priority elements respectively depending on priority order taken by user that is if user consider lower number as higher priority or higher number as higher priority.
- ▶ If two elements in the queue have the same priority value then the first in first out rule is followed for these two elements alone i.e. the element that entered the priority queue first will be the first to be removed.

Example

- Now, understand these properties with the help of an example. Consider you have to insert 7, 2, 45, 32, and 12 in a priority queue. The element with the least value has the highest property. Thus, you should maintain the lowest element at the front node.

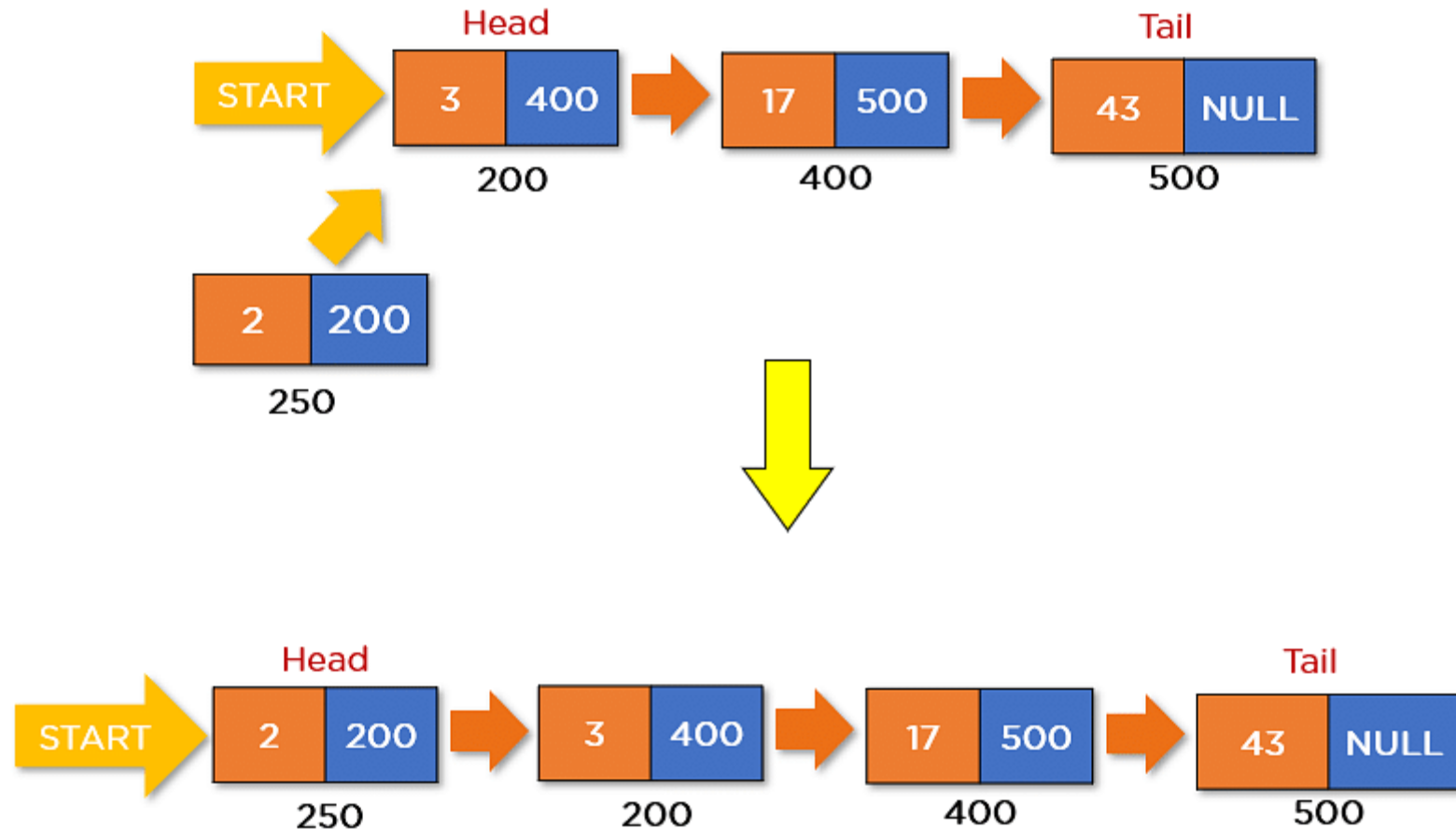


Representation of Priority Queue

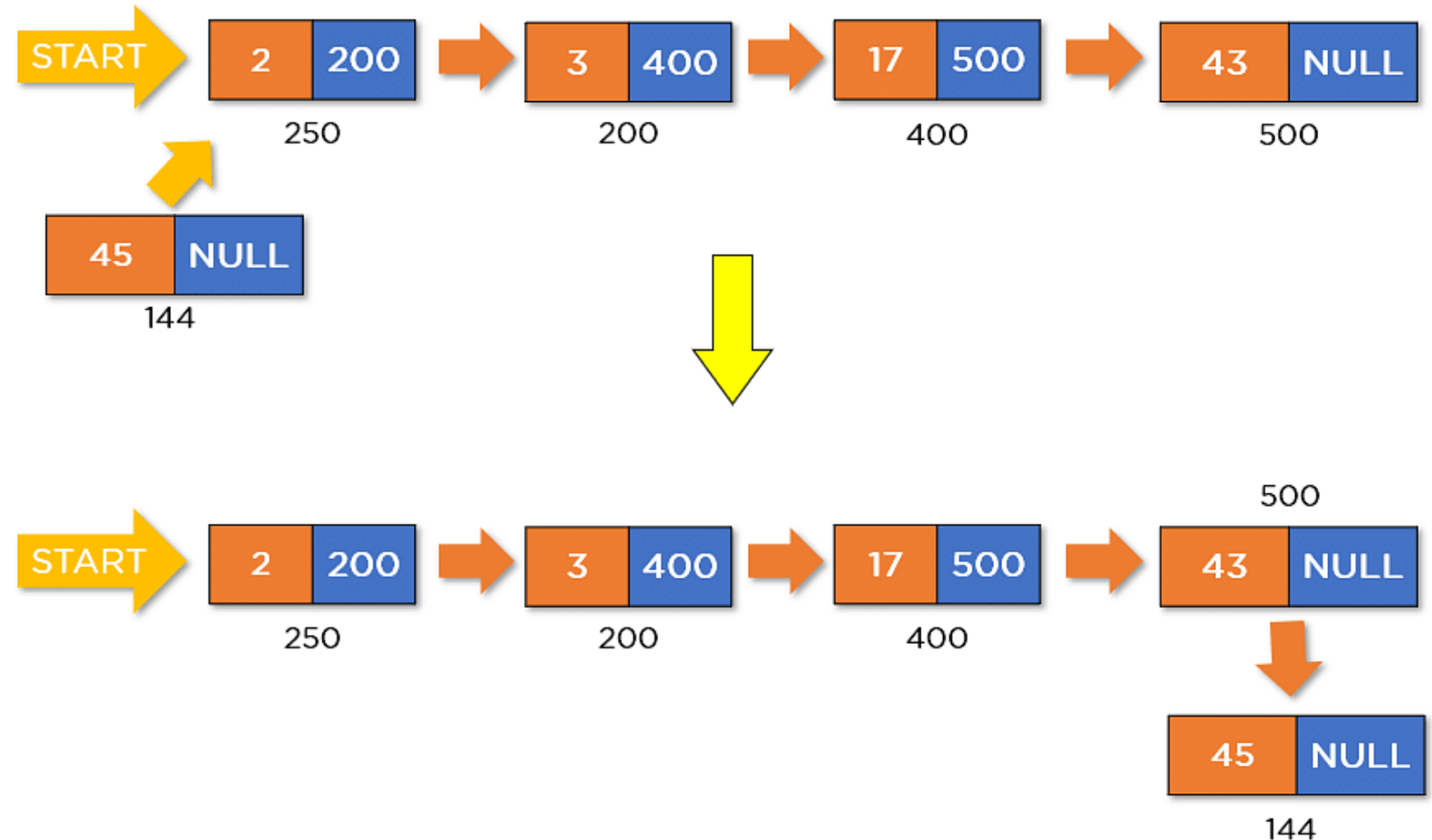
- ▶ You can also implement a priority queue using arrays, however, if you consider array implementation of the priority queue, then inserting elements into the sorted array will cost you $O(n)$. In general, processing each element will further cost you $O(n^2)$.
- ▶ Implementing a priority queue using arrays is not an ideal approach. Hence, we will see the representation of the priority queue using a linked list.

- Consider a linked queue having 3 data elements 3, 17, 43, respectively. It arranges all these elements according to priority. But, what if you have to insert a new node into the linked queue consisting of value 2

- ❑ Since 2 is smaller than the element at the front (head) node, insertion from the front will be more efficient.
- ❑ Additionally, it will allow you to have $O(1)$ time-complexity during deletion.



- For instance, say you want to insert a node consisting of element 45. Here, it will compare element 45 with each element inside the queue. However, this insertion will cost you $O(N)$. Representation of the linked queue below displays how it will insert element 45 in a priority queue.



Different Implementation Strategies for Priority Queue

- ▶ Three approaches implement a priority queue in data structure with a complexity less than $O(n^2)$. They are shown in the table below:

Implementation	Insert	Remove	Peek
Linked List	$O(N)$	$O(1)$	$O(N)$
Binary Heap	$O(\log n)$	$O(\log n)$	$O(1)$
Binary Tree	$O(\log n)$	$O(\log n)$	$O(1)$

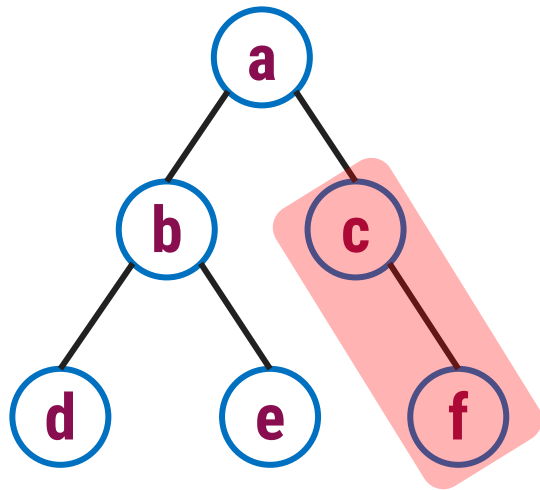
Heap

- ▶ A heap is a tree-like data structure that forms a complete tree and satisfies the heap invariant.
- ▶ The heap invariant states that, if A is a parent node of B, then A is ordered with respect to B for all nodes A and B in a heap.
- ▶ This means a parent node's value is always greater than or equal to the value of the child nodes for all nodes in a heap. Or the value of the parent node is less than or equal to the value of the child node for all nodes in a heap.

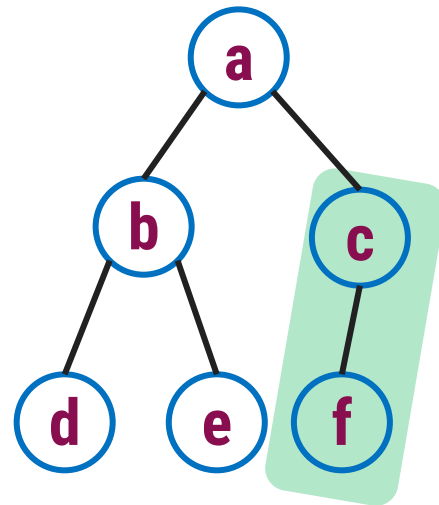
Introduction

► A heap data structure is a binary tree with the following two properties.

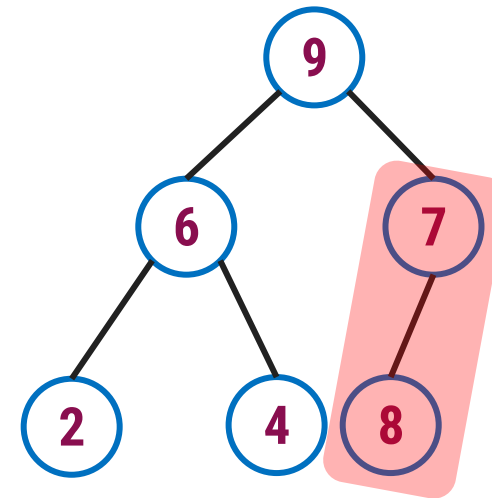
1. It is a complete binary tree: Each level of the tree is completely filled, except possibly the bottom level. At this level it is filled from left to right.
2. It satisfies the **heap order** property: the data item stored in each node is **greater than or equal** to the data item stored in **its children** node.



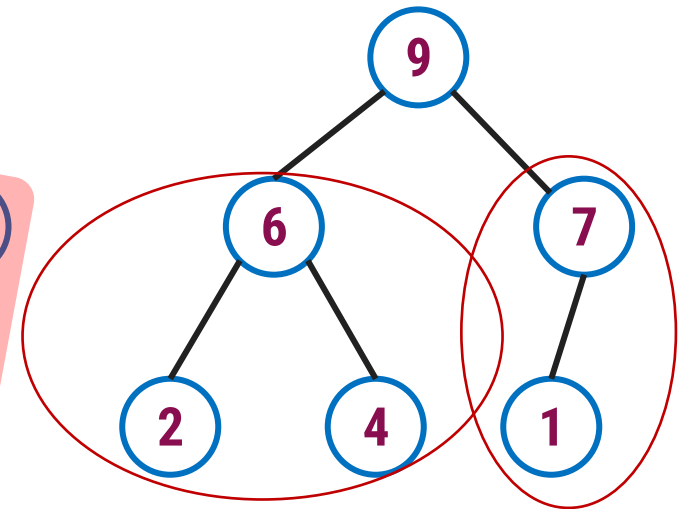
Binary Tree but not a Heap



Complete Binary Tree - Heap



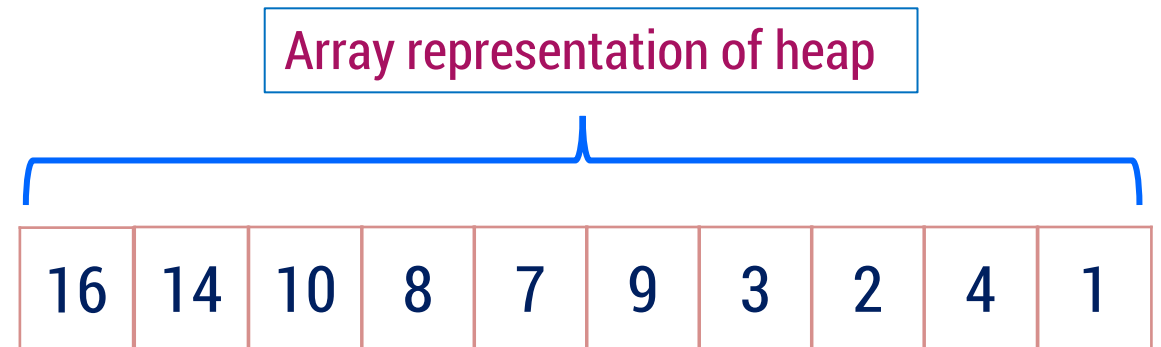
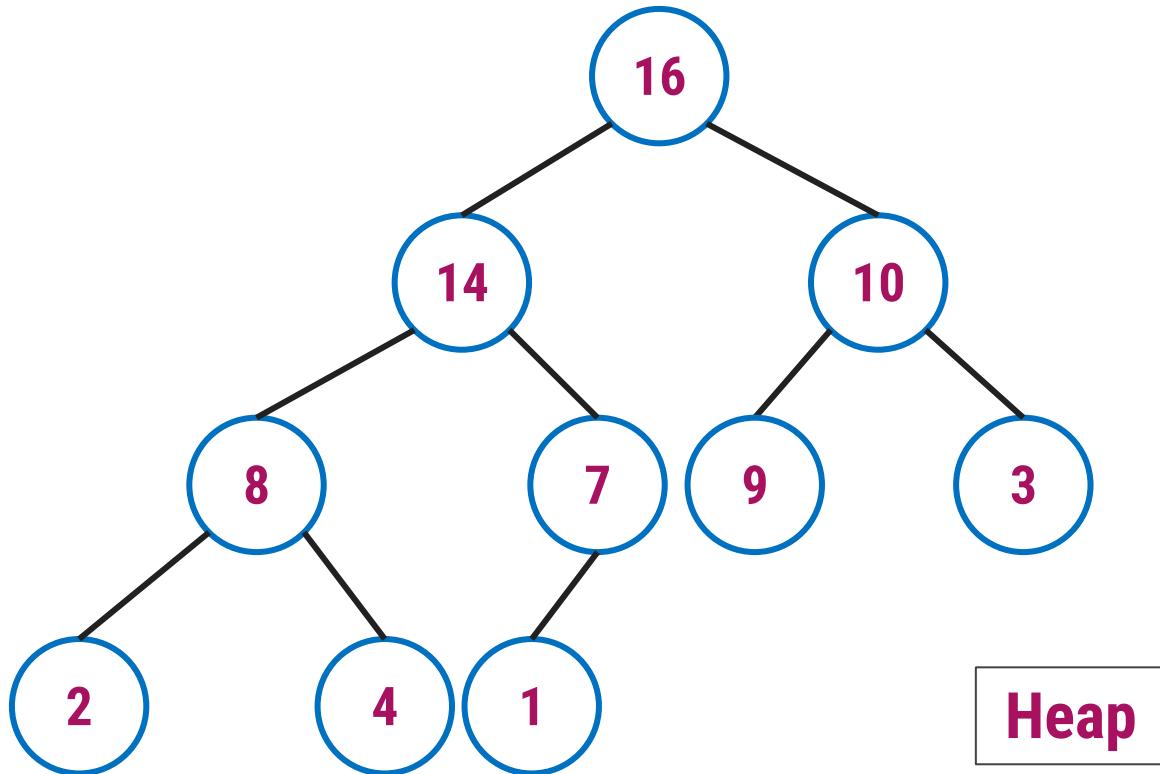
Not a Heap



Heap

Array Representation of Heap

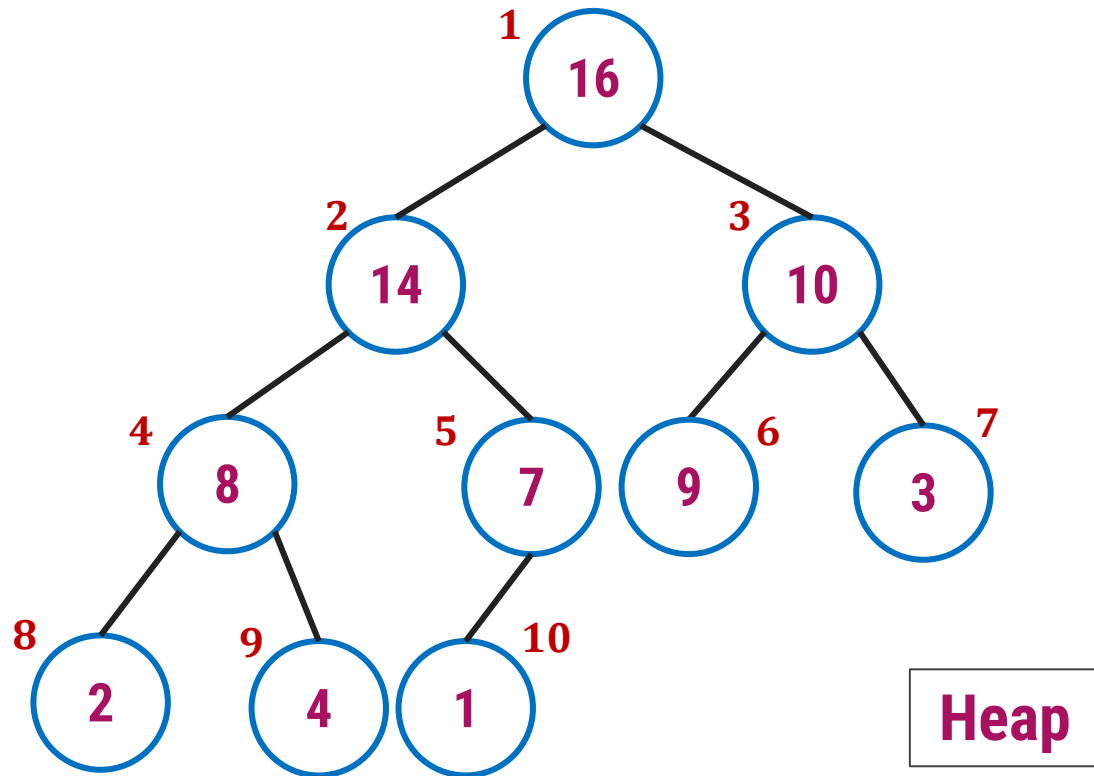
- ▶ Heap can be implemented using an Array.
- ▶ An array A that represents a heap is an object with two attributes:
 1. $length[A]$, which is the number of elements in the array, and
 2. $heap-size[A]$, the number of elements in the heap stored within array A



Array Representation of Heap

► In the array A , that represents a heap

1. $\text{length}[A] = \text{heap-size}[A]$
2. For any node i the parent node is $i/2$
3. For any node i , its left child is $2i$ and right child is $2i + 1$



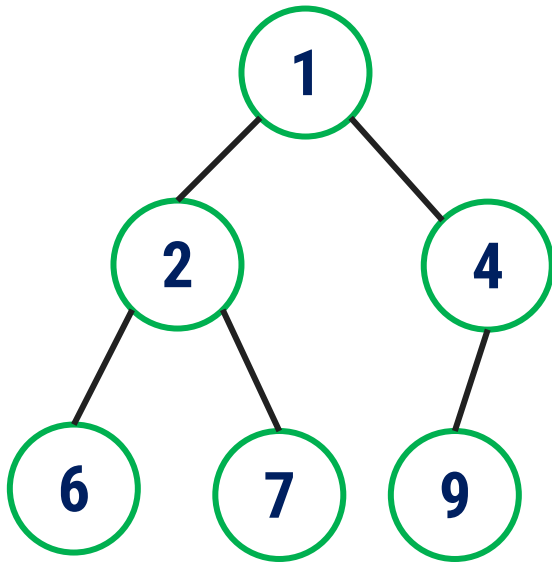
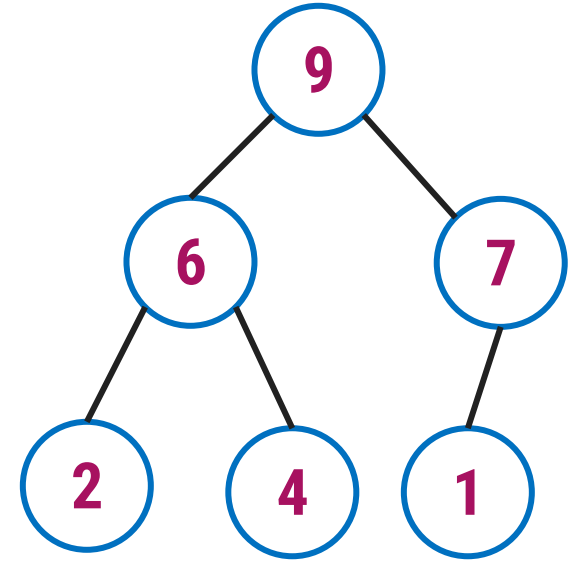
For node $i = 4$, parent node is $4/2 = 2$

For node $i = 4$,
Left child node is $2 * 4 = \text{node } 8$
Right child is $2 * 4 + 1 = \text{node } 9$

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Types of Heap

1. **Max-Heap** – Where the value of the root node is **greater than or equal to** either of its children.



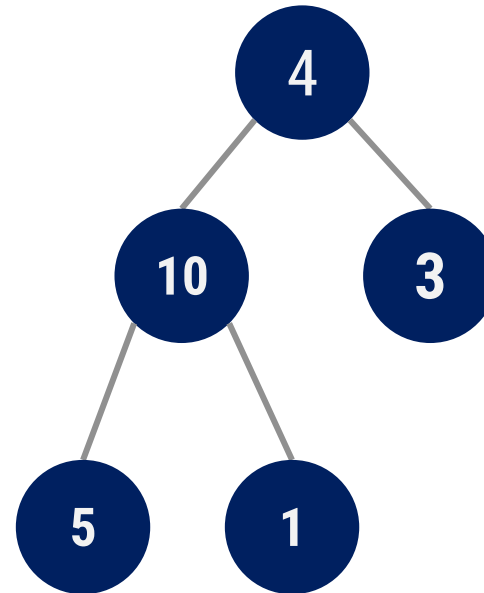
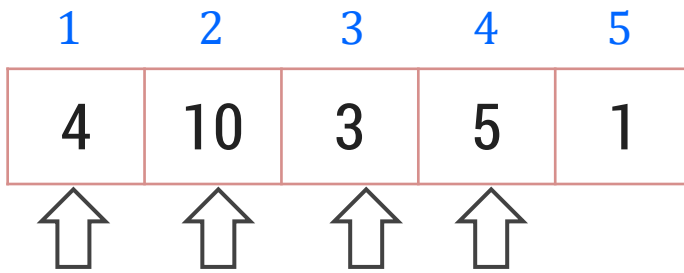
2. **Min-Heap** – Where the value of the root node is **less than or equal to** either of its children.

Heap – Example 1

Sort the following elements in Ascending order

4	10	3	5	1
---	----	---	---	---

Step 1 : Create Complete Binary Tree



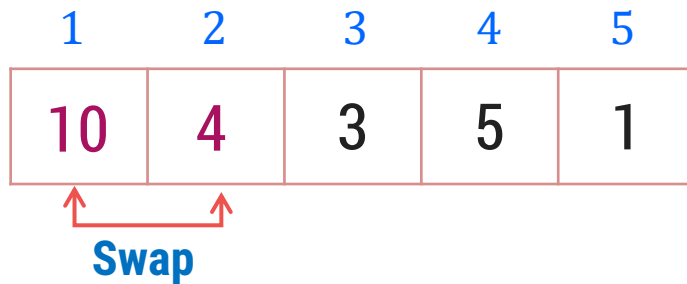
Now, a binary tree is created and we have to convert it into a Heap.

Heap- Example 1

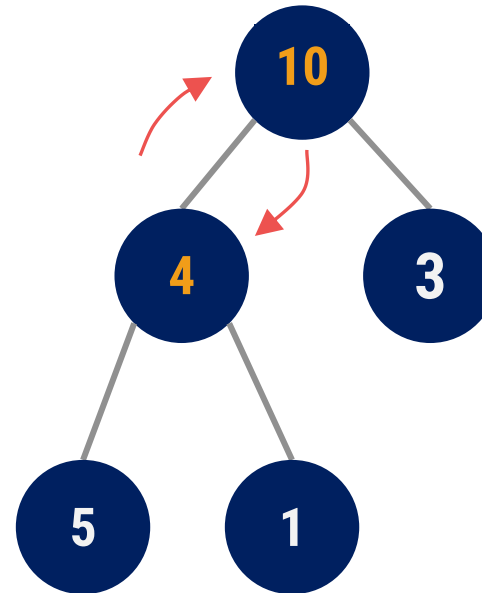
Sort the following elements in Ascending order

4	10	3	5	1
---	----	---	---	---

Step 2 : Create Max Heap



In a Max Heap, parent node is always greater than or equal to the child nodes.



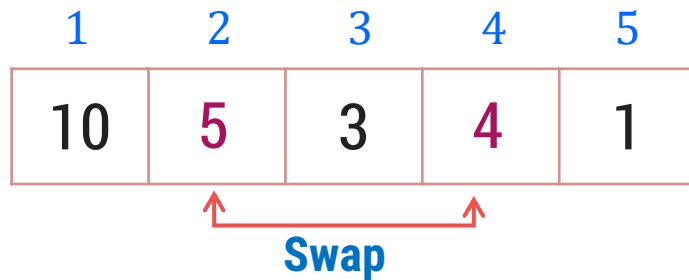
10 is greater than 4
So, swap 10 & 4

Heap – Example 1

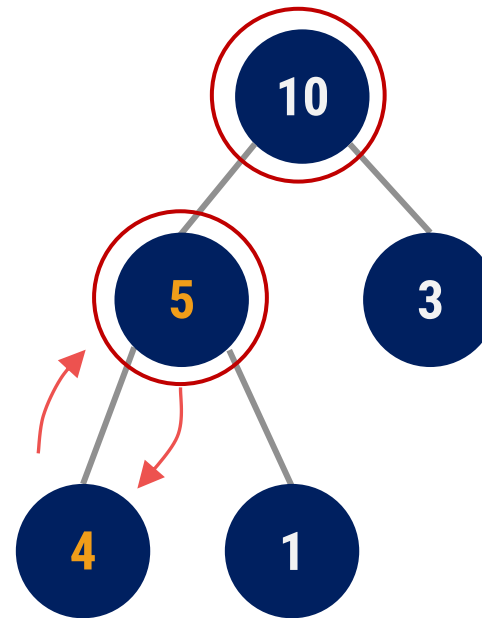
Sort the following elements in Ascending order

4	10	3	5	1
---	----	---	---	---

Step 2 : Create Max Heap



In a Max Heap, parent node is always greater than or equal to the child nodes.

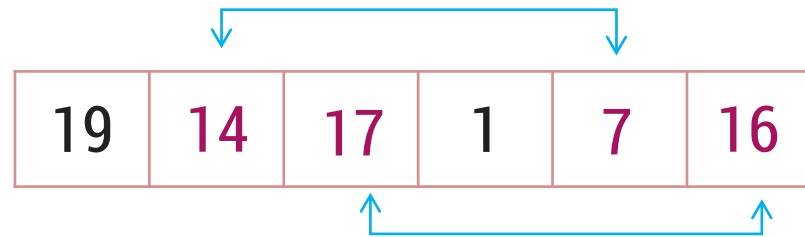


5 is greater than 4
So, swap 5 & 4

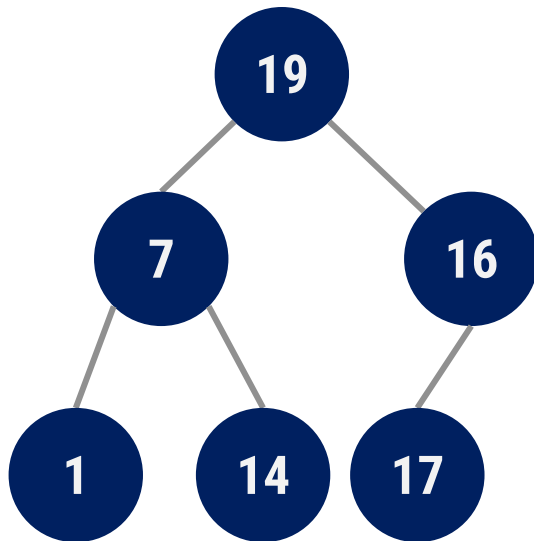
Max Heap is created

Heap – Example 2

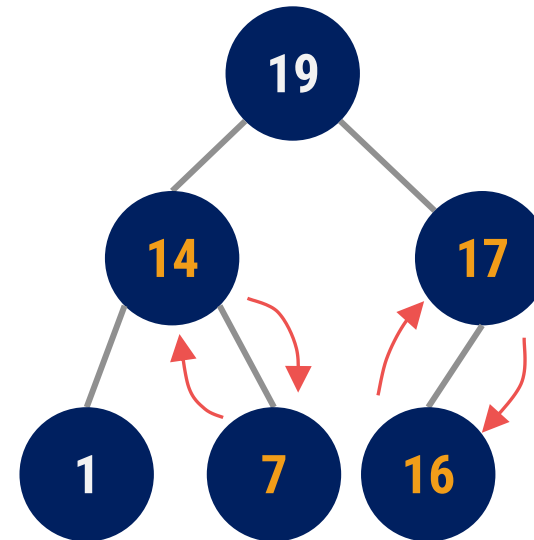
- Create Maxheap with the given element. 19, 7, 16, 1, 14, 17



Step 1: Create binary tree



Step 2: Create Max-heap

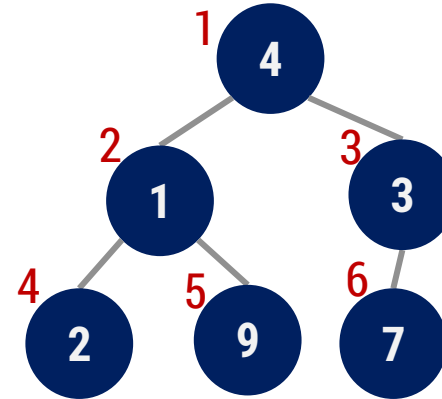


Build_Maxheap– Algorithm

```
Algorithm: BUILD-MAX-HEAP(A)
heap-size[A] ← length[A]
for i ← [length[A]/2] downto 1
  do MAX-HEAPIFY(A, i)
```

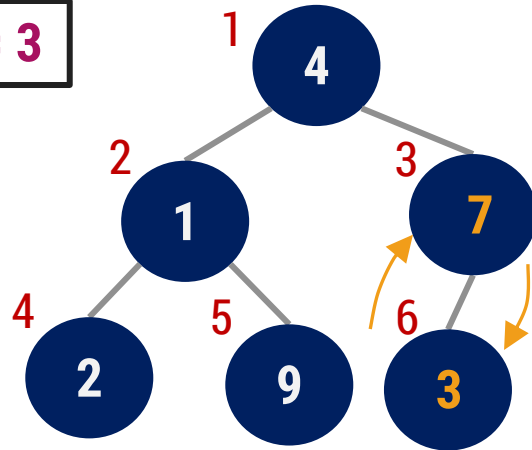
heap-size[A] = 6

4	1	3	2	9	7
---	---	---	---	---	---



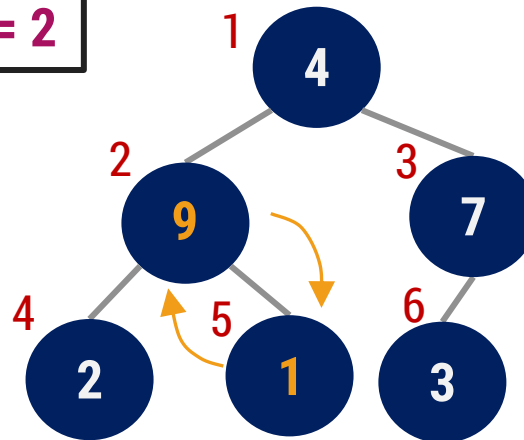
4	1	7	2	9	3
---	---	---	---	---	---

i = 3



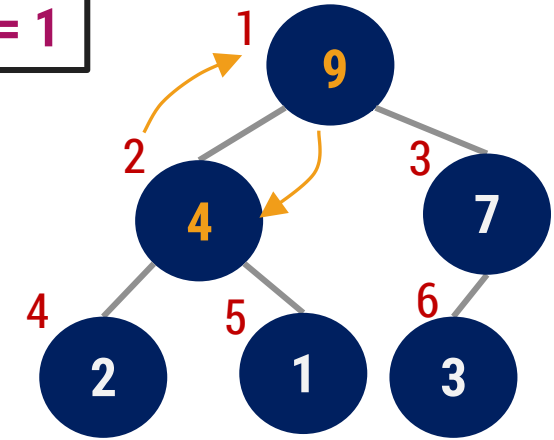
4	9	7	2	1	3
---	---	---	---	---	---

i = 2



9	4	7	2	1	3
---	---	---	---	---	---

i = 1



Max-heapify– Algorithm

Algorithm: Max-heapify(A, i, n)

$l \leftarrow \text{LEFT}(i)$

$l \leftarrow 2$

1

$r \leftarrow \text{RIGHT}(i)$

$r \leftarrow 3$

if $l \leq n$ and $A[l] > A[i]$

Yes

$\text{largest} \leftarrow 2$

then $\text{largest} \leftarrow l$

else $\text{largest} \leftarrow i$

Yes

if $r \leq n$ and $A[r] > A[\text{largest}]$

$\text{largest} \leftarrow 3$

then $\text{largest} \leftarrow r$

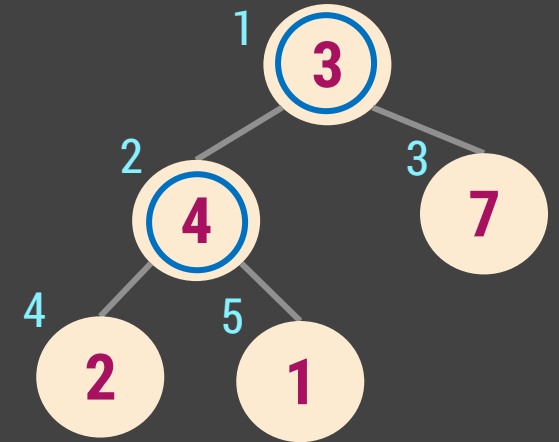
Yes

if $\text{largest} \neq i$

then exchange $A[i] \leftrightarrow A[\text{largest}]$

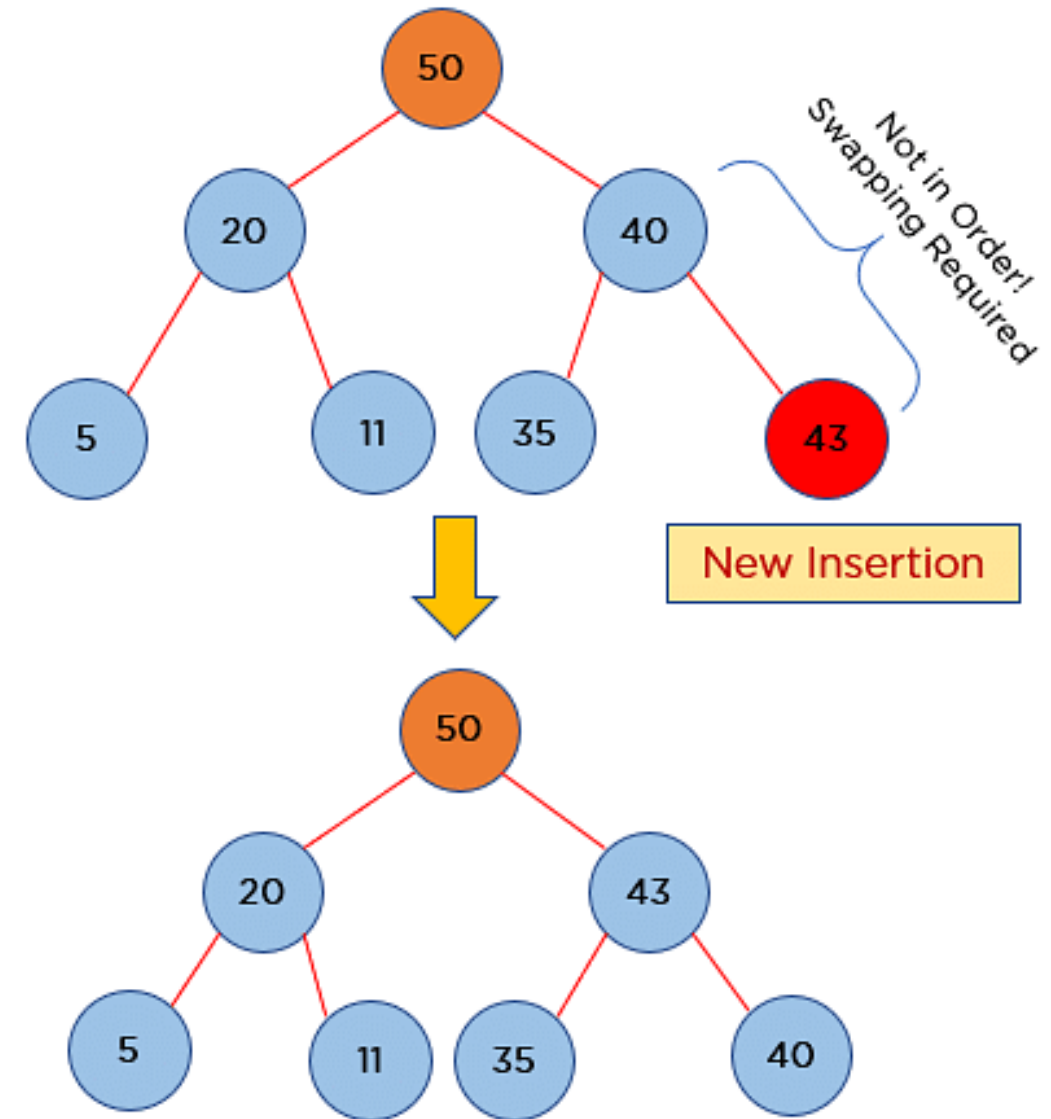
MAX-HEAPIFY($A, \text{largest}, n$)

3	4	7	2	1	9
---	---	---	---	---	---



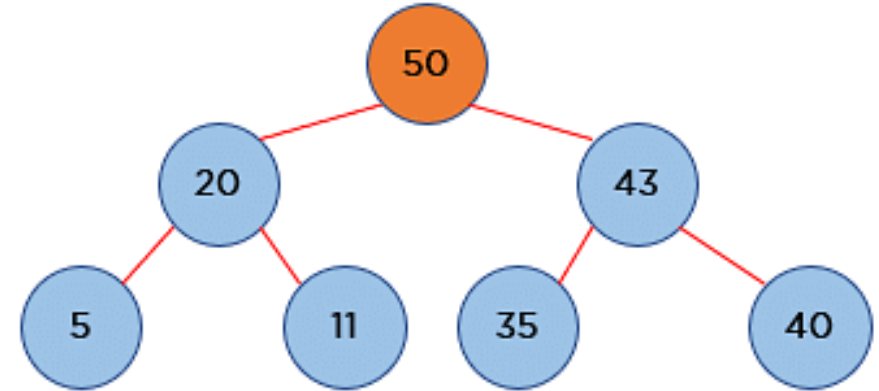
Priority Queue Operations: Using Heap

- ▶ **Inserting** the Element in a Priority Queue:
Once you perform the new insertion, the new element will move to the empty space from top to bottom and left to right.
- ▶ Additionally, if the element is not in the correct position, it will compare it with the parent node.
- ▶ Following that, if the element is not in proper order, then it swaps the elements. The swapping process continues until all the elements inside the queue are in the correct positions.

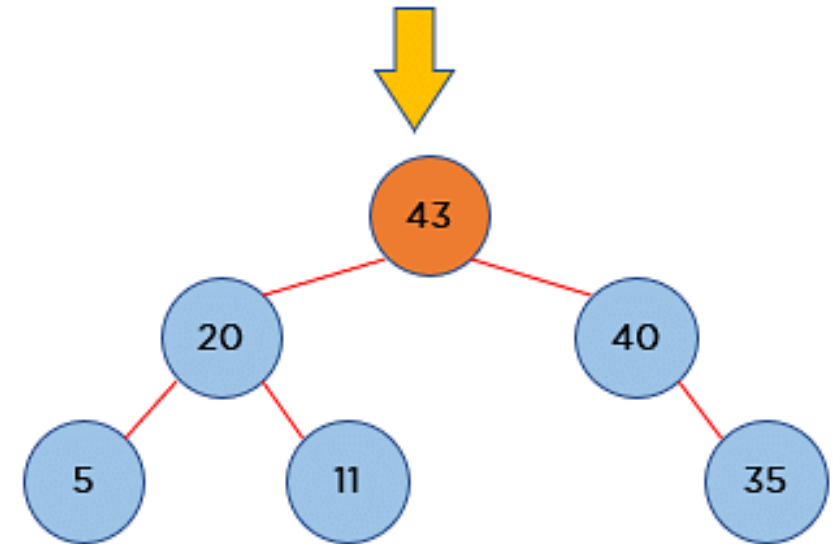


Priority Queue Operations: Using Heap

- ▶ **Deletion** in Priority Queue: As you know that in a max heap, the maximum element is the root node. And it will remove the element which has maximum priority first.
- ▶ Thus, you remove the root node from the queue. This removal creates an empty slot, which will be further filled with new insertion.
- ▶ Then, it compares the newly inserted element with all the elements inside the queue to maintain the heap invariant.



The deletion of element will happen from root node as it has higher priority. After that, Using Heapify the Heap property will be maintained.



Exercise

► Create max heap with the following elements

→ 65, 77, 5, 23, 32, 45, 99, 83, 69, 81

→ 20, 50, 30, 75, 90, 65, 25, 10, 40